ECC504 - AI&ML

We consider a **deep learning classifier model** that can be thought of as a **nonlinear stacked tensor transformer model with dense layer functions**.

- **Input**: A feature vector of dimension **7** in the embedding space.
- **Layer 1**: Dense layer with 10 neurons (ReLU activation).
- **Layer 2**: Dense layer with 6 neurons (ReLU activation).
- **Layer 3 (Output)**: Dense layer with 4 neurons (Softmax activation, for 4-class classification).

1. Compute the number of trainable parameters for each layer.
2. Find the **total number of trainable parameters** in the model.
3. **Draw a picture** of the neural network structure showing input, each layer, and the output layer with dimensions.
4. If we **add another dense layer with 5 neurons before the last layer**, how many additional parameters will be introduced?

1. **Parameters per layer**

- **Layer 1 (7 → 10):** $7 \times 10 + 10 = 70 + 10 = \mathbf{80}$
- **Layer 2 (10 → 6):** $10 \times 6 + 6 = 60 + 6 = \mathbf{66}$
- **Layer 3 / Output (6 → 4):** $6 \times 4 + 4 = 24 + 4 = \mathbf{28}$

2. **Total parameters**

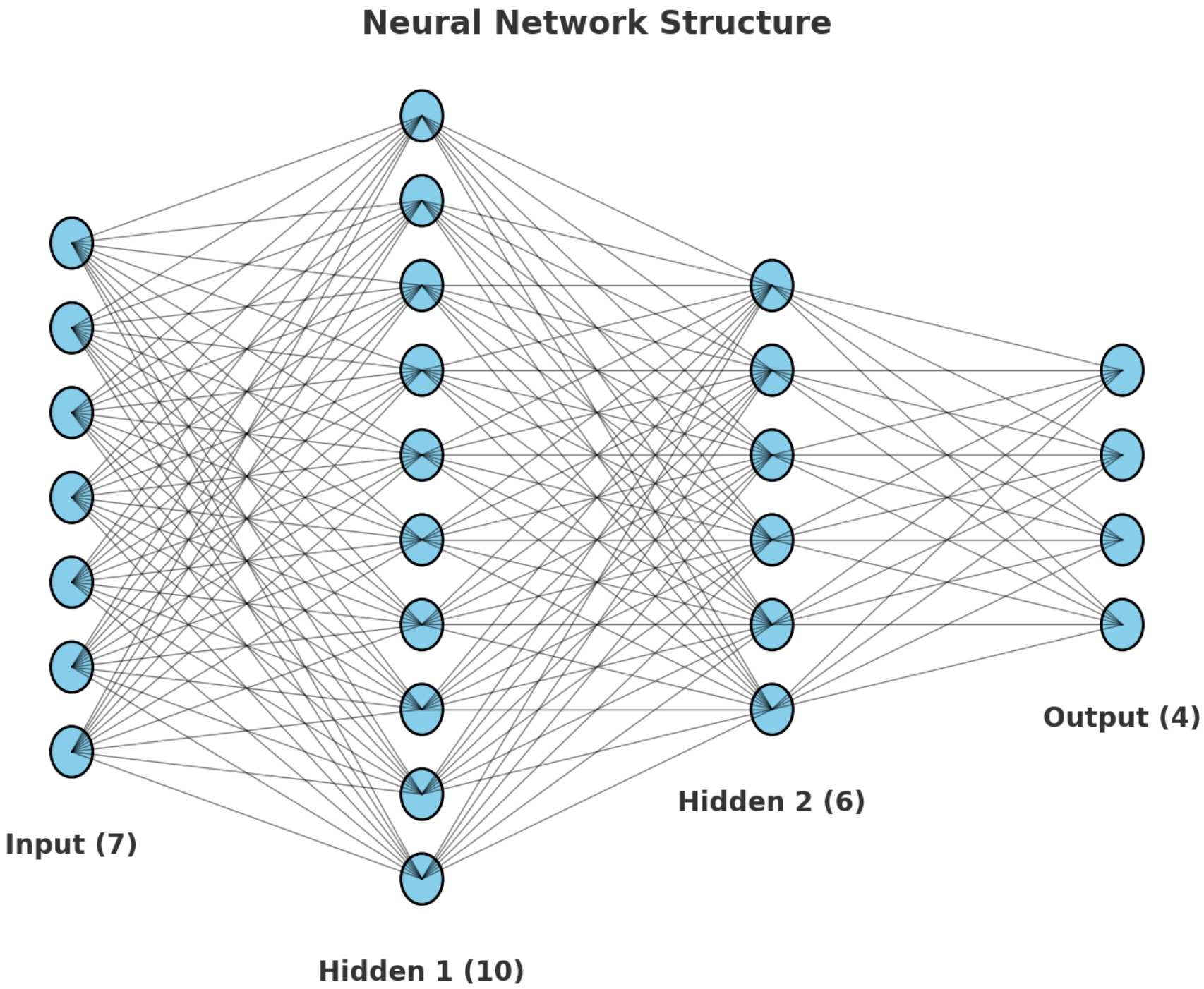
   $$\mathbf{80 + 66 + 28 = 174}$$

Add a new layer with 5 neurons before the last layer

- New layer (6 → 5): $6 \times 5 + 5 = 30 + 5 = \mathbf{35}$
- Output layer now (5 → 4): $5 \times 4 + 4 = 20 + 4 = \mathbf{24}$ (was 28)

Net increase in parameters: $35 - (28 - 24) = 35 - 4 = \mathbf{31}$.

(Equivalently: new total $= 174 + 31 = \mathbf{205}$.)


Neural Network Structure
Input (7)
Hidden 1 (10)
Hidden 2 (6)
Output (4)

“In machine learning, our goal is not just to fit the training data, but to generalize well to unseen data.”

…but why is generalization difficult?

Two types of errors that affect generalization

Bias

Variance

Another factor: Irreducible factor (noise) → Beyond our control

What is Bias?

- Error from overly simplistic assumptions in the model
- Effect: Model misses important patterns (underfitting)

What is Variance?

- Error from excessive sensitivity to training data
- Effect: Model captures noise as if it were signal (overfitting)

Let’s express mathematically…

Bias–Variance Decomposition

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2}_{\text{systematic error}} + \underbrace{\text{Variance}}_{\text{model sensitivity}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

- Bias²: Error due to wrong assumptions
- Variance: Error due to model instability
- Noise (σ²): Error we cannot remove

Generalization Error

$$Err(x) = \left(E[\hat{f}(x)] - f(x)\right)^2 + E\left[\left(\hat{f}(x) - E[\hat{f}(x)]\right)^2\right] + \sigma_e^2$$

$$Err(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

$$\text{Bias}(x) = \left(E[\hat{f}\,(x)] - f(x)\right)$$

f(x): the true (but unknown) function.

- $\hat{f}(x)$: the model trained on a dataset.
- $\mathbb{E}[\hat{f}(x)]$: the *average prediction* if you could train the model on infinitely many datasets.

👉 So **bias = how far the model's "average guess" is from the truth.**

$$\text{Variance}(x) = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]$$

- $\hat{f}(x)$: prediction of the model trained on a random dataset.
- $\mathbb{E}[\hat{f}(x)]$: the *average* prediction across many datasets.
- Variance measures how **spread out** the predictions are around their mean.

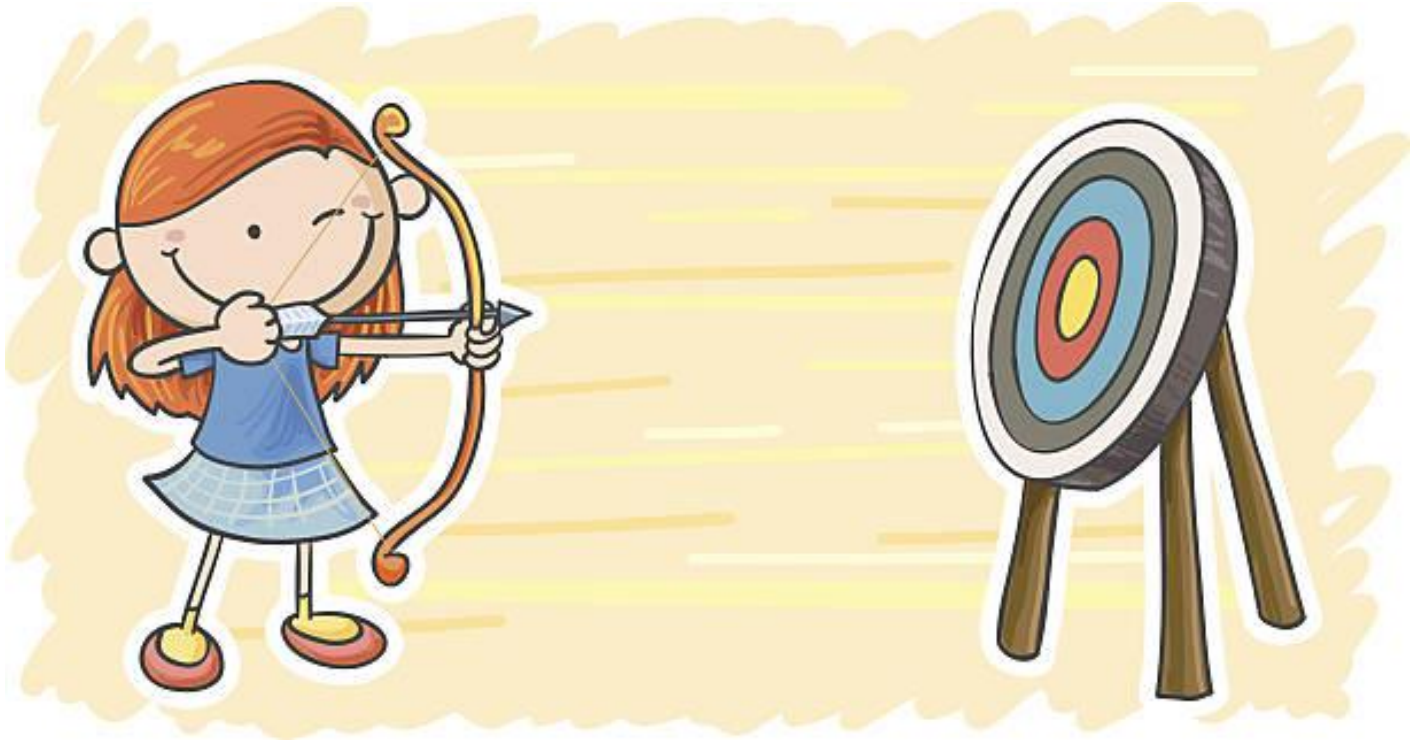
👉 In plain words: **variance = how much the model's predictions jump around depending on the specific training data it saw.**

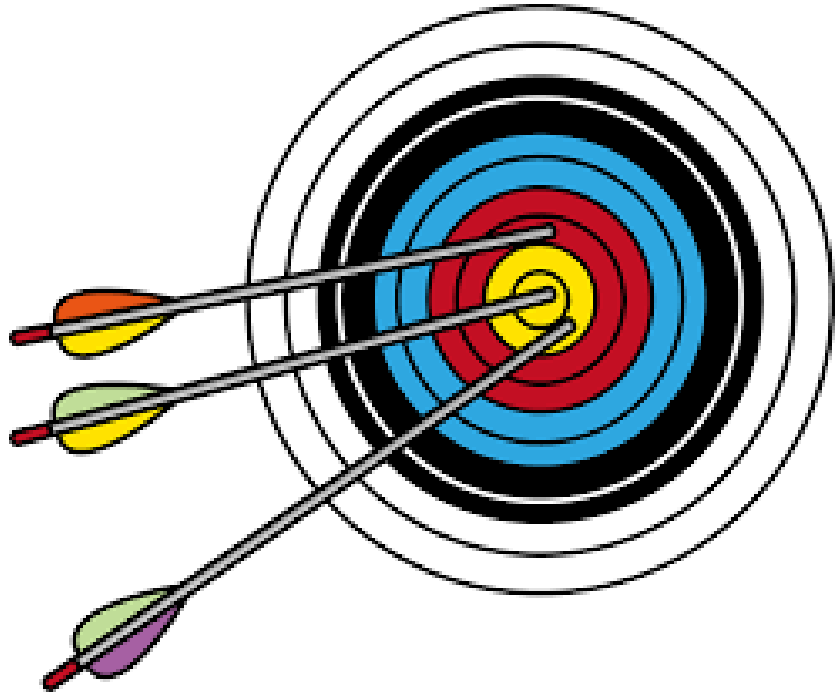
- Bias = average prediction vs truth → *systematic error*.
- Variance = predictions vs their own mean → *instability due to data sensitivity*.

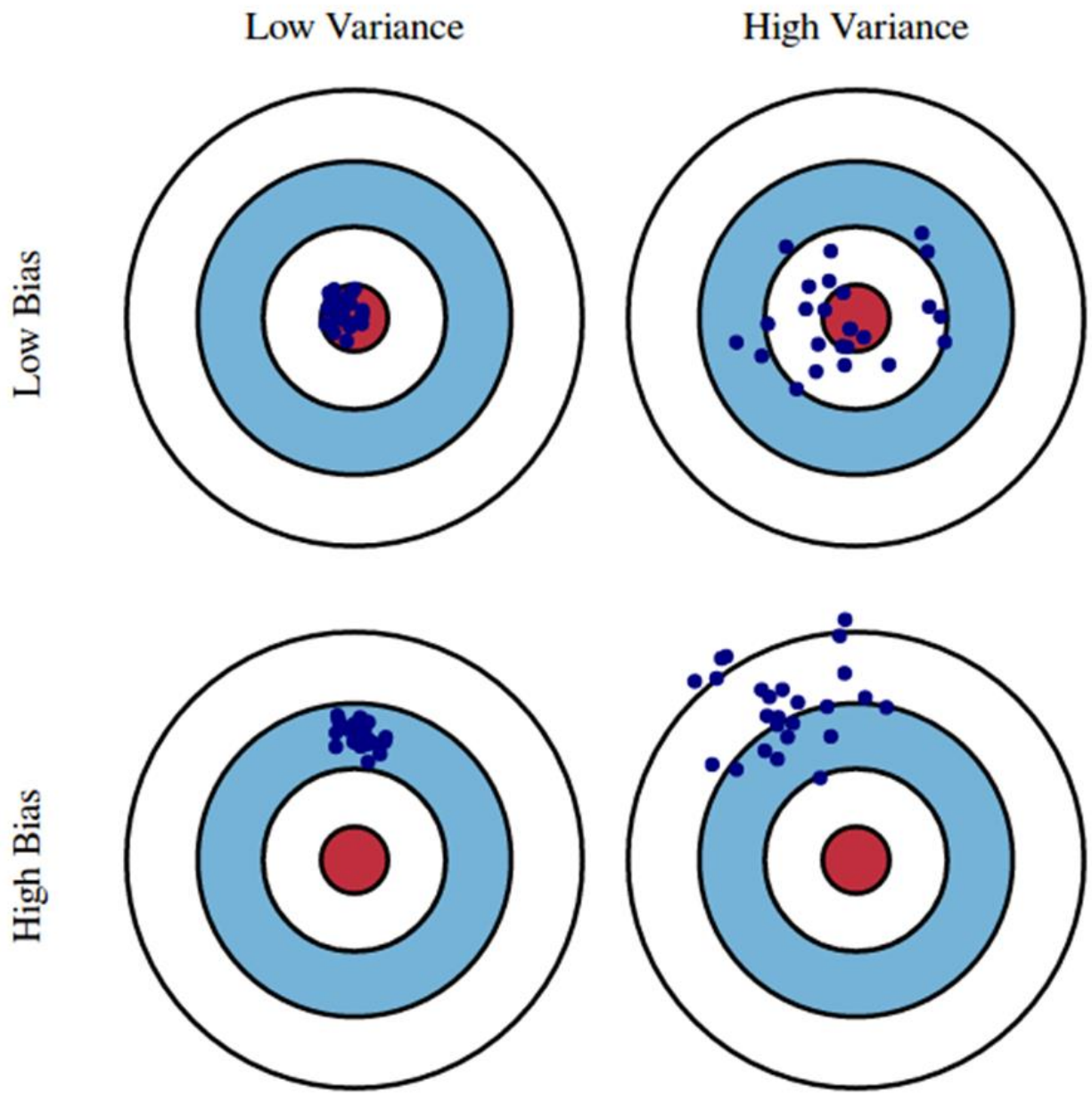
🎯 Archery Target Analogy

Imagine a dartboard or target. Each arrow = a trained model's prediction.

- **Bias** → how far the **average arrow location** is from the bullseye (the truth).
- **Variance** → how spread out the arrows are around their center.






Low Variance
High Variance
Low Bias
High Bias

👉 Four cases you can draw on slides:

- **High bias, low variance**: All arrows tightly clustered but far from the bullseye (systematically wrong).
- **Low bias, high variance**: Arrows centered on the bullseye but scattered widely (inconsistent).
- **High bias, high variance**: Arrows scattered, and the average is far from the bullseye (worst case).
- **Low bias, low variance**: Arrows tightly clustered around the bullseye (best case).

Bias-Variance Analysis

"Theoretically, can we state which factors bias and variance depend on?"

"How do we assess the bias–variance trade-off of a trained model through experiments?"

"What strategies can be applied to handle high bias and high variance cases?"

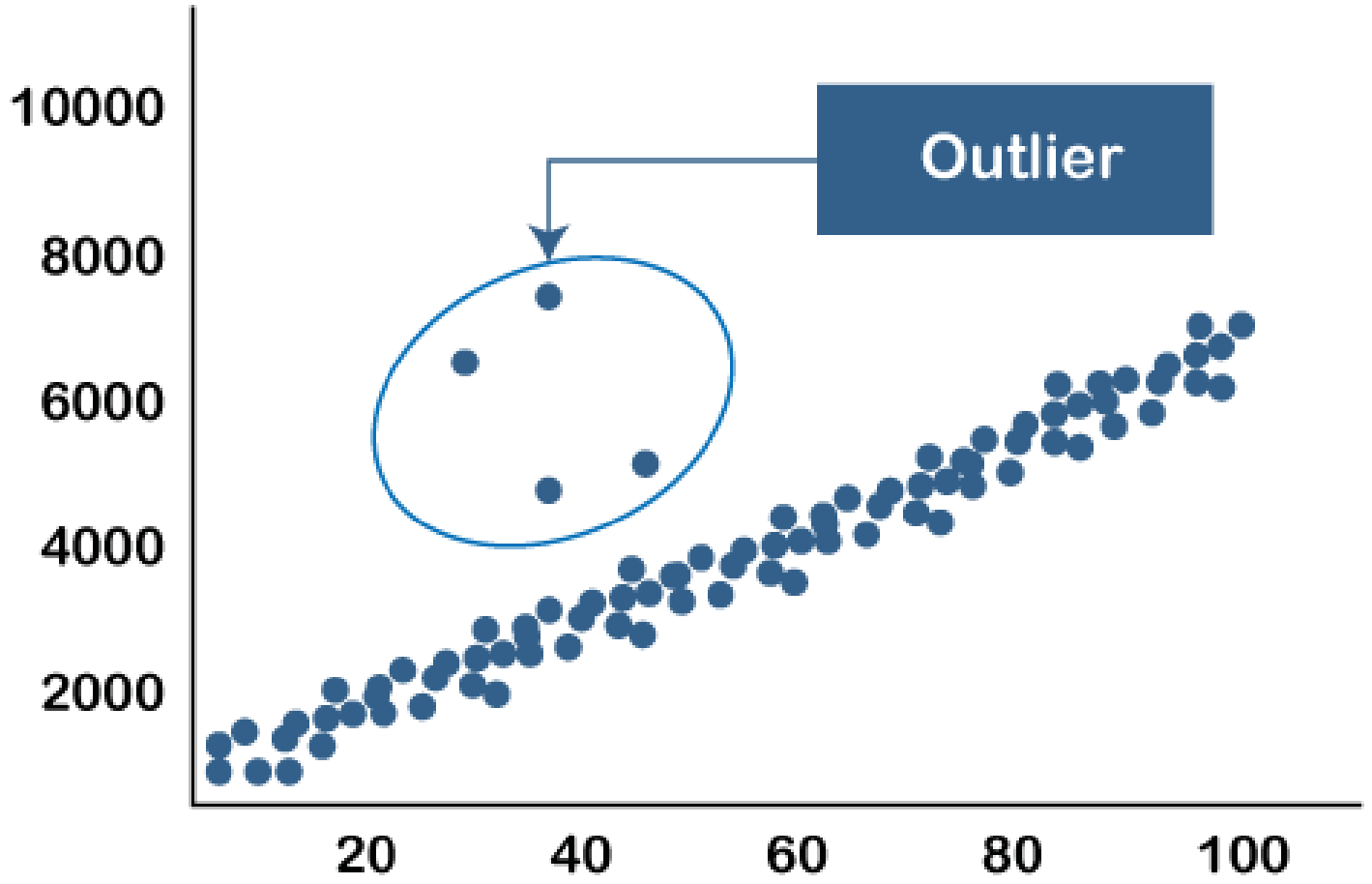
"Theoretically, can we state which factors bias and variance depend on?"

Bias

- Model selection
- Model complexity
- Feature representation
- Optimization algorithm quality
- Hyperparameters
- Amount and Quality of Data

Variance

- Model complexity
- Training data size
- Noise in data (outliers)
- Hyperparameters (related to learning algorithm or optimization)
- Feature dimensionality


10000
8000
6000
4000
2000
Outlier
20
40
60
80
100

- **Shallow ML = good when data is small, structured, and interpretability matters.**
- **Deep Learning = best when data is large, unstructured, and you want automatic feature learning.**

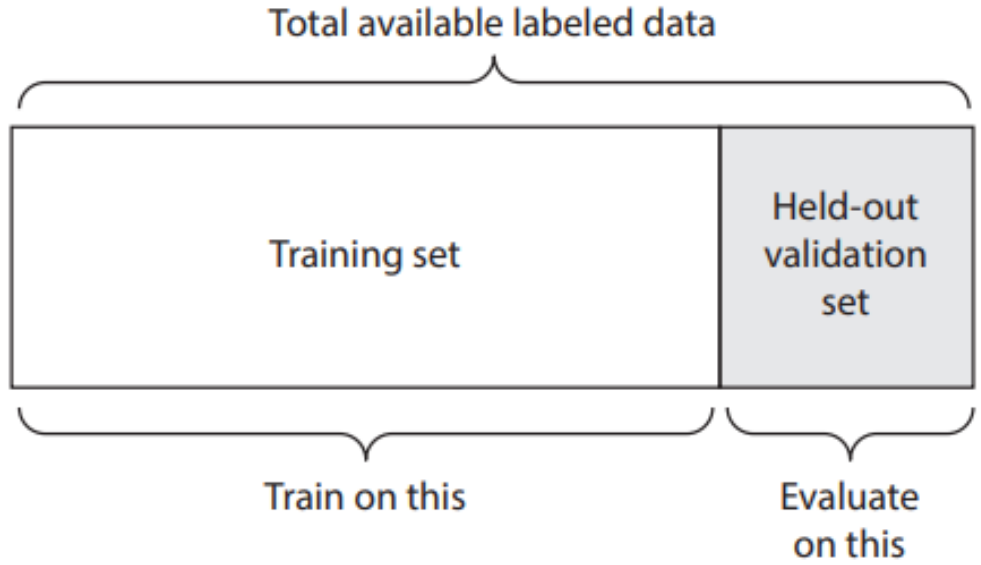
1. Held-Out Validation

👉 **Best for**: Large datasets (millions of samples)

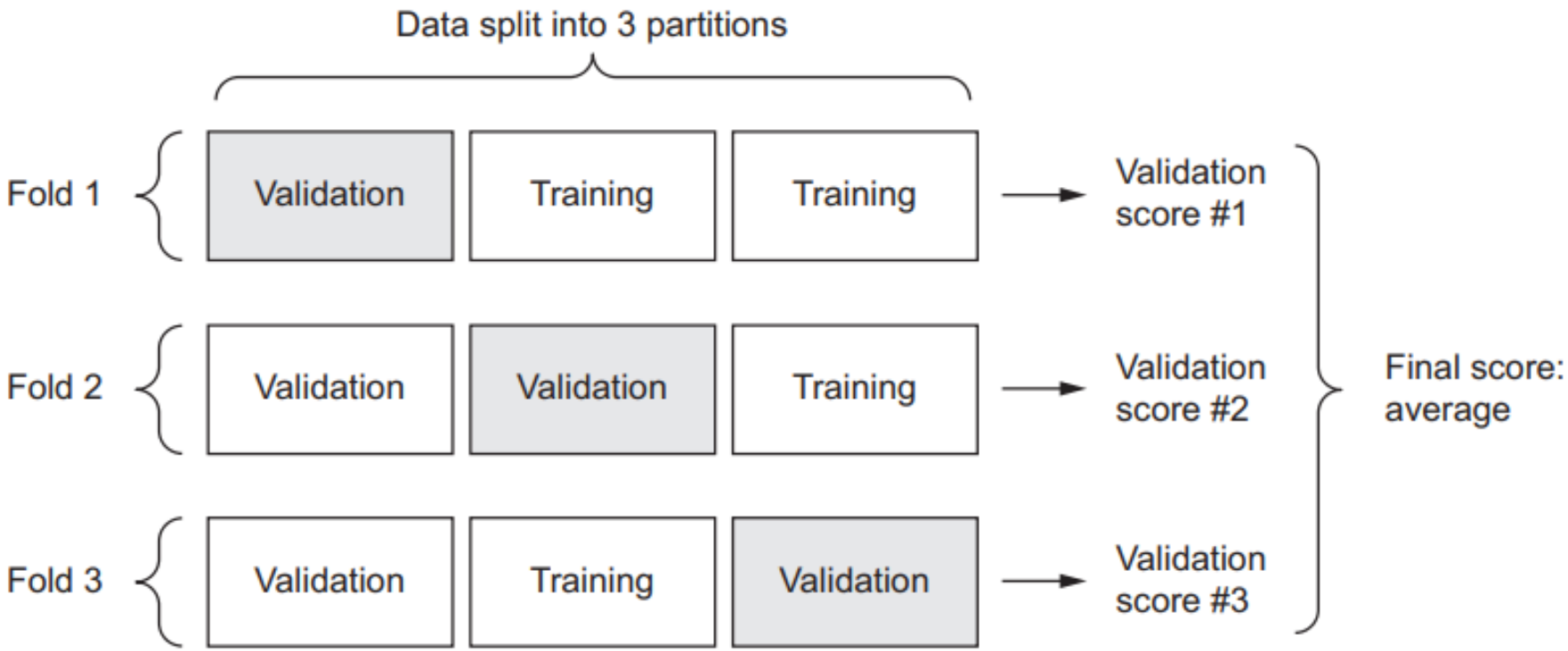
2. k-Fold Cross-Validation

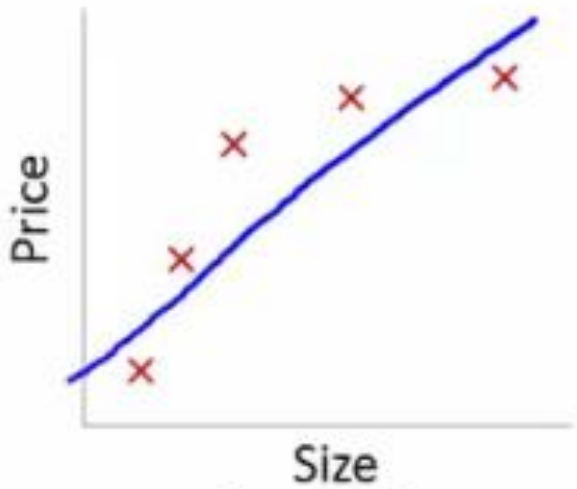
👉 **Best for**: Small or medium-sized datasets, especially in classical ML

1. Held-Out Validation

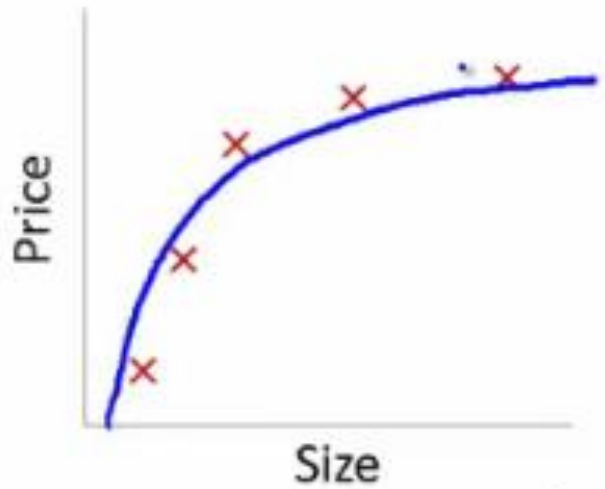


2. k-Fold Cross-Validation

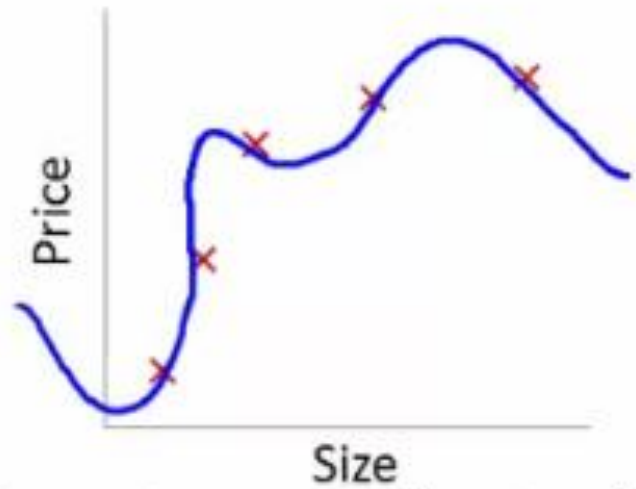




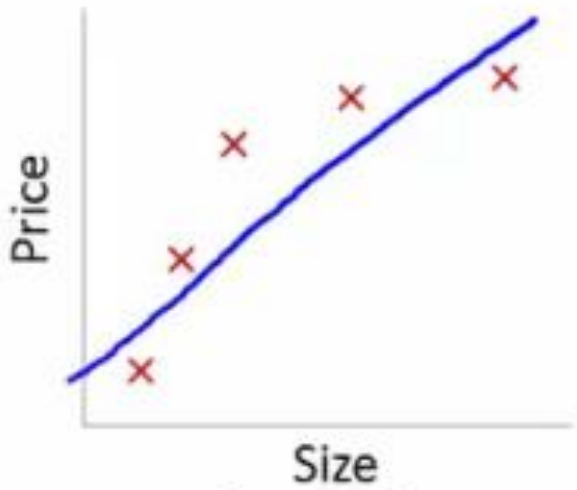
$$\theta_0 + \theta_1 x$$



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

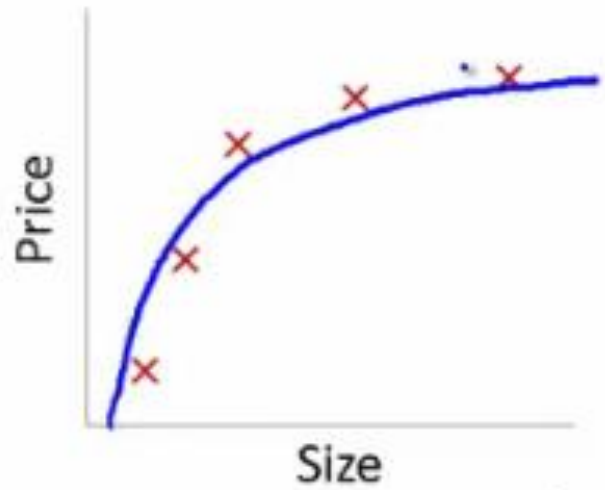


$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$



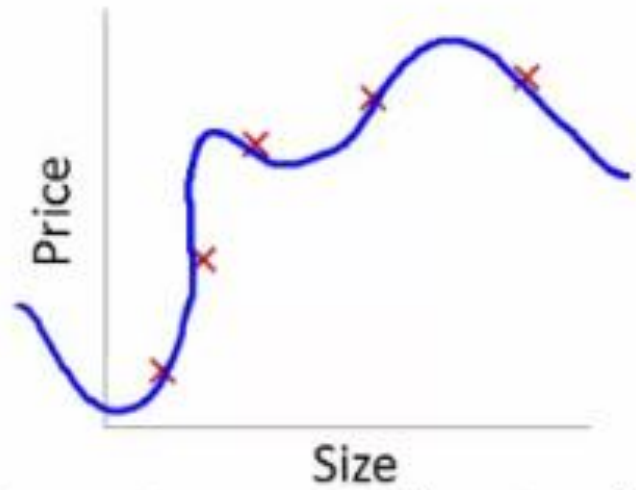
$\theta_0 + \theta_1 x$

High bias
(underfit)



$\theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

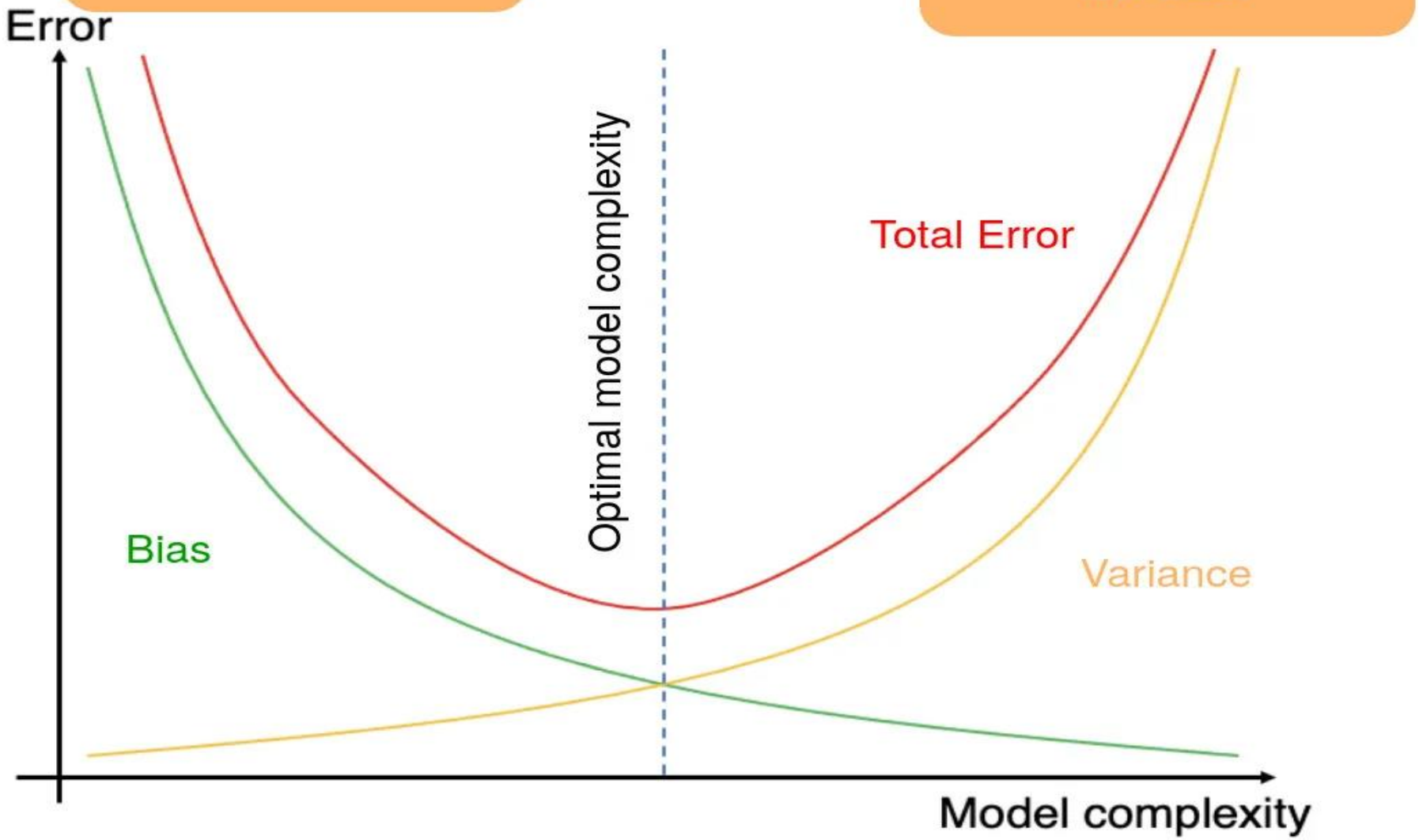


$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

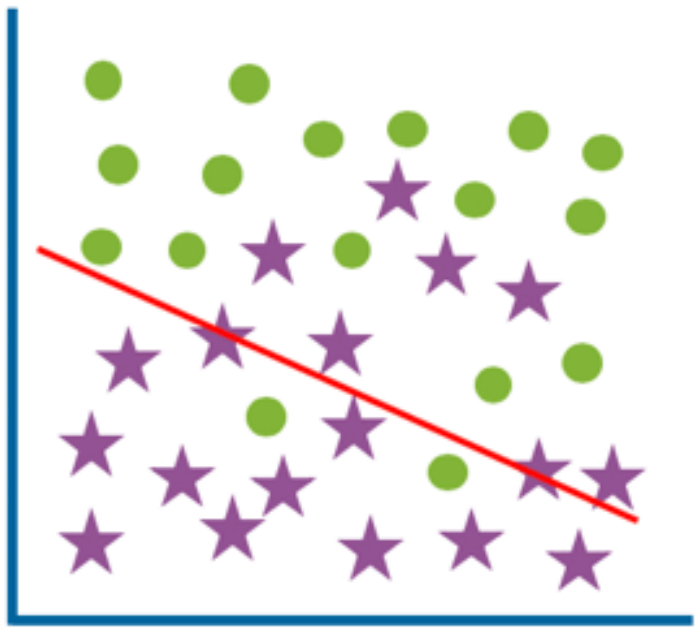
High variance
(overfit)

Underfitting:
- Low variance and high bias

Overfitting:
- High variance and low bias.

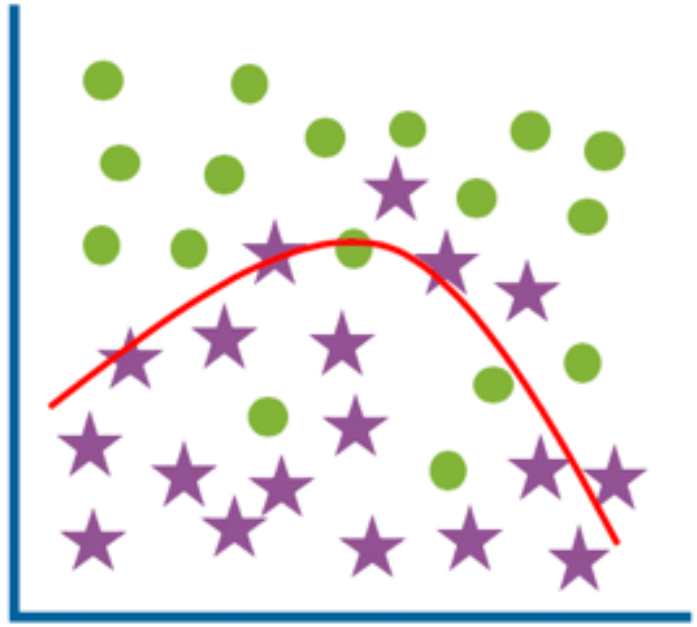


Underfit
(high bias)



High training error
High test error

Optimum



Low training error
Low test error

Overfit
(high variance)

Low training error
High test error

Anatomy of a neural network

- Architecture (layer)
- Input data presentation
- Loss function
- Optimizer

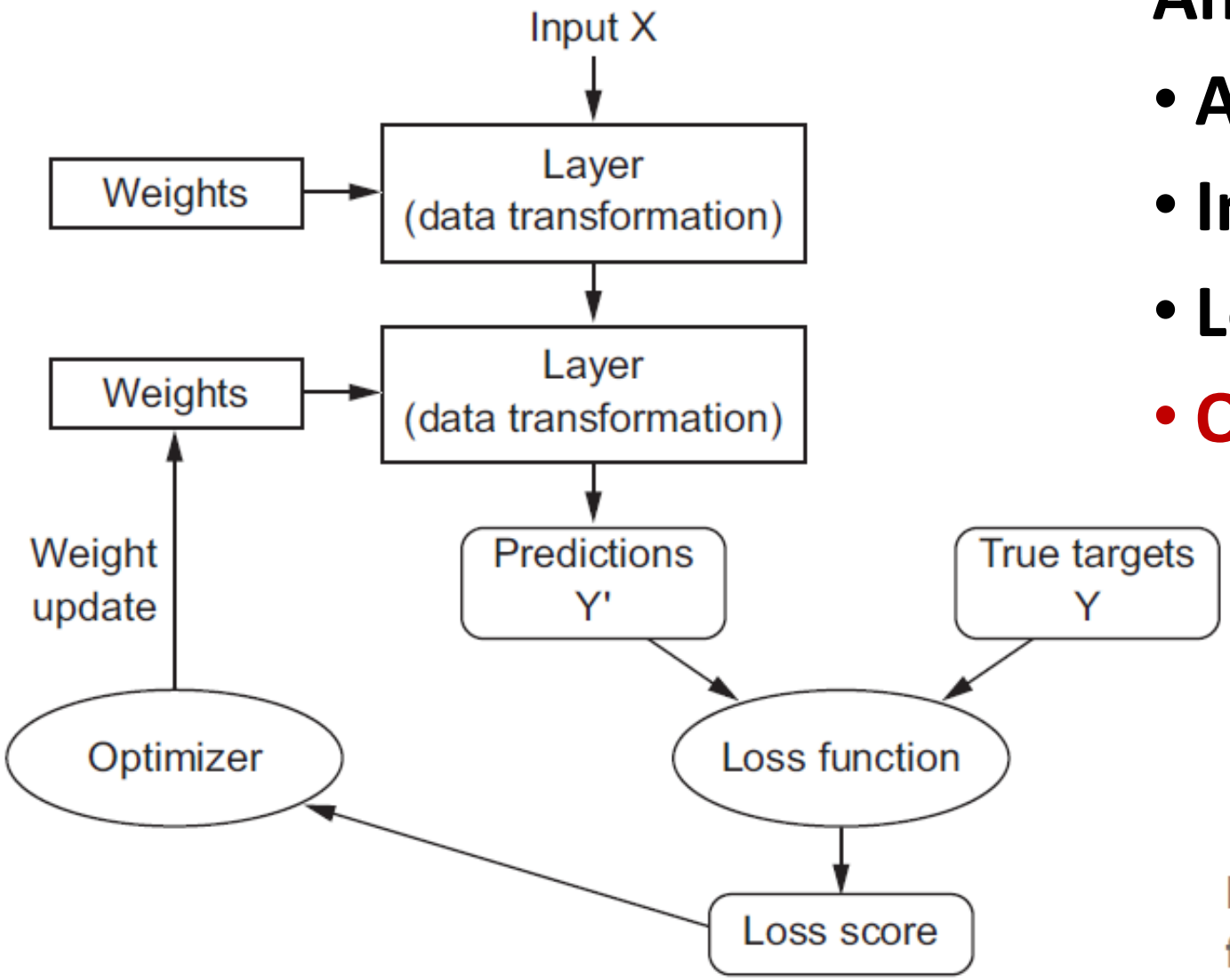
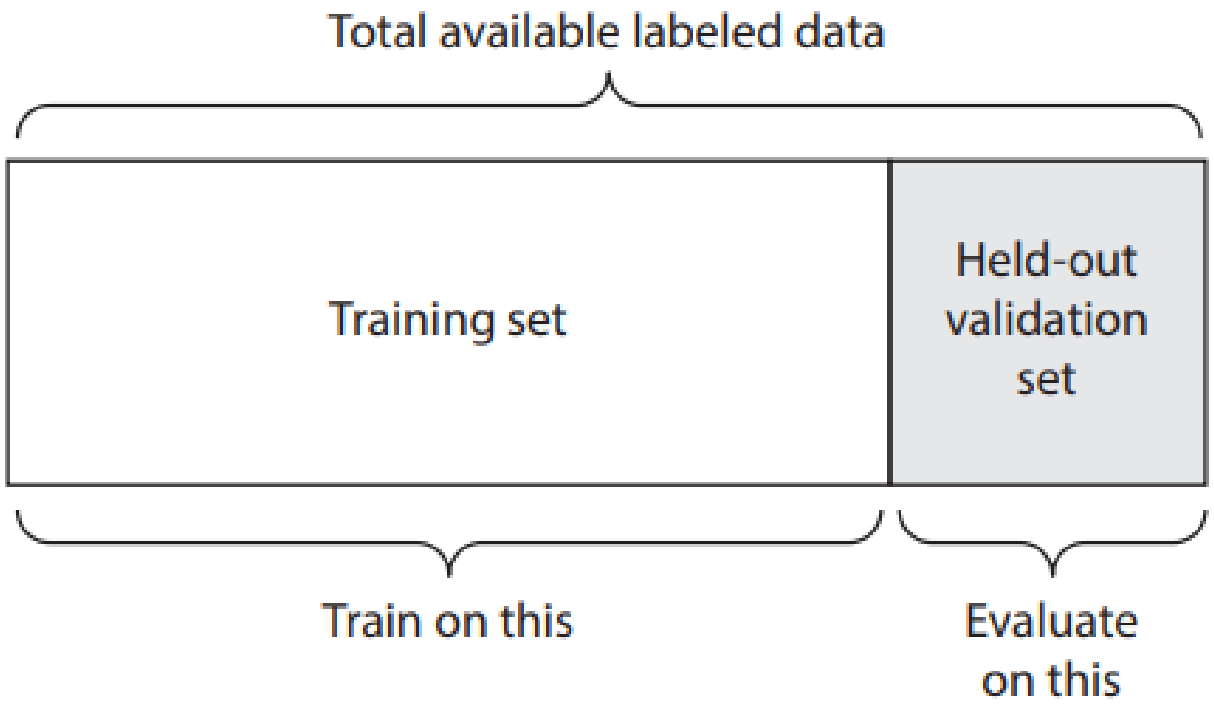


Figure 1.9 The loss score is used as a feedback signal to adjust the weights.

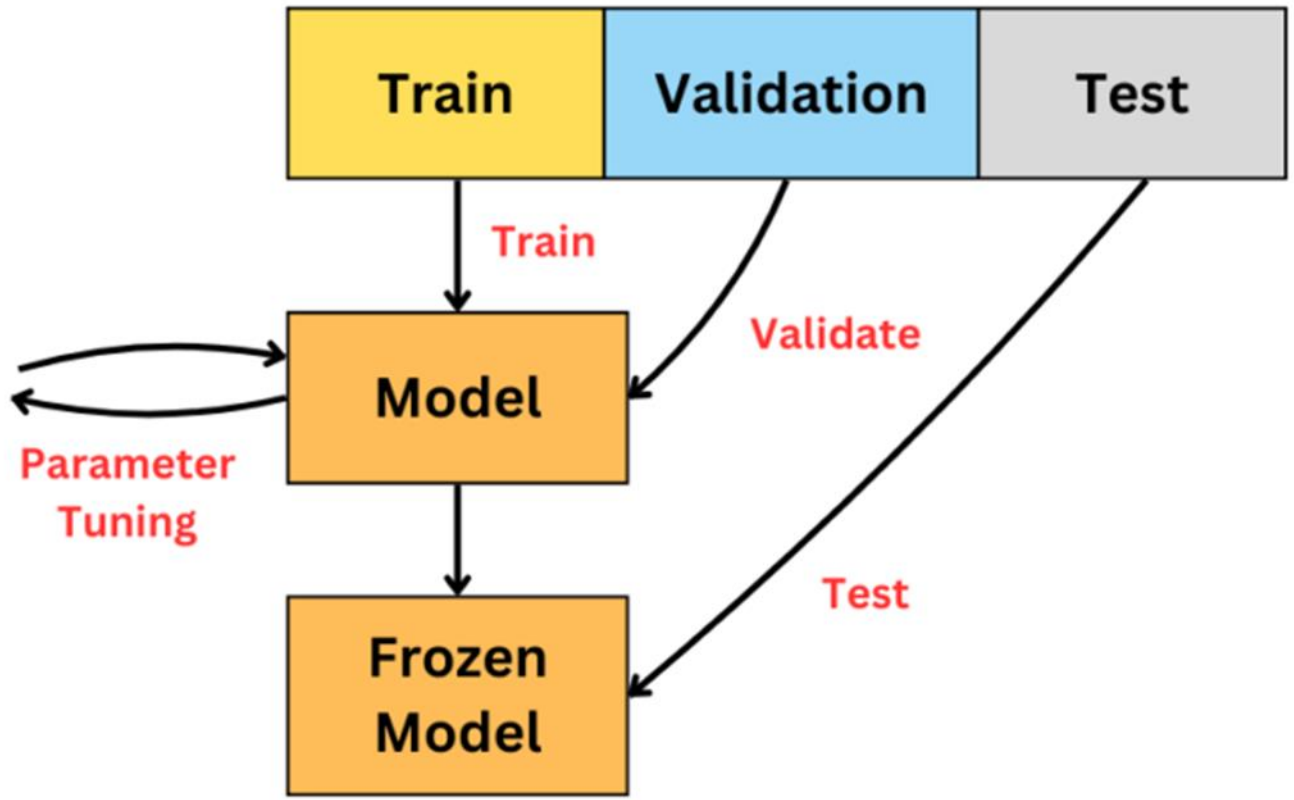
1. Held-Out Validation



Dataset Split: Train, Validation and Test Set

Hyperparameters

- no. training iterations
- no. of layers (depth)
- no. of hidden units (width)
- optimization algorithm
- learning rate
- batch size
- epochs
- .....



Steps to train deep learning model

1. Set network architecture using hyperparameters
2. Weight and bias (parameters) initialization
3. Forward pass all the data points
4. Calculate the total error in prediction using a **loss function** on both **train and val data**
5. Use **the optimization method** to update the parameters (w and b).
6. Repeat step 2 to step 4 until the convergence

How to Diagnose Bias & Variance from Errors

- During training, we store/save the total error(loss) on train and validation data for every epoch/iteration, and then we can plot it wrt epoch progressively

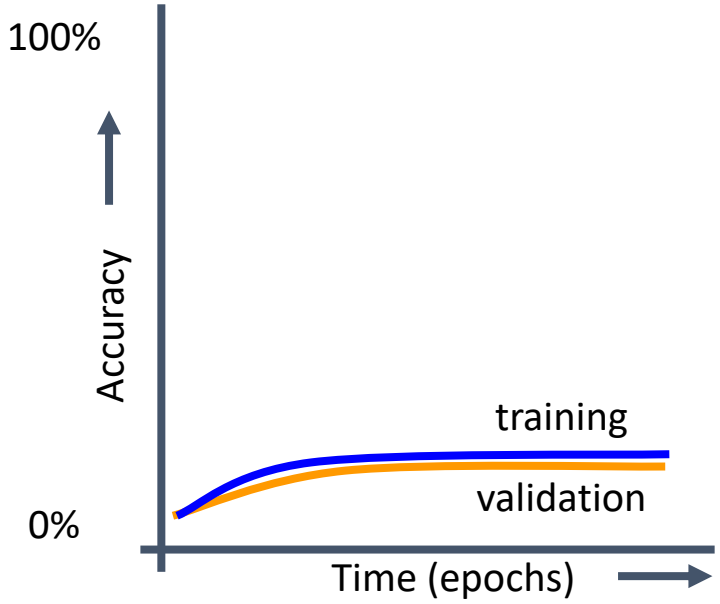
→ **Training history plot.**

- **The train vs validation error gap is the main diagnostic tool.**
- **Instead of error/loss, we can also look at performance (accuracy in case of classification) gap.**

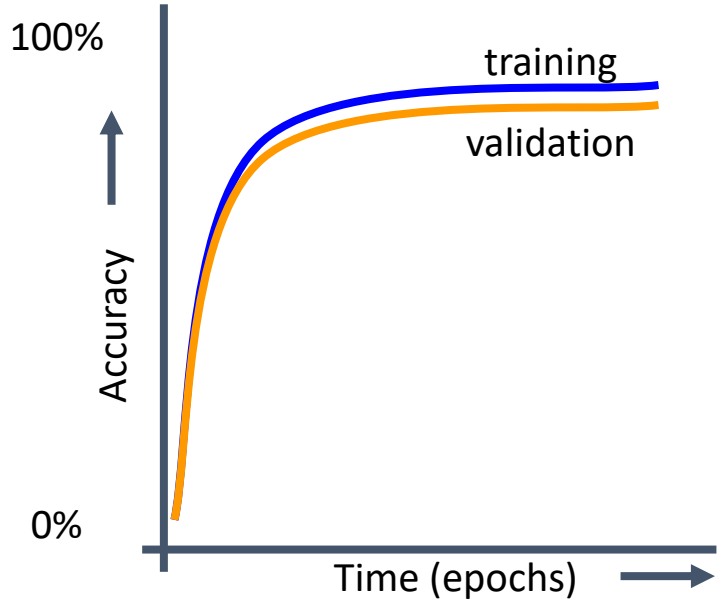
How to Diagnose Bias & Variance from Errors

Case	Training Error	Validation Error
High Bias (Underfitting)	High	High (similar to train error)
High Variance (Overfitting)	Low	High (much higher than train error)
Good Fit (Balanced)	Low	Low (close to each other)
Extreme Case	Very low train error, validation error diverges over epochs	Severe overfitting

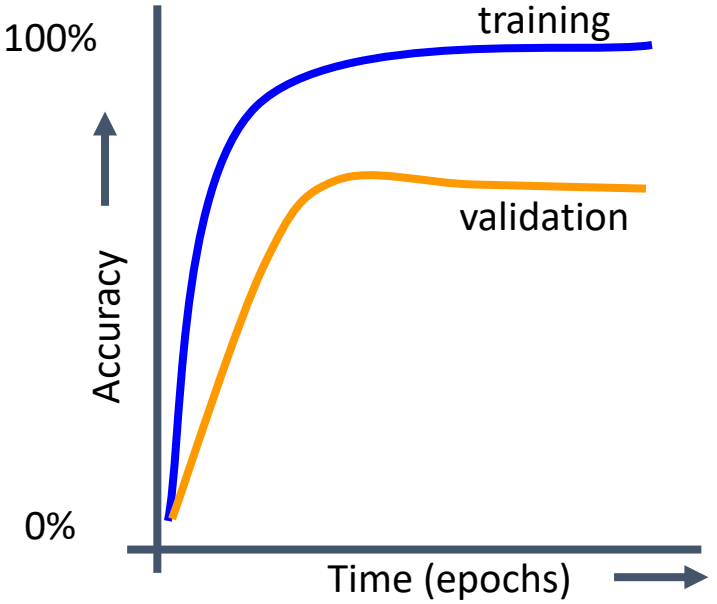
Spotting Underfitting and Overfitting



Underfit: Model performs poorly on training and validation data

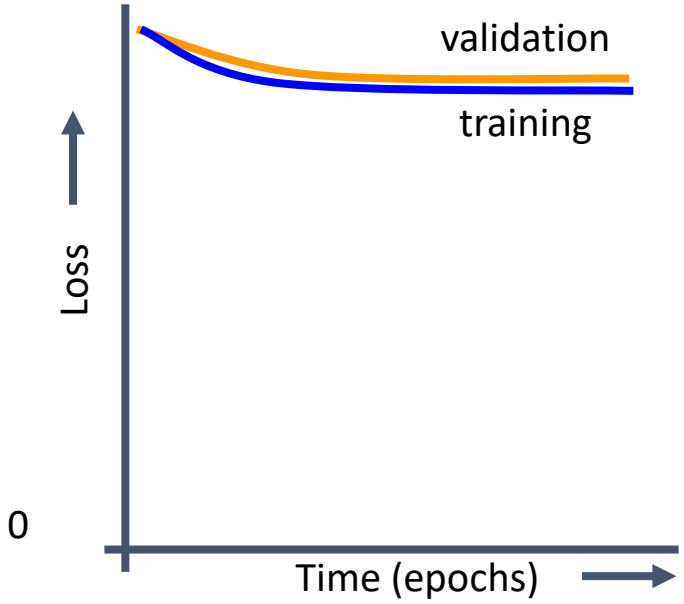


Good fit: Model generalizes well from training to validation data

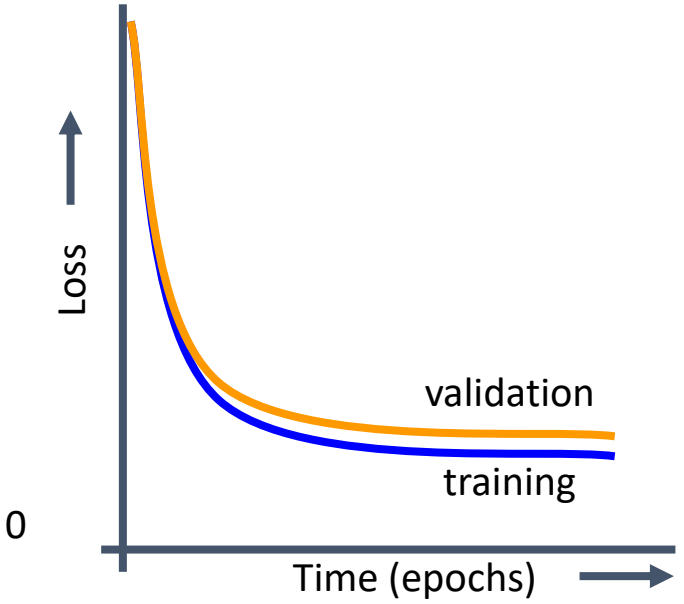


Overfit: Model predicts training data well but fails to generalize to validation data

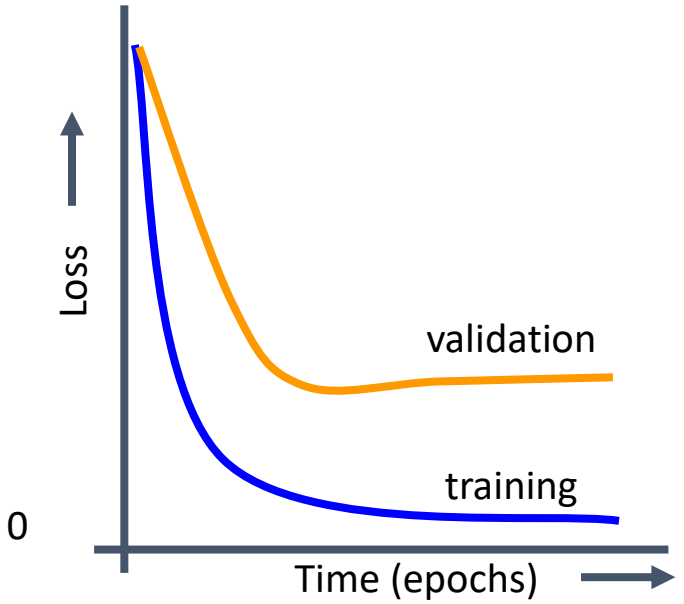
Spotting Underfitting and Overfitting



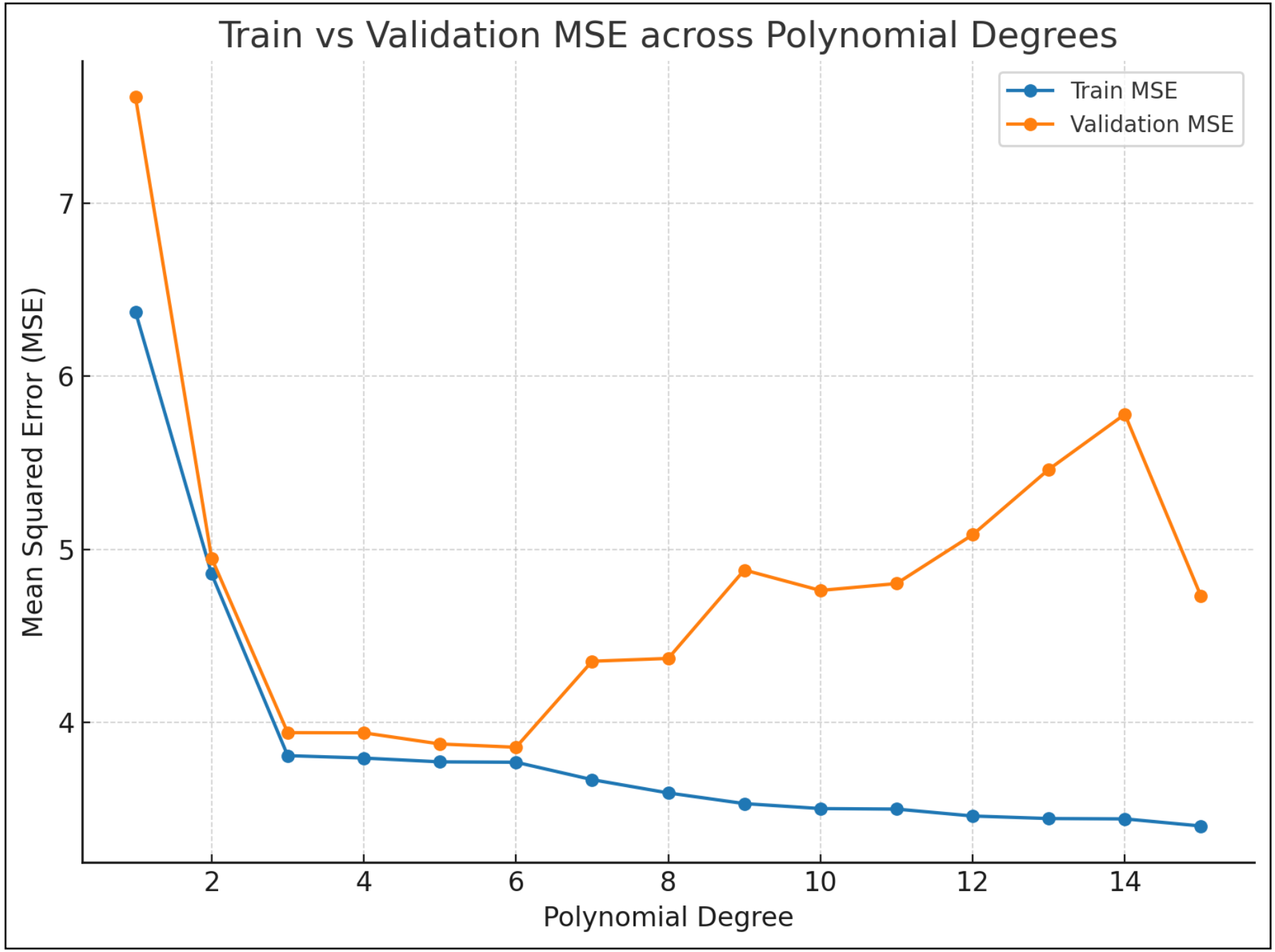
Underfit: Model performs poorly on training and validation data



Good fit: Model generalizes well from training to validation data

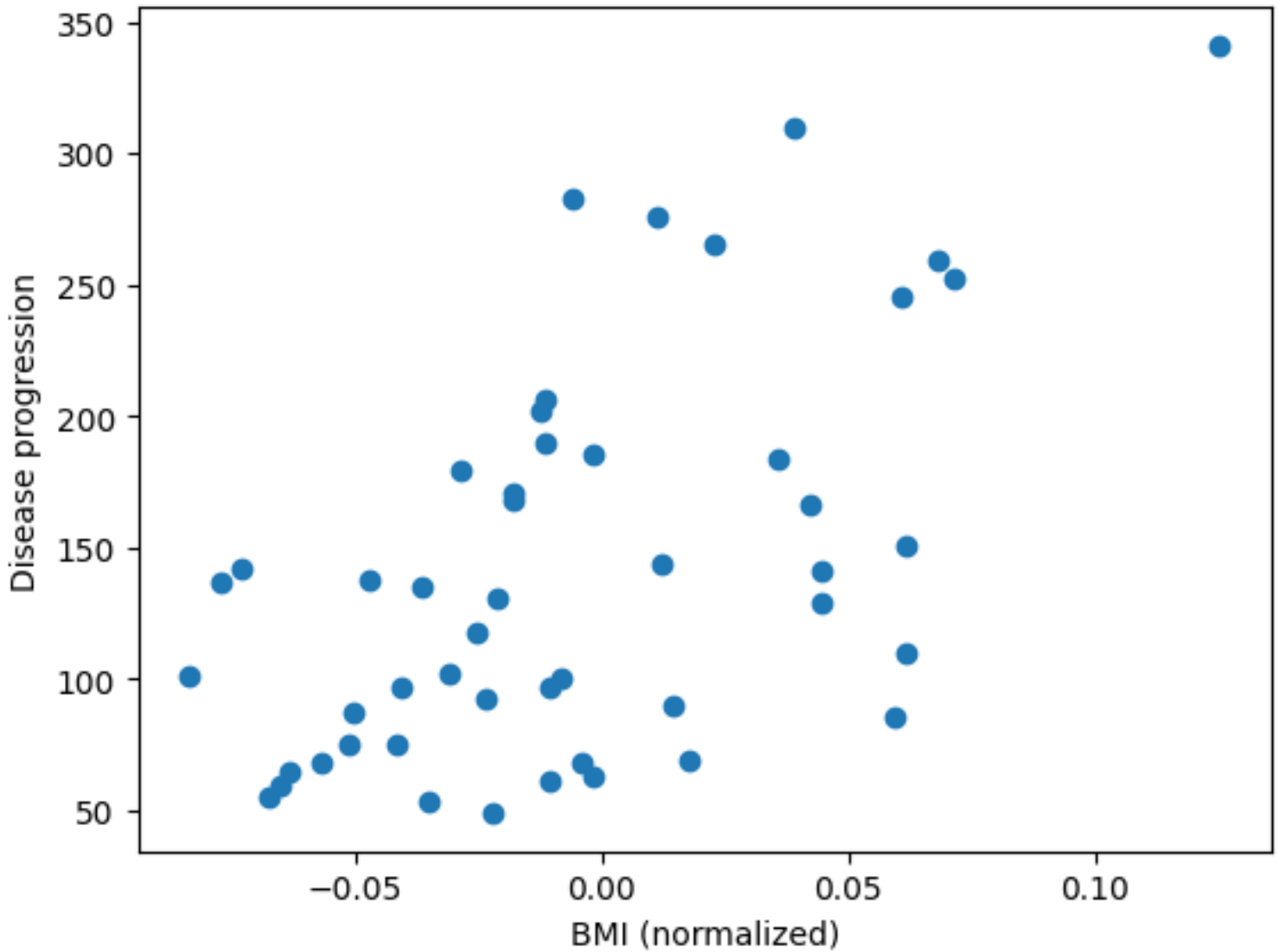


Overfit: Model predicts training data well but fails to generalize to validation data


Train vs Validation MSE across Polynomial Degrees
Train MSE
Validation MSE
Mean Squared Error (MSE)
7
6
5
4
2
4
6
8
10
12
14
Polynomial Degree

Diabetes prediction dataset

A Comprehensive Dataset for Predicting Diabetes with Medical & Demographic Data



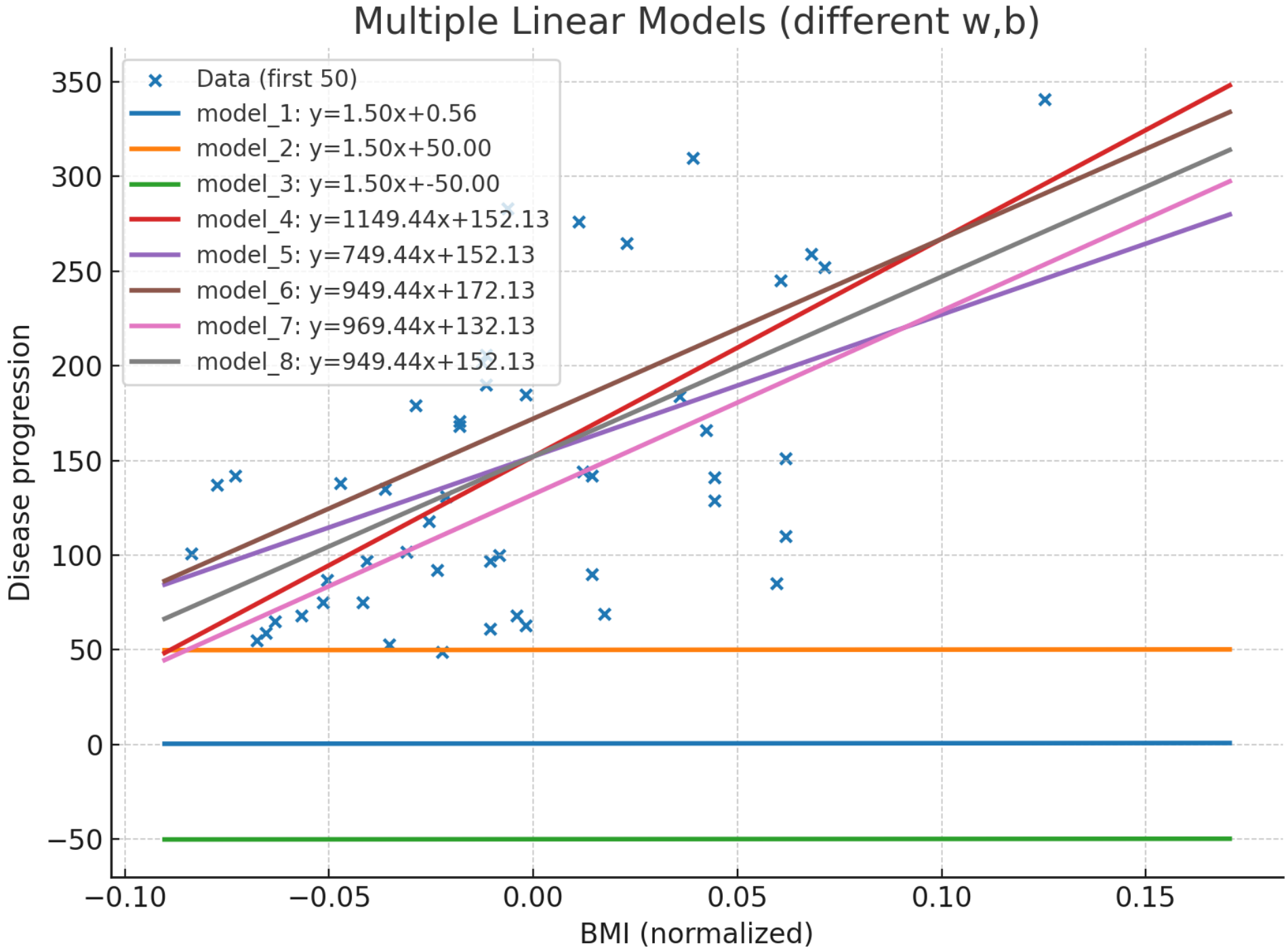
Dataset and model

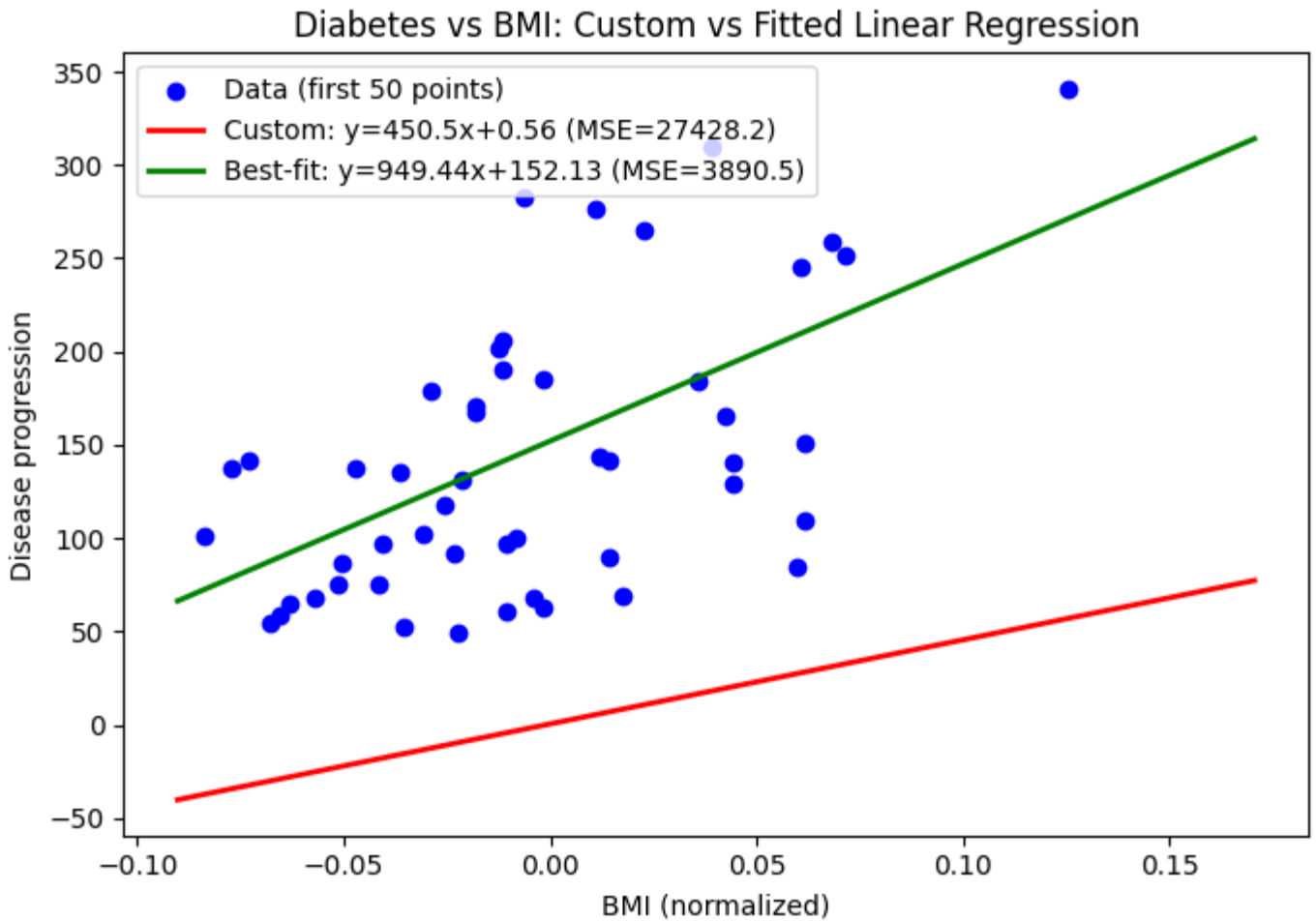
- Data: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$
- Parameters: $\theta = (w, b)$
- Model: $f(x; \theta) = wx + b$

$$\min_{w,b} \; J(w,b) \quad \text{where} \quad J(w,b) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - (wx_i + b)\right)^2$$

$$(w^*, b^*) = \arg\min_{w,b} \; \frac{1}{N} \sum_{i=1}^{N} (y_i - (wx_i + b))^2$$

👉 *"Choose the slope w^* and intercept b^* such that the mean squared error over all data points is as small as possible."*


Multiple Linear Models (different w,b)
Data (first 50)
model_1: y=1.50x+0.56
model_2: y=1.50x+50.00
model_3: y=1.50x+-50.00
model_4: y=1149.44x+152.13
model_5: y=749.44x+152.13
model_6: y=949.44x+172.13
model_7: y=969.44x+132.13
model_8: y=949.44x+152.13
Disease progression
BMI (normalized)
350
300
250
200
150
100
50
0
−50
−0.10
−0.05
0.00
0.05
0.10
0.15


Diabetes vs BMI: Custom vs Fitted Linear Regression
Data (first 50 points)
Custom: y=450.5x+0.56 (MSE=27428.2)
Best-fit: y=949.44x+152.13 (MSE=3890.5)
Disease progression
BMI (normalized)
350
300
250
200
150
100
50
0
−50
−0.10
−0.05
0.00
0.05
0.10
0.15

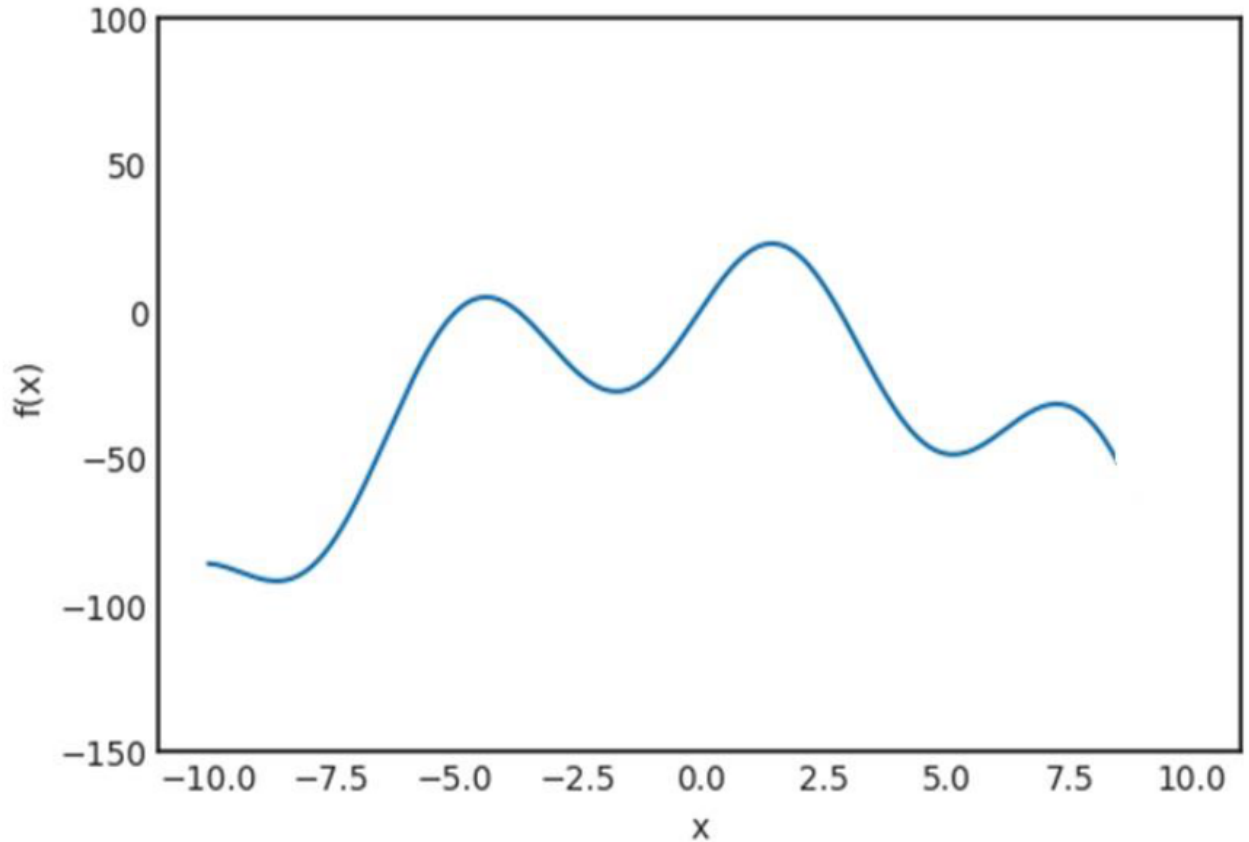
Optimization in Machine (Deep) Learning

Anatomy of a Learning Algorithm

1. Cost function/ Error function
2. An optimization criterion based on the loss function
3. An optimization routine that leverages training data to find a solution to the optimization criterion.

What is optimization?

$$\boldsymbol{x}^* = \arg\min f(\boldsymbol{x}).$$



Learning means finding a combination of model parameters that minimizes a loss function for a given set of training data samples and their corresponding targets.

1. **Loss functions in ML are rarely solvable analytically**
    - For most machine learning models, the loss function does not have a closed-form solution.
    - This means we cannot directly compute the “best” parameters using algebraic methods.

2. **Need for iterative parameter update methods**
    - Instead of exact solutions, we use *iterative optimization algorithms*.
    - These methods update parameters step by step, gradually improving the loss value.

Challenges in optimization

- **Time complexity**: Iterative updates can be slow, especially for large datasets and deep models.
- **Convergence**: Ensuring the method actually reaches (or approaches) an optimum.
- **Quality of solutions**: In non-convex problems

Gradient Descent (GD) as a smart update rule

- GD leverages gradient information (slope/derivative) to decide both the **direction** and **amount** of parameter updates.
- Moves parameters towards the minima of the loss function.

. Start with the Intuition

- **Goal**: Minimize a function (loss function in ML context).
- **Idea**: If you are on a hill (the curve), you want to walk downhill until you reach the lowest point (minimum).
- **Key concept**: The *slope (derivative)* tells you which way is "downhill."

Main Criteria for Applying Gradient Descent

1. Differentiability (at least almost everywhere)

- The function $J(w)$ must be **differentiable** with respect to parameters w.
- Gradients are the "engine" of GD — without derivatives, we cannot compute update directions.
- In practice:
    - Functions like MSE, cross-entropy, softmax are differentiable.

Convexity (ideal but not required)

Convexity (ideal but not required)

- If $J(w)$ is convex:
  - GD is guaranteed to converge to the global minimum.
- If non-convex (like neural nets):
  - GD may converge to a local minimum or saddle point.
  - Still works well in high dimensions, where many local minima are “good enough.”

Why GD works well in practice

- Scales efficiently to large datasets.
- Can handle millions of parameters in neural networks.

Gradient Descent is not the only optimization method, but it is the **most scalable, practical, and effective** choice for training large ML models and deep neural networks.

For training neural networks and large ML models, GD is the only practical choice.

- **Second-order methods** (Newton, Quasi-Newton):
  - Use curvature (Hessian), converge faster, but too costly for DNNs.
- **Derivative-free heuristics** (GA, PSO, SA):
  - Work in black-box problems, but not scalable.
- **Coordinate Descent**:
  - Update one parameter at a time, good for sparse optimization, not for DNNs.
- **Gradient Descent (GD)** and its **variants (SGD, Momentum, RMSProp, Adam, etc.)**:
  - Balance efficiency, scalability, and convergence.
  - That's why they dominate **deep learning** training.

1. Loss Function as a Straight Line (1D)

Suppose:

$$J(w) = aw + c$$

where a, c are constants.

- Derivative:

$$\frac{dJ}{dw} = a$$

- This derivative is a **constant**, independent of w.

- If $a > 0$, the function always increases with w. The minimum is at $w \to -\infty$.
- If $a < 0$, the function always decreases with w. The minimum is at $w \to +\infty$.
- If $a = 0$, the function is flat (any w is optimal).

So **gradient descent doesn't stop anywhere** unless slope = 0.

- Function:

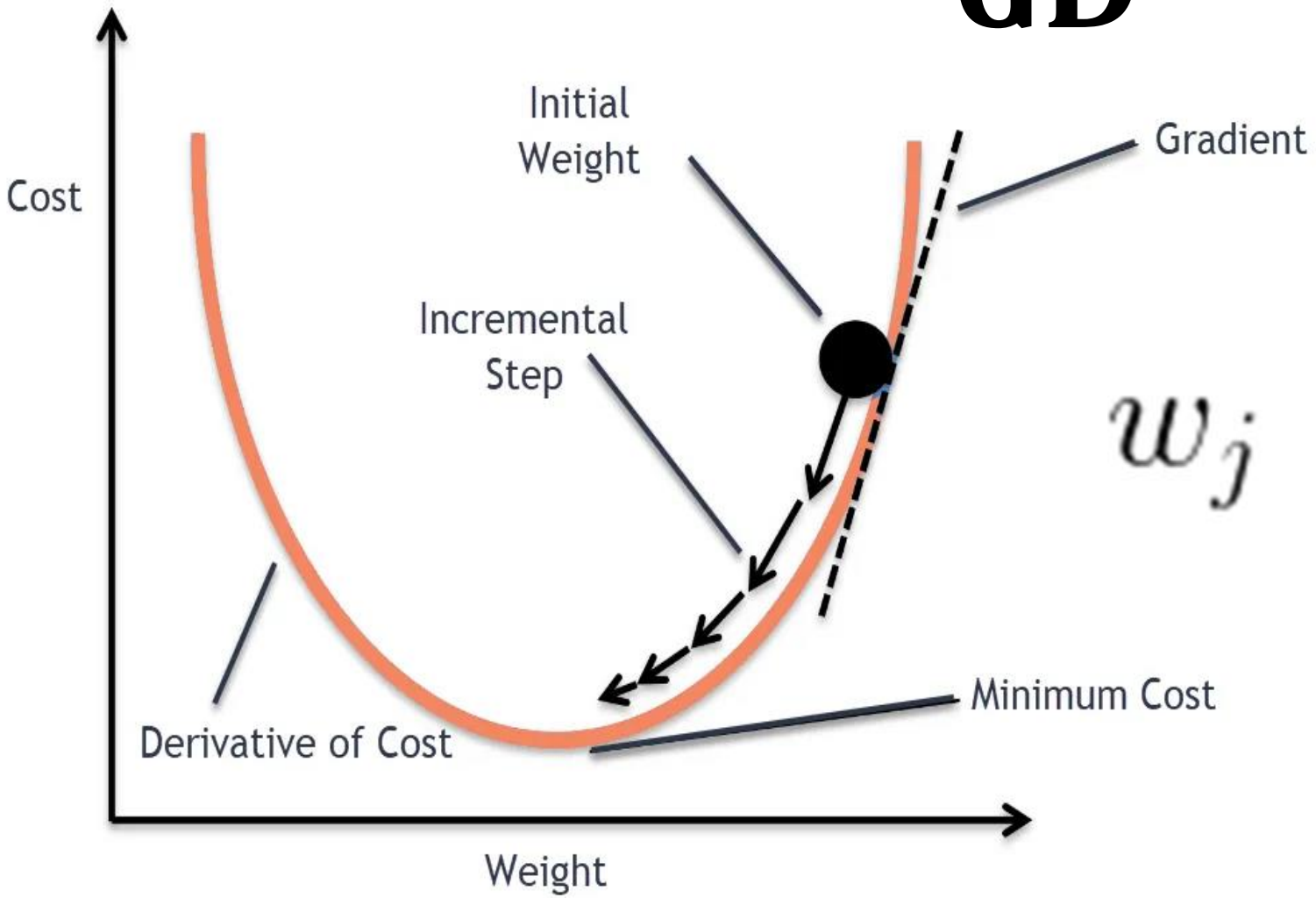
$$f(w) = w^2$$

- Derivative (gradient in 1D):

$$f'(w) = 2w$$

- The **sign** of the gradient gives the **direction** to move.
- The **magnitude** of the gradient gives the **amount** to move.

GD



$$w_j := w_j - \eta \frac{\partial J}{\partial w_j}$$

Gradient descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \eta f'(w)$$

- If $w > 0$:
  - $f'(w) = 2w > 0$.
  - Update: $w_{\text{new}} = w_{\text{old}} - \eta(positive)$.
  - So w moves **left (toward zero)**.
- If $w < 0$:
  - $f'(w) = 2w < 0$.
  - Update: $w_{\text{new}} = w_{\text{old}} - \eta(negative)$.
  - So w moves **right (toward zero)**.

👉 The **sign of the gradient** determines the **direction** of movement:

- Positive gradient → move left.
- Negative gradient → move right.

Amount of Update

- Magnitude of the update = $\eta|f'(w)| = \eta|2w|$.
- Far from the minimum (large $|w|$):
  - Gradient is large → big steps.
- Close to the minimum (small $|w|$):
  - Gradient is small → tiny steps.

👉 The **magnitude of the gradient** automatically adjusts the **step size**:

- Big error → larger corrections.
- Near optimum → fine-tuned small adjustments.

For $f(w) = w^2$, the gradient not only says **"go toward zero"** (direction), but also **"go in smaller steps as you get closer"** (amount). That's why gradient descent is efficient.

Example Numerical Walk

Let's say $\eta = 0.1$.

- Start: $w = 4$.
  - Gradient = $2(4) = 8$.
  - Update: $w \leftarrow 4 - 0.1(8) = 3.2$.
- Next: $w = 3.2$.
  - Gradient = 6.4.
  - Update: $w \leftarrow 3.2 - 0.1(6.4) = 2.56$.
- Next: $w = 2.56$.
  - Gradient = 5.12.
  - Update: $w \leftarrow 2.56 - 0.1(5.12) = 2.048$.

🔹 Why Do We Need a Learning Rate (η)?

1. The Update Rule

Gradient descent update (1D example):

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot f'(w)$$

- $f'(w)$ = gradient (slope) tells **direction + raw magnitude**.
- But if we take the **full gradient step** (without scaling), we may jump too far.

That's where the **learning rate** (η) comes in — it's a **scaling factor**.

2. Without Learning Rate

Imagine minimizing $f(w) = w^2$.

- Gradient: $f'(w) = 2w$.
- Update without η:

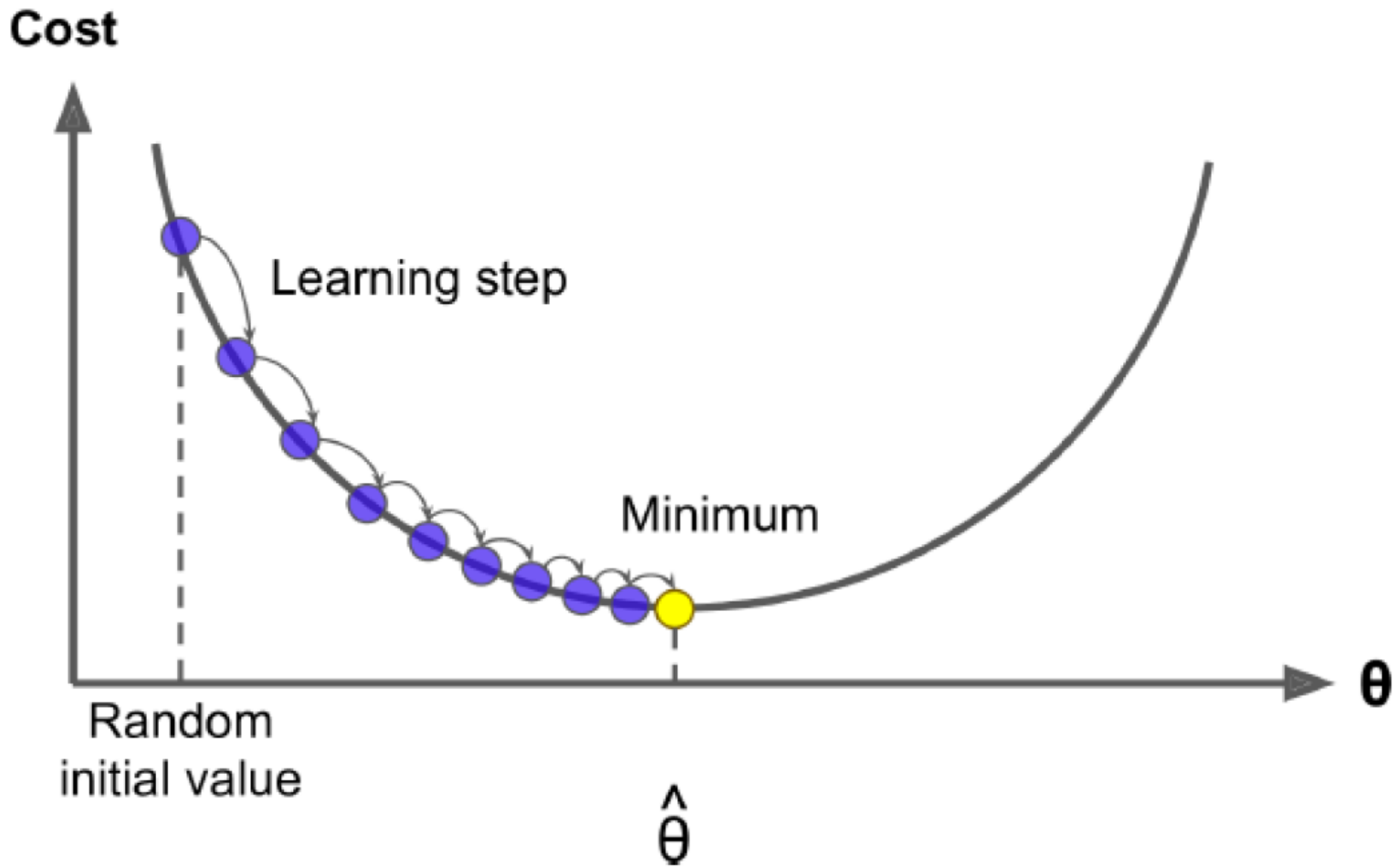
$$w_{\text{new}} = w_{\text{old}} - (2w_{\text{old}}) = -w_{\text{old}}$$

So if $w = 4$:

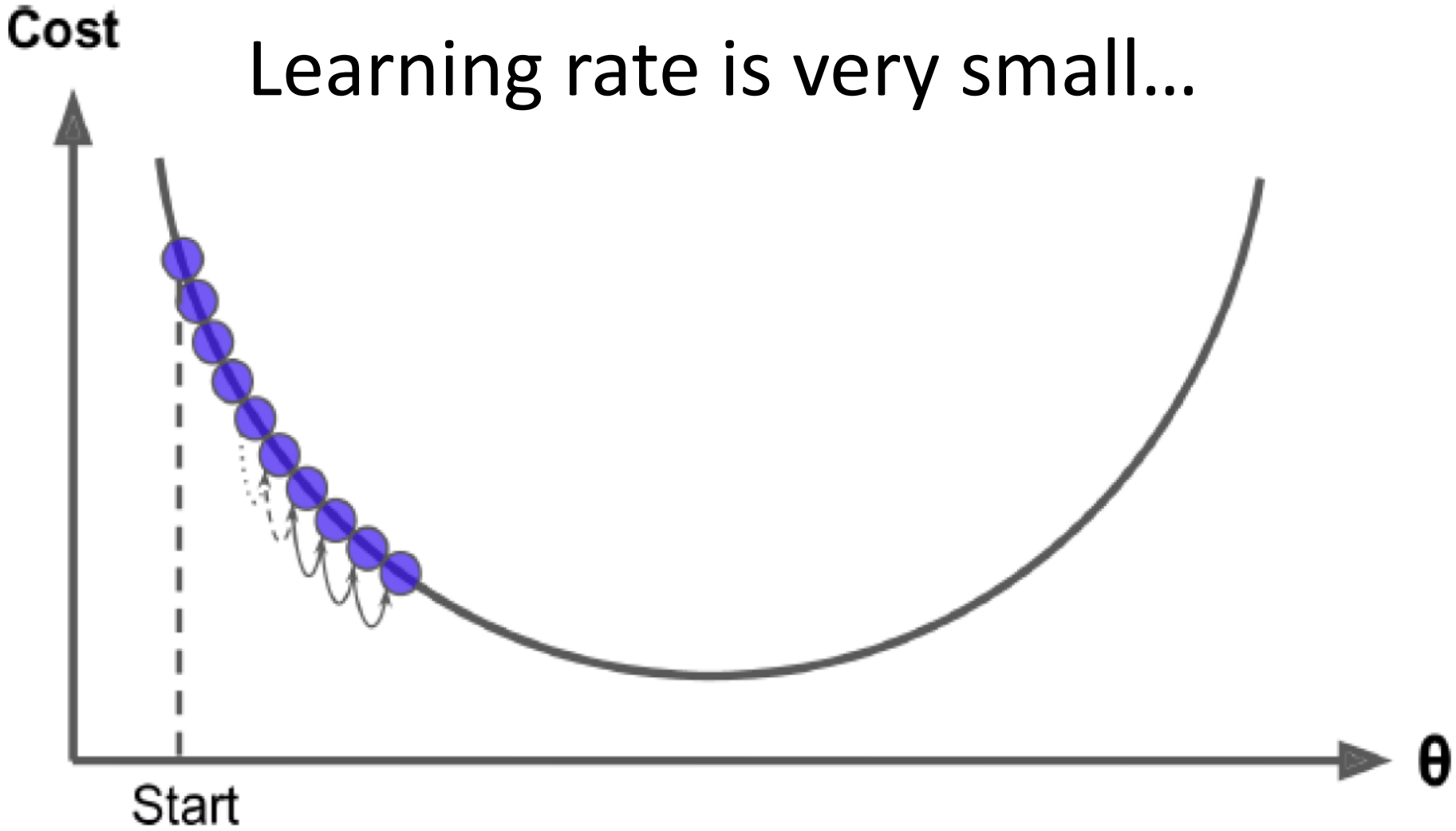
$$w_{\text{new}} = -4$$

👉 You jump from $+4$ to -4, then back to $+4$, and so on → **oscillation, never converges.**

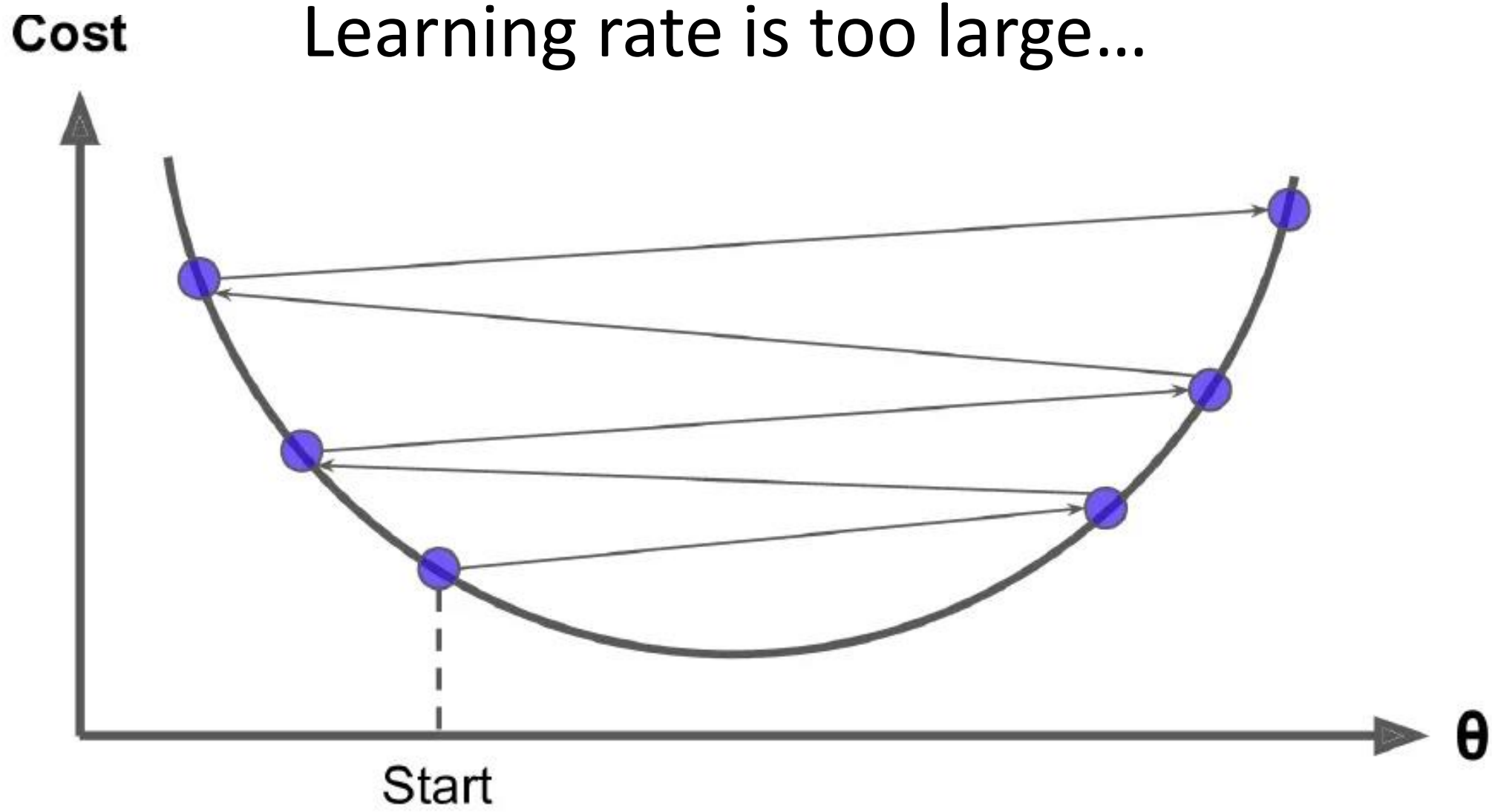
Gradient Based Optimization



Learning rate is very small…



Learning rate is too large…



3. With Learning Rate

Add η:

$$w_{\text{new}} = w_{\text{old}} - \eta(2w_{\text{old}})$$

- If $0 < \eta < 1$:
  - Steps are smaller.
  - The sequence w shrinks toward 0.
  - **Convergence achieved**.
- If η too large (>1 in this case):
  - Steps overshoot.
  - Divergence happens.
- If η too small:
  - Steps are very tiny.
  - Convergence is guaranteed, but **very slow**.

Generalize to 2 Parameters (Parabolic Surface)

Function:

$$f(w, b) = w^2 + b^2$$

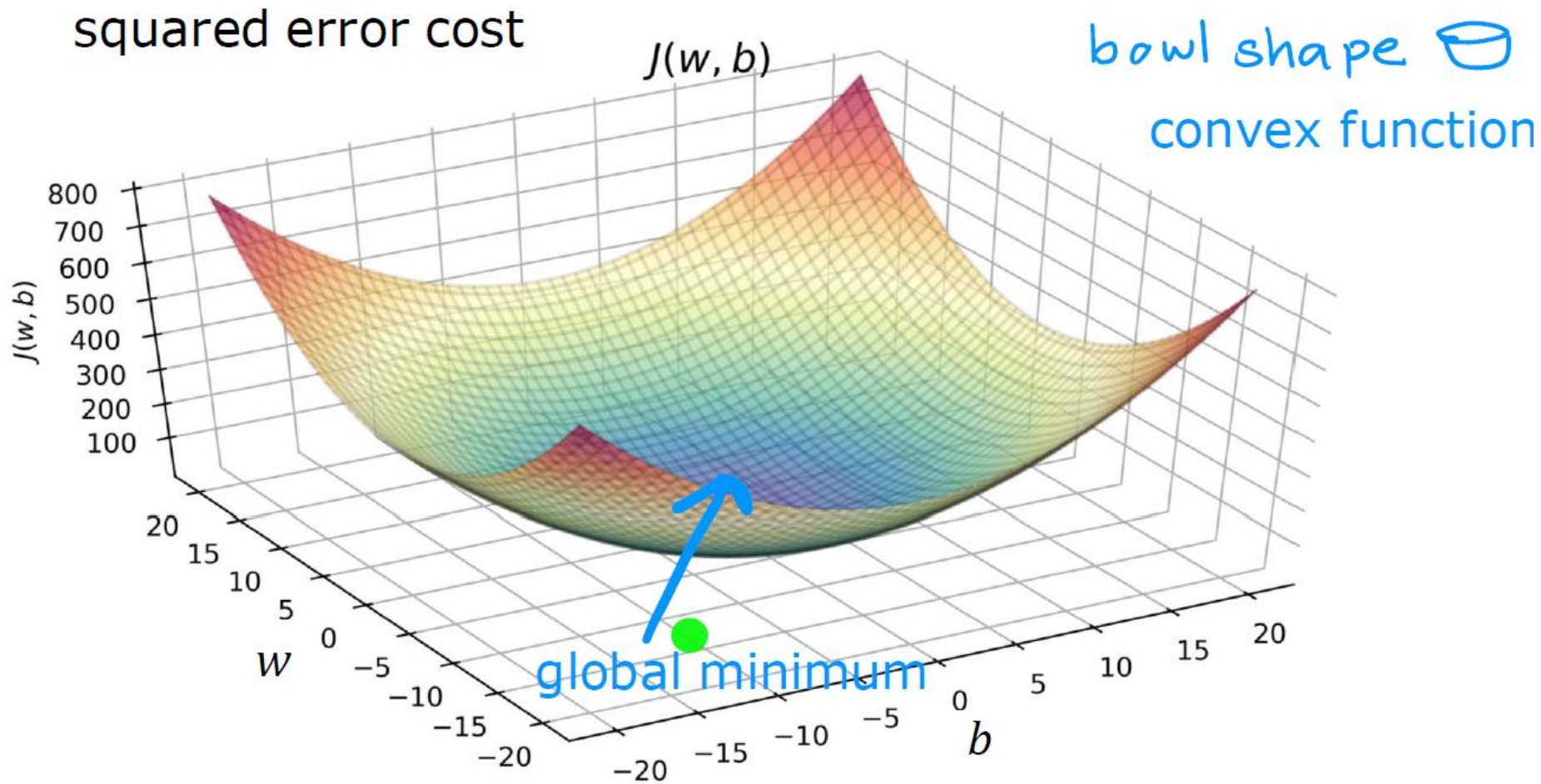
- Gradient:

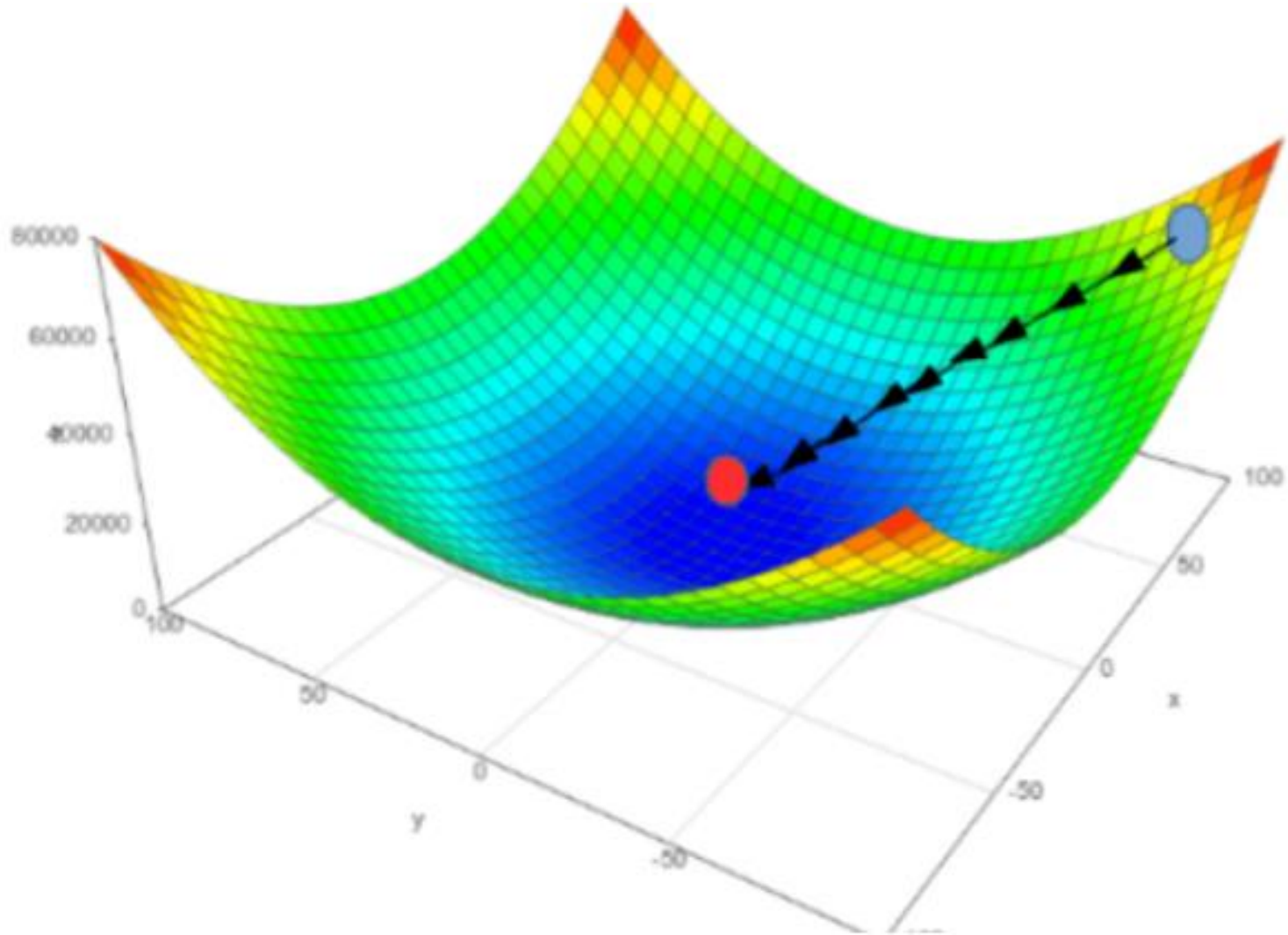
$$\nabla f(w, b) = [2w,\ 2b]$$

- Updates:

$$w \leftarrow w - \eta(2w), \quad b \leftarrow b - \eta(2b)$$

Instead of one slope, now both w and b get updated **simultaneously**.


squared error cost
$J(w, b)$
bowl shape
convex function
800
700
600
500
400
300
200
100
$J(w, b)$
20
15
10
5
0
−5
−10
−15
−20
w
global minimum
−20
−15
−10
−5
0
5
10
15
20
b


80000
60000
40000
20000
0
100
50
0
-50
y
100
50
0
-50
x

Multi-Dimensional Extension

Gradient descent extends naturally to multiple dimensions because the gradient (vector of partial derivatives) is the direct generalization of the slope in 1D. The update rule is the same: move opposite to the gradient, just now in vector space instead of along a line.

- In **multi-D**, we need a way to capture slopes along *all directions*.
- That's exactly what the **gradient vector** does:

$$\nabla f(\mathbf{w}) = \left[\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}, \ldots, \frac{\partial f}{\partial w_n} \right]$$

So instead of a single number (slope), we now have a vector pointing toward **steepest ascent**.

Descent = Opposite of Ascent

- The gradient points to where the function increases fastest.
- If we want to **minimize**, we move in the **opposite direction**.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t)$$

This is exactly the same logic as 1D, just extended to a vector.

Multi-dimensional case with two sets of parameters:

- $w \in \mathbb{R}^p$ (e.g., feature weights in ML)
- $b \in \mathbb{R}^q$ (e.g., biases, or another set of parameters)

1. Define the Objective Function

We want to minimize some los:

$$J(w, b)$$

where

- $w = (w_1, w_2, \ldots, w_p)$
- $b = (b_1, b_2, \ldots, b_q)$.

Update Rules

1. Non-Vectorized (One Parameter at a Time)

At iteration t, update each parameter **individually**:

For each weight

$$w_j^{(t+1)} = w_j^{(t)} - \eta \frac{\partial J}{\partial w_j}, \quad j = 1, 2, \ldots, p$$

For each bias

$$b_k^{(t+1)} = b_k^{(t)} - \eta \frac{\partial J}{\partial b_k}, \quad k = 1, 2, \ldots, q$$

This is the "**parameter-by-parameter**" update.

Gradient wrt **w**:

$$\nabla_w J(w,b) = \left[\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \ldots, \frac{\partial J}{\partial w_p}\right]$$

Gradient wrt **b**:

$$\nabla_b J(w,b) = \left[\frac{\partial J}{\partial b_1}, \frac{\partial J}{\partial b_2}, \ldots, \frac{\partial J}{\partial b_q}\right]$$

We apply gradient descent **independently** to both parameter groups:

$$w^{(t+1)} = w^{(t)} - \eta_w \nabla_w J(w^{(t)}, b^{(t)})$$

$$b^{(t+1)} = b^{(t)} - \eta_b \nabla_b J(w^{(t)}, b^{(t)})$$

- η_w, η_b = learning rates (can be same or different).
- Both sets are updated **simultaneously each iteration**.

What factors are important for GD convergence?

- **Speed (how long it will take to converge)**
- **Solution quality when GD converges**
- **Computational complexity**

What parameters are important to help GD to converge to an optimal (near-optimal point)?

- **#Iterations (number of epochs)**
- **Learning rate**
- **Parameter update procedure (Batch size)**

We will not call these "Parameters"

"Hyperparameters of the learning algorithm" (GD-based optimization)

Convergence

- In optimization, convergence means the iterative algorithm has found a stable, near-optimal, or optimal solution, where further iterations do not significantly improve the objective function or design variables.

- It signifies that the algorithm's output is settling on a consistent result, indicating the process has reached a point of stability and is no longer making meaningful progress toward the best possible answer.

🔹 Learning Rate Schedule

1. What is it?

A **learning rate schedule** is a strategy to **change the learning rate (η) during training** instead of keeping it fixed.

$$w_{t+1} = w_t - \eta_t \nabla J(w_t)$$

- Here η_t depends on the training step/epoch.
- Goal: balance **fast learning at the start** with **stable convergence at the end**.

(a) Step Decay (most common)

- Start with η_0.
- Drop LR by factor (e.g., 0.1) every few epochs.
- Example:
  - Epochs 0–10: $\eta = 0.01$
  - Epochs 11–20: $\eta = 0.001$
  - Epochs 21–30: $\eta = 0.0001$

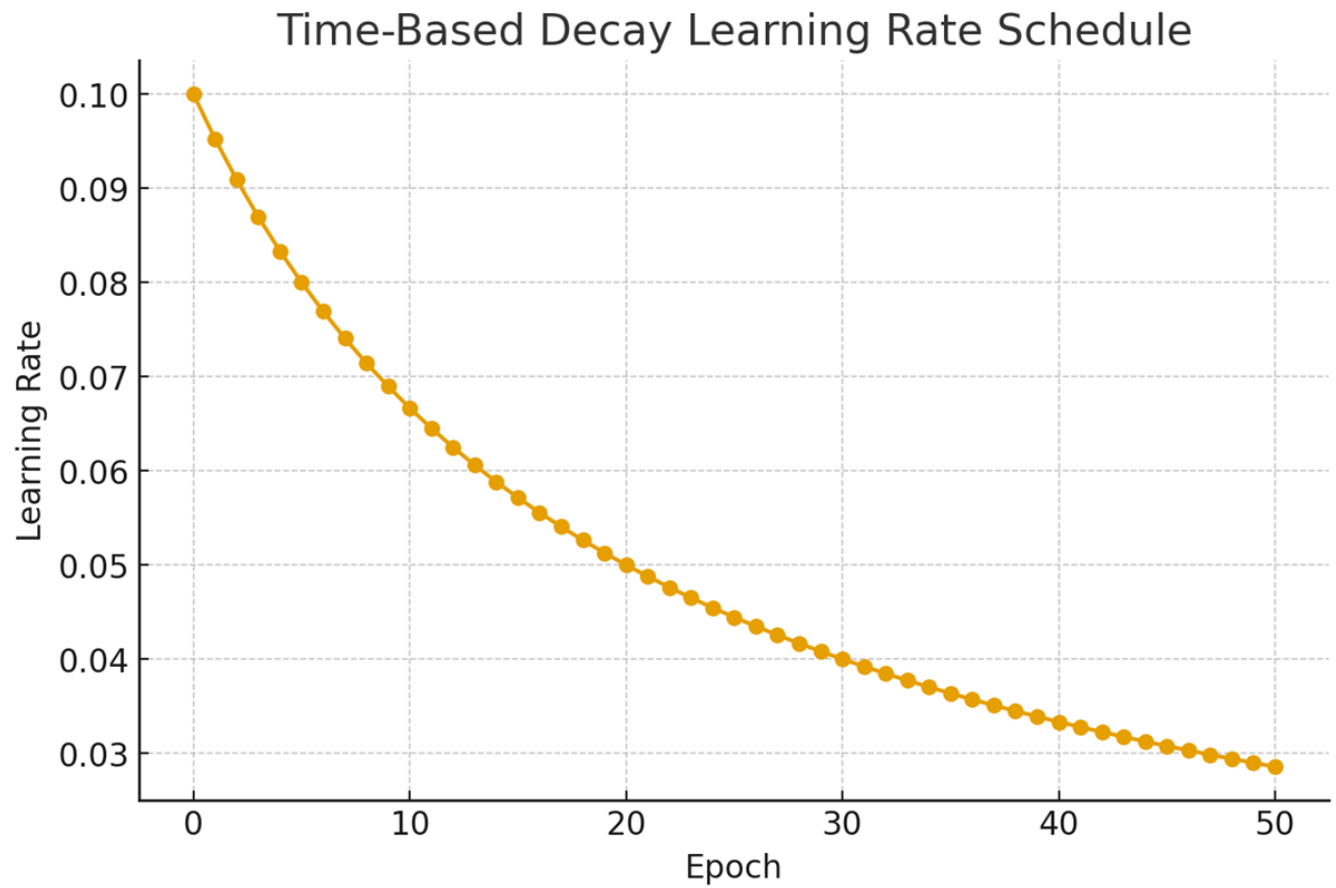
Time-Based Decay

1. Formula

At iteration/epoch t:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

- η_0 = initial learning rate
- k = decay rate (constant, controls how fast LR decreases
- t = epoch or iteration number



🔹 2. Exponential Decay

Formula

$$\eta_t = \eta_0 \cdot e^{-kt}$$

- Decays **multiplicatively** over time.
- LR reduces quickly at first, then flattens out.

Example

- $\eta_0 = 0.1, k = 0.05$
- Epoch 0: LR = 0.1
- Epoch 10: LR ≈ 0.06
- Epoch 50: LR ≈ 0.008

🔷 Gradient Descent Variants: Full Details

We want to minimize the average loss over m training samples:

$$J(w) = \frac{1}{m} \sum_{i=1}^{m} L(y^{(i)}, f(x^{(i)}; w))$$

1. Batch Gradient Descent (BGD)

Analogy: Like measuring everyone's opinion in a country before making one policy change. Very accurate, but very slow.

Update Rule

$$w \leftarrow w - \eta \cdot \nabla J(w)$$

where

$$\nabla J(w) = \frac{1}{m} \sum_{i=1}^{m} \nabla_w L^{(i)}$$

- Uses **all samples** to compute the gradient.

✅ Pros:

- Stable updates.
- Moves smoothly toward minimum.

❌ Cons:

- Very **slow** when dataset is huge.
- Requires loading full dataset into memory.

2. Stochastic Gradient Descent (SGD)

- Uses **one training example at a time** to compute gradient.

$$w \leftarrow w - \eta \nabla_w L^{(i)}$$

✅ Pros:

- Updates are **very fast** (only one sample).
- Adds **randomness**, which helps escape local minima.

❌ Cons:

- Very noisy updates → path zig-zags, doesn't move smoothly.

👉 **Analogy**: Asking **one random person's opinion** before making a policy change. Fast, but unstable.

3. Mini-Batch Gradient Descent

- Compromise between batch and SGD.
- Uses a **small batch of B samples** ($1 < B < m$) each time.

$$\nabla J_{\text{mini}}(w) = \frac{1}{B} \sum_{i=1}^{B} \nabla_w L^{(i)}$$

Update:

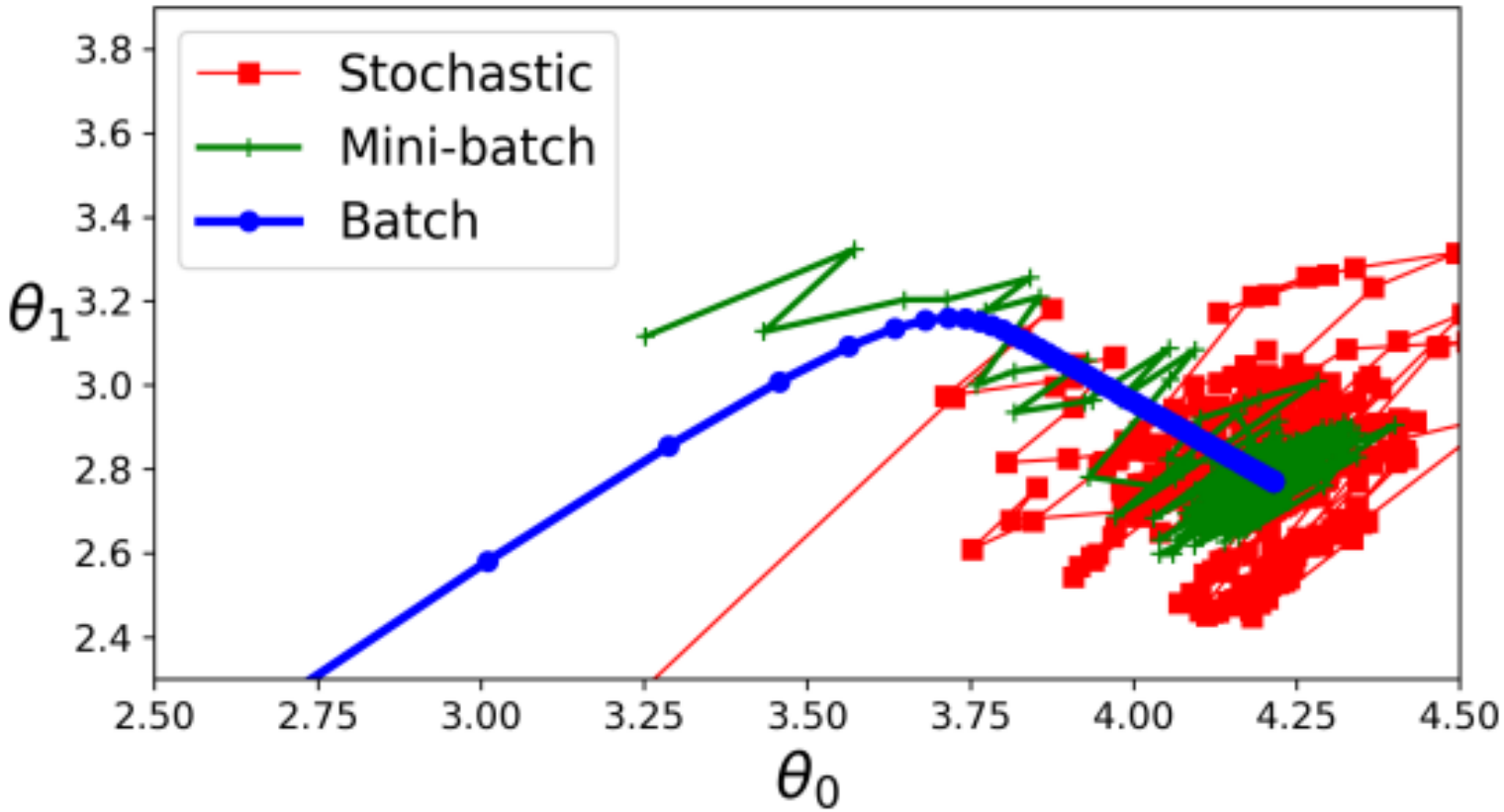
$$w \leftarrow w - \eta \nabla J_{\text{mini}}(w)$$

✅ Pros:

- Faster than full batch.
- More stable than pure SGD.
- Works well with **vectorized hardware (GPU/TPU)**.

❌ Cons:

- Still noisy, but controlled.

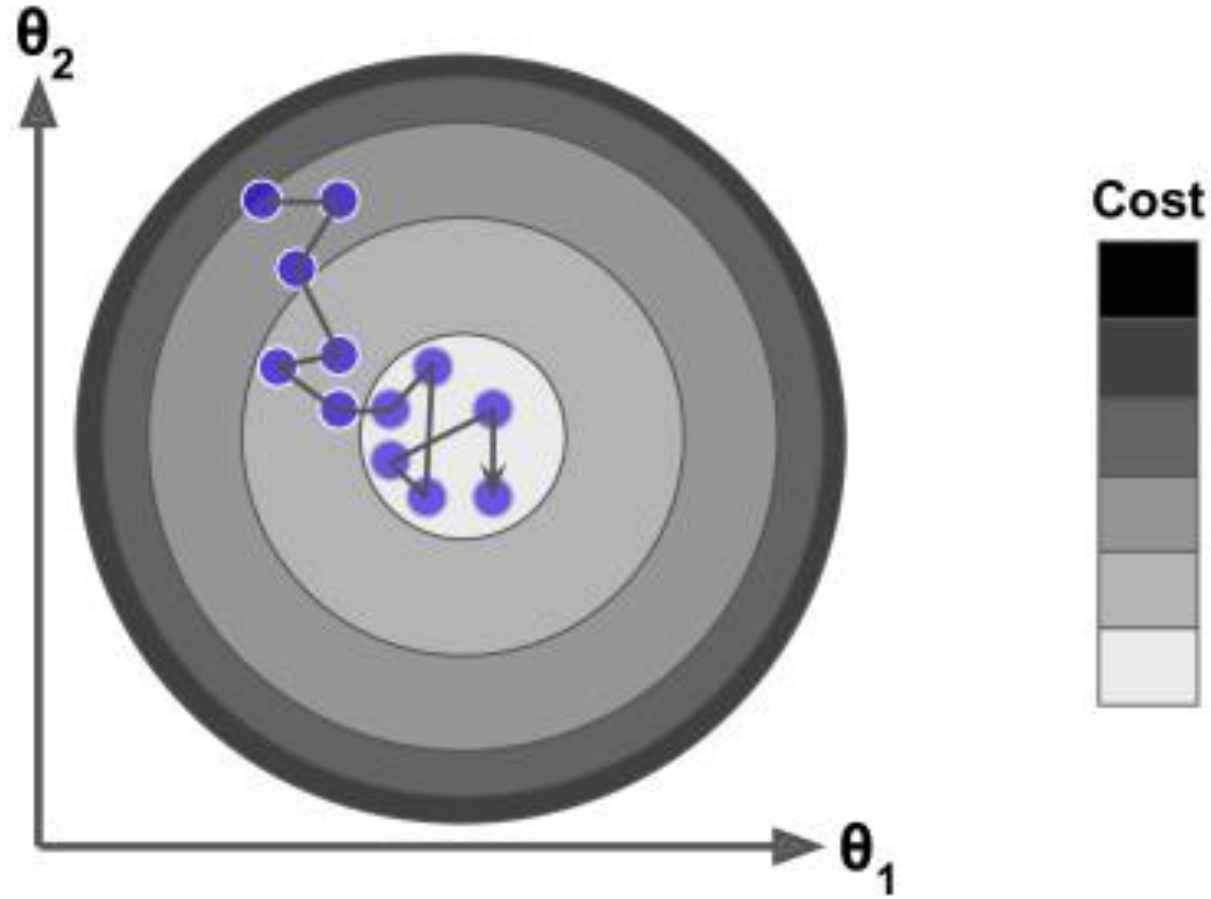

Stochastic
Mini-batch
Batch
3.8
3.6
3.4
3.2
3.0
2.8
2.6
2.4
θ_1
2.50
2.75
3.00
3.25
3.50
3.75
4.00
4.25
4.50
θ_0

🔹 Comparison Table

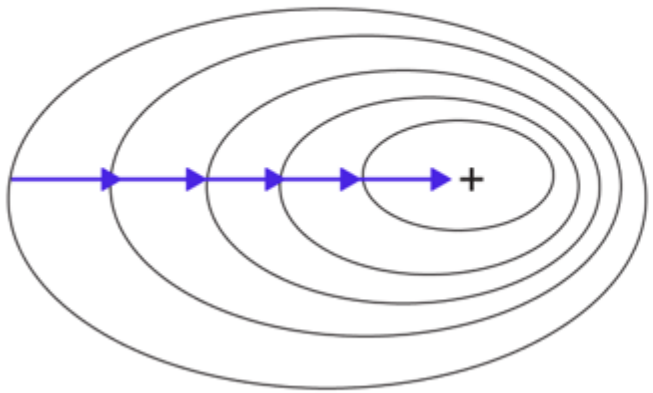
Method	Data per update	Update Cost	Path	Convergence	Use Case
Batch GD	All samples (m)	High	Smooth	Precise but slow (wall-clock)	Small datasets
SGD	1 sample	Very low	Noisy zig-zag	Quick to get close, slow to settle	Online/streaming data
Mini-Batch GD	B samples (32–256)	Medium	Balanced	Fastest in practice	Standard in deep learning

What is a Contour Plot?

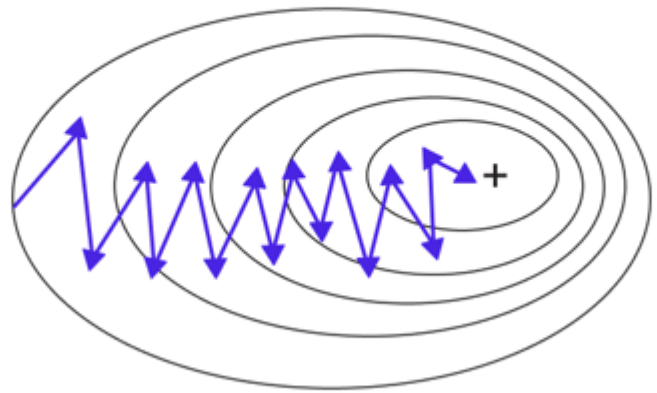
- A **contour plot** shows level curves of a function $f(w_1, w_2)$ in 2D.
- Each contour line = set of points where function has the same value.
- Think of it like a **topographic map** (lines of equal altitude).
- In optimization:
  - Lower contours = lower loss.
  - The **minimum** = center of contours.


θ2
θ1
Cost

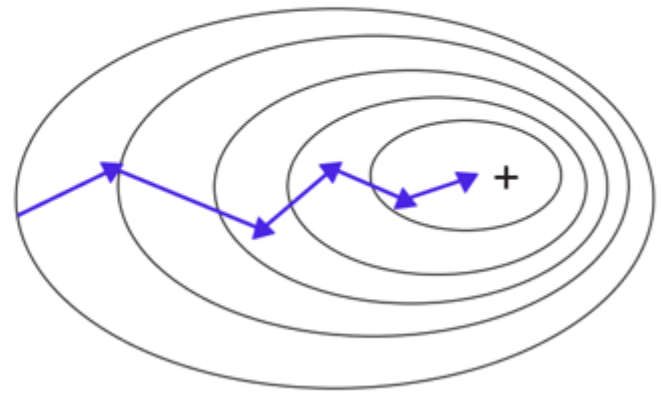
Batch Gradient Descent

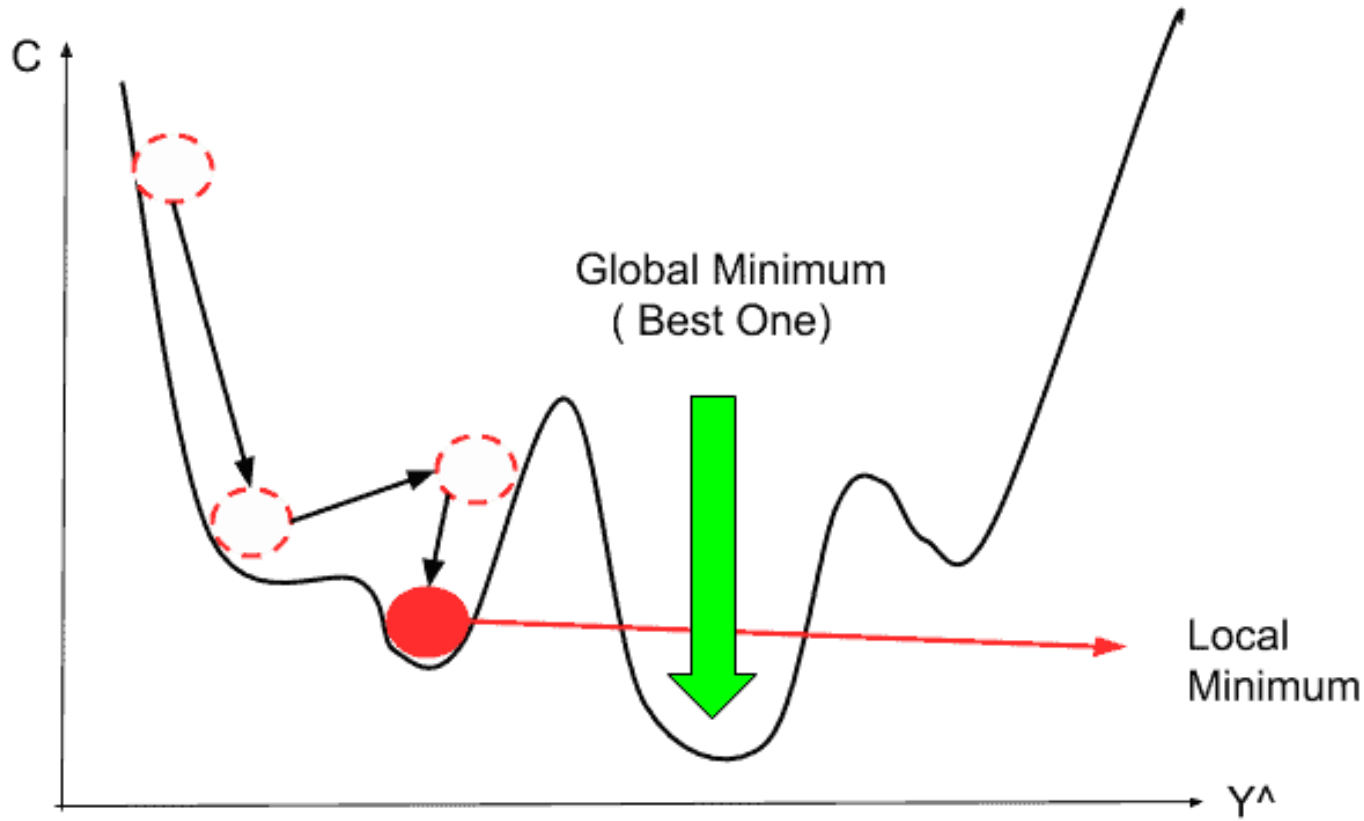


Stochastic Gradient Descent

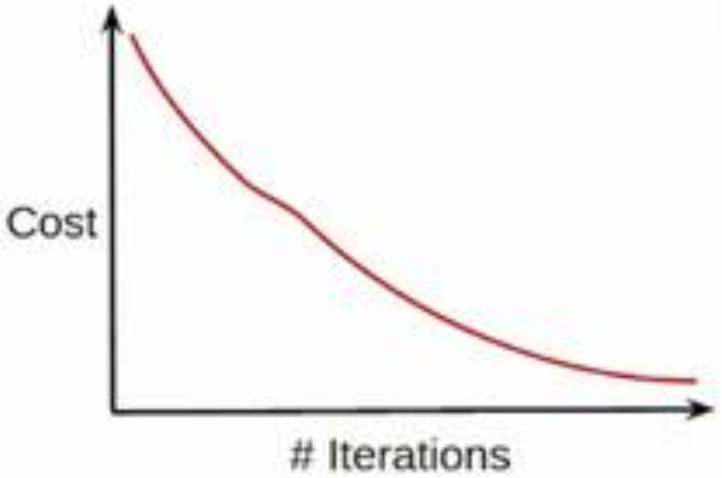


Mini-Batch Gradient Descent

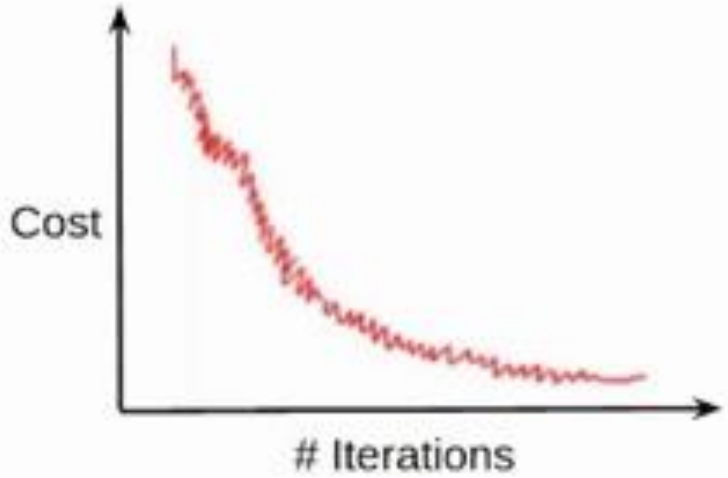



C
Global Minimum
(Best One)
Local
Minimum
Y^

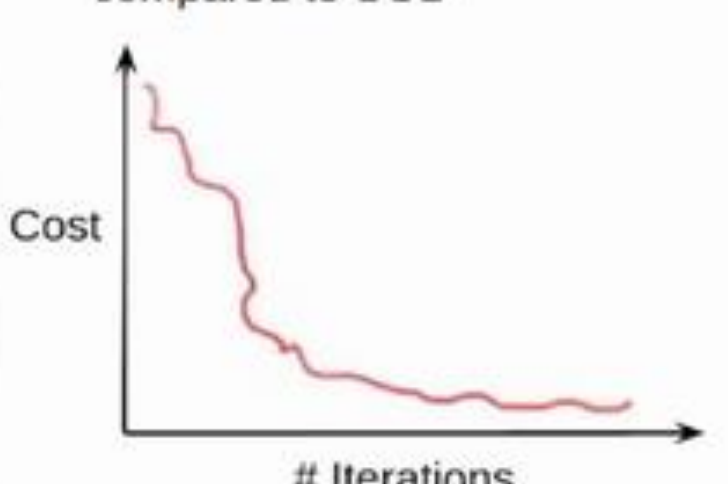
- Cost function reduces smoothly



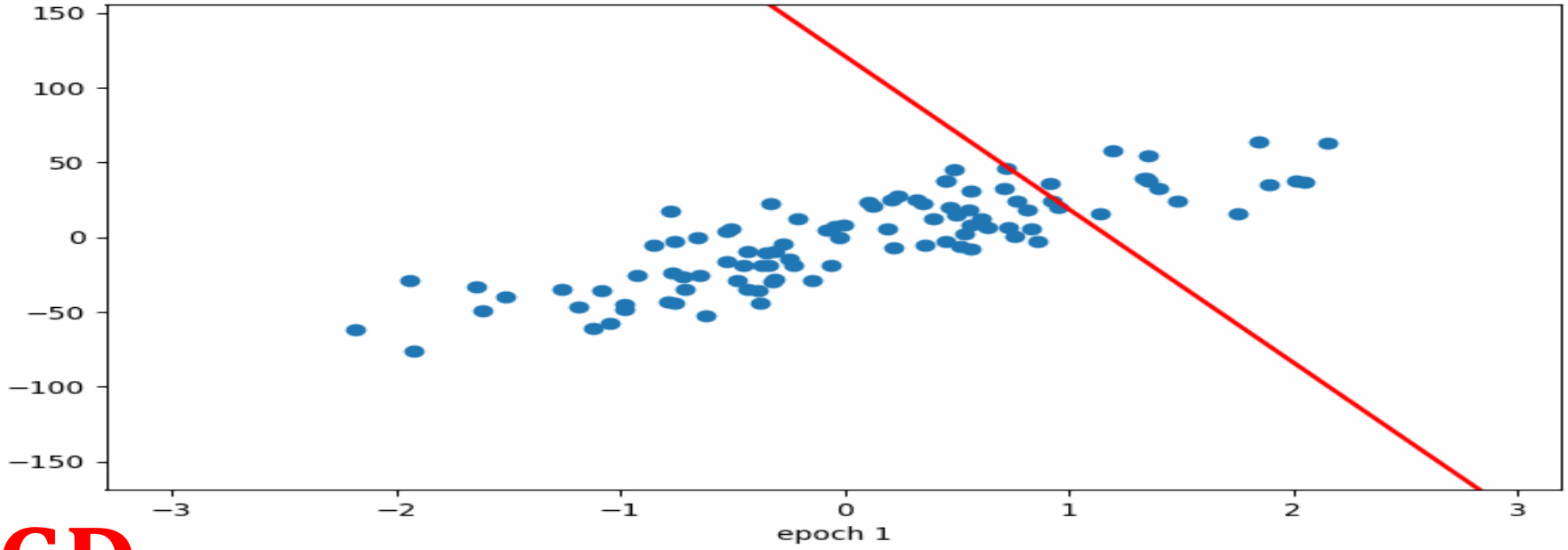
- Lot of variations in cost function

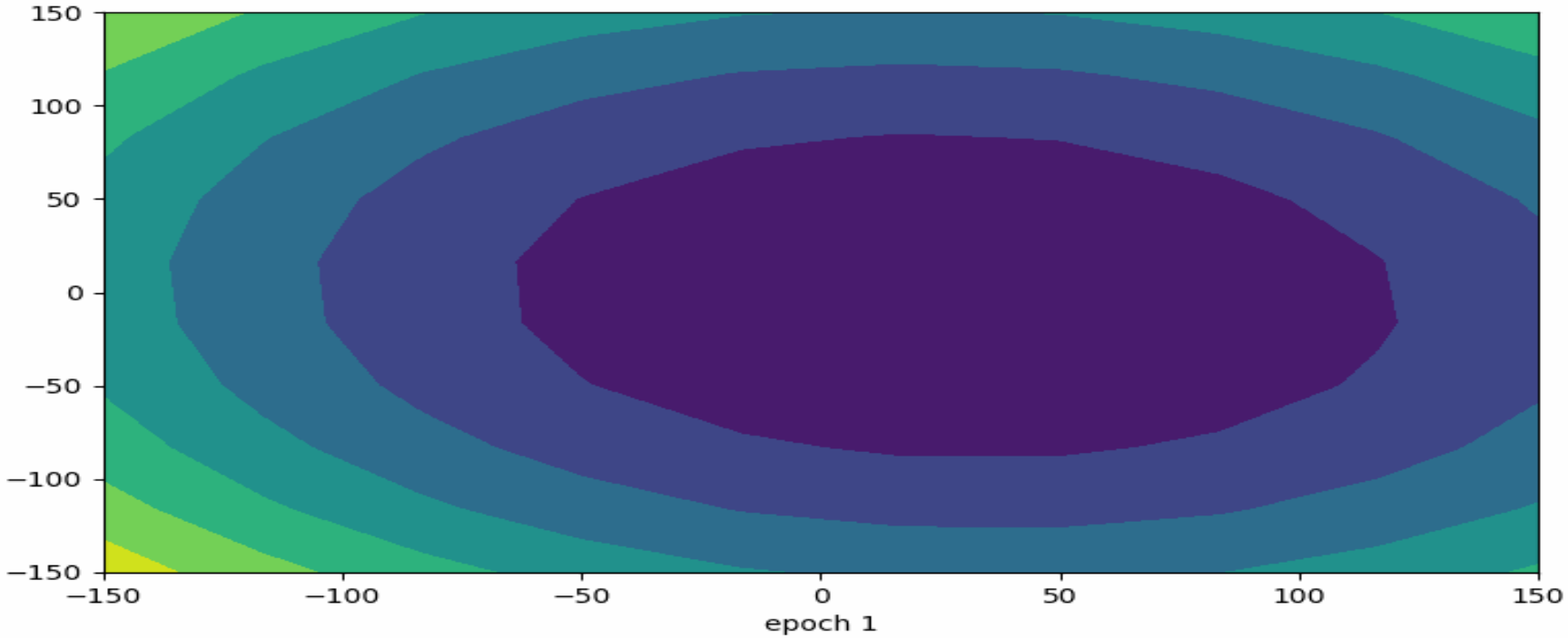


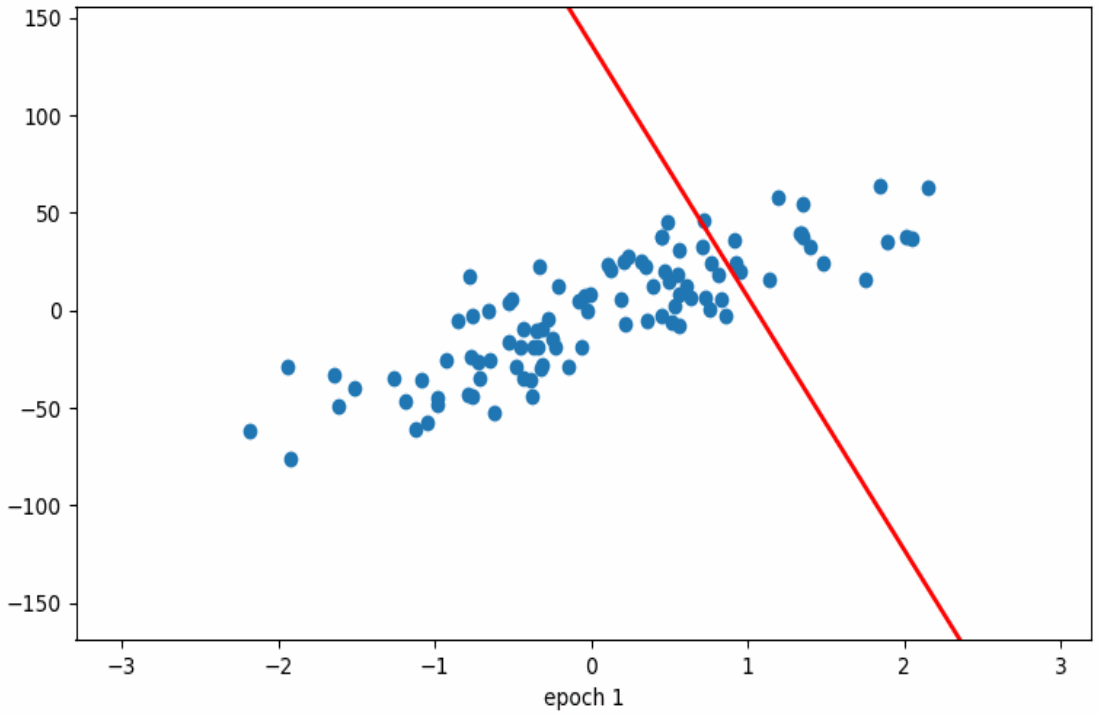
- Smoother cost function as compared to SGD



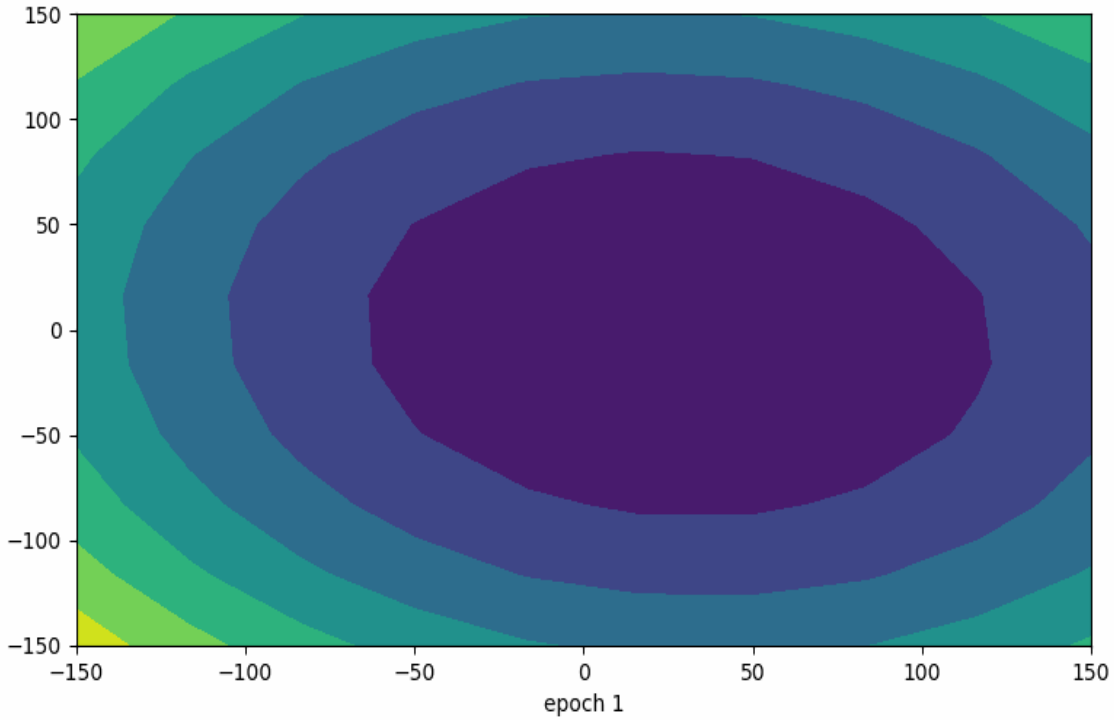
Batch GD


150
100
50
0
−50
−100
−150
−3
−2
−1
0
1
2
3
epoch 1

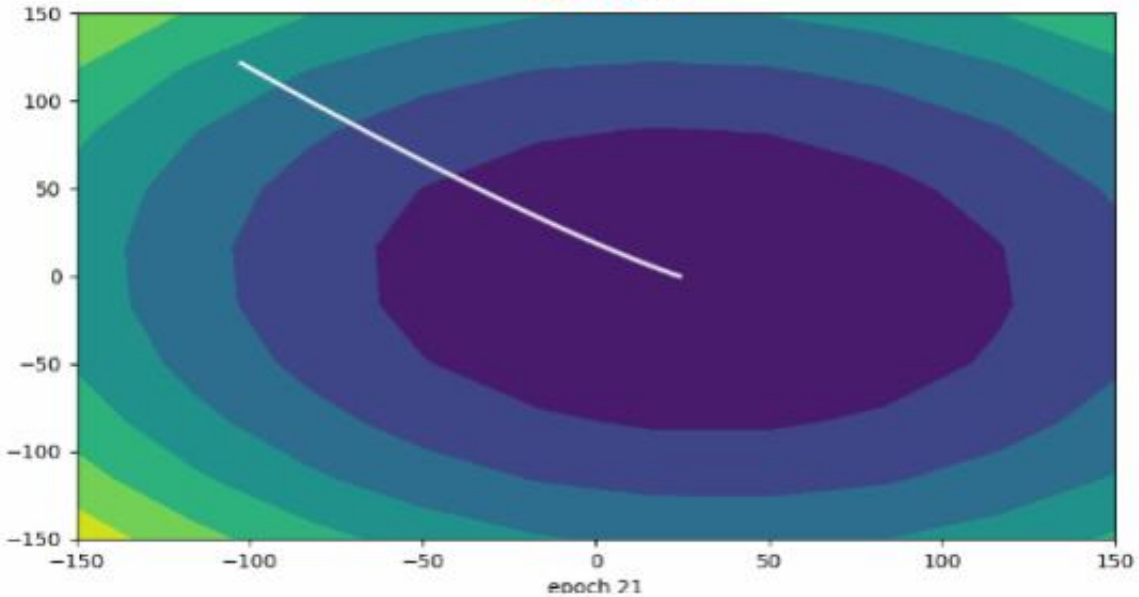

150
100
50
0
−50
−100
−150
−150
−100
−50
0
50
100
150
epoch 1


150
100
50
0
−50
−100
−150
−3
−2
−1
0
1
2
3
epoch 1

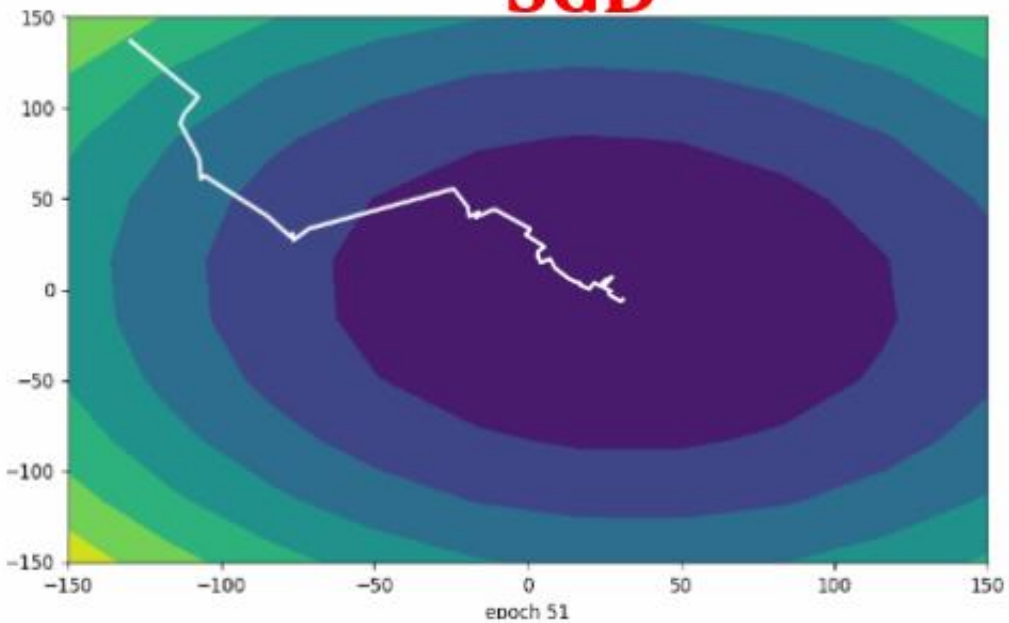
SGD

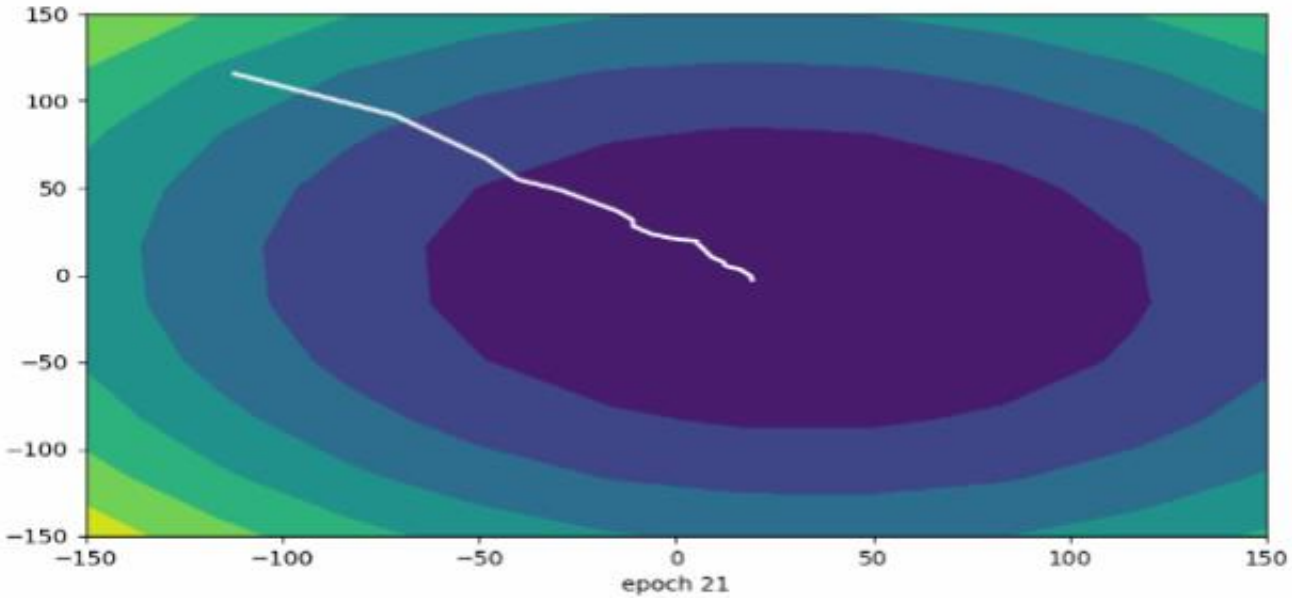

150
100
50
0
−50
−100
−150
−150
−100
−50
0
50
100
150
epoch 1

BGD


150
100
50
0
−50
−100
−150
−150
−100
−50
0
50
100
150
epoch 21

SGD


150
100
50
0
−50
−100
−150
−150
−100
−50
0
50
100
150
epoch 51


150
100
50
0
−50
−100
−150
−150
−100
−50
0
50
100
150
epoch 21

MBGD

Can we use MSE (loss function) for a classification task?

- ML task: *Object classification with 4 classes (say Cat 🐱, Dog 🐶, Car 🚙, Plane ✈️).*
- Each data point: (x, y).
- Label y represented as one-hot probability distribution:

$$P = [1, 0, 0, 0] \quad \text{if Cat is true.}$$

- Model predicts probabilities:

$$Q = [q_1, q_2, q_3, q_4], \quad \sum q_i = 1$$

If $P = [1, 0, 0, 0]$ and model says $Q = [0.9, 0.05, 0.03, 0.02]$:

"How do we measure how good/bad this prediction is?"

Can we use MSE?

- MSE for one sample:

MSE ≈ 0.0025

$$L = \frac{1}{4}\sum(p_i - q_i)^2$$

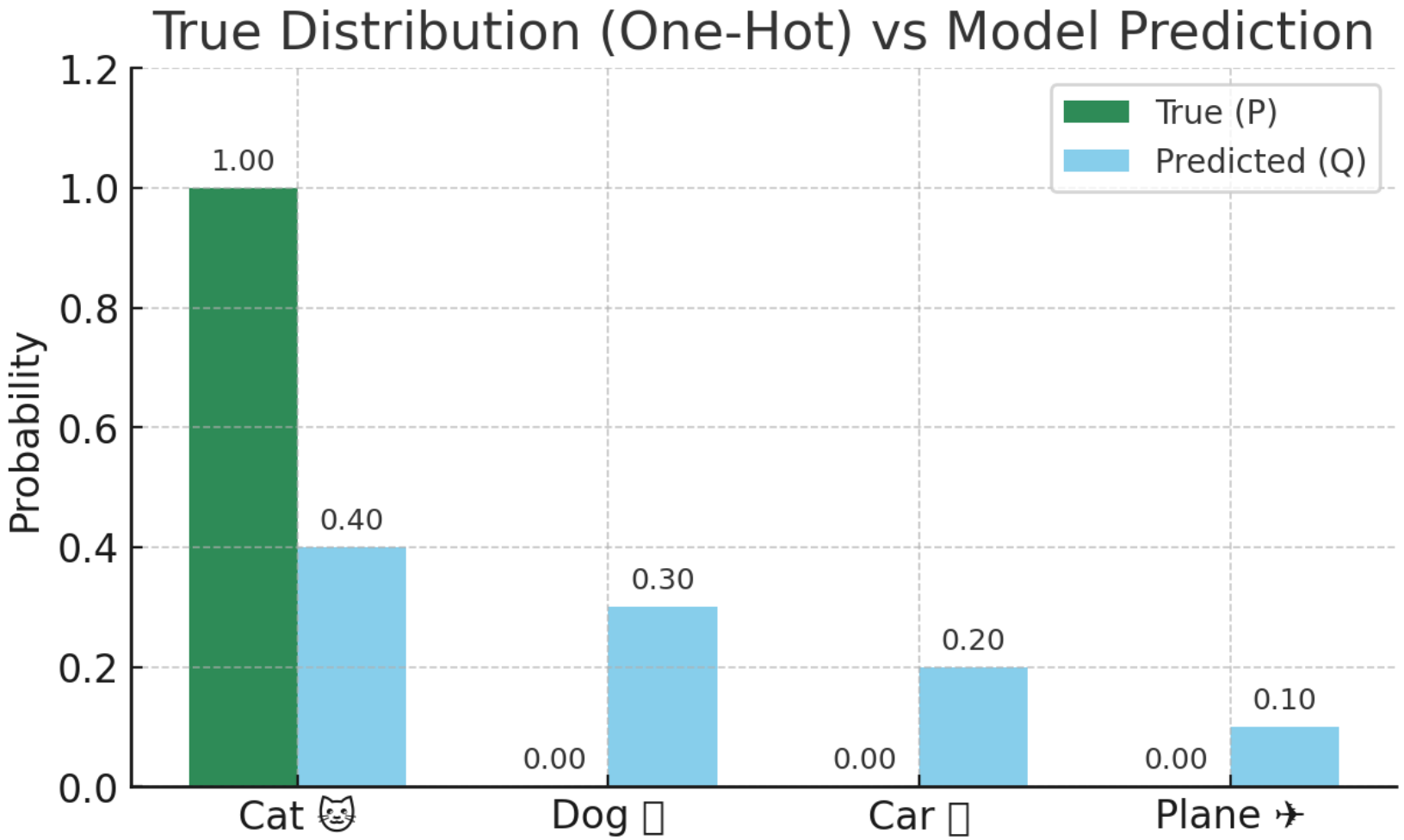
If $P = [1, 0, 0, 0]$ and model says $Q = [0.9, 0.05, 0.03, 0.02]$:

Problems with MSE in classification:

- Doesn’t consider/respect probabilities
- **Treats outputs as continuous regression, not categorical choices**.

If $P = [1, 0, 0, 0]$ and model says $Q = [0.9, 0.05, 0.03, 0.02]$:

$$\sum q_i = 1$$



- Labels are encoded as **one-hot vectors**.
- Predictions are given as **probability distributions**.
- The loss function must measure the gap between the two.

General Problem: Comparing Two Distributions

- $P(x)$ = the **true distribution** (ground truth, nature, teacher).
- $Q(x)$ = the **model's distribution** (approximation, guess, student).

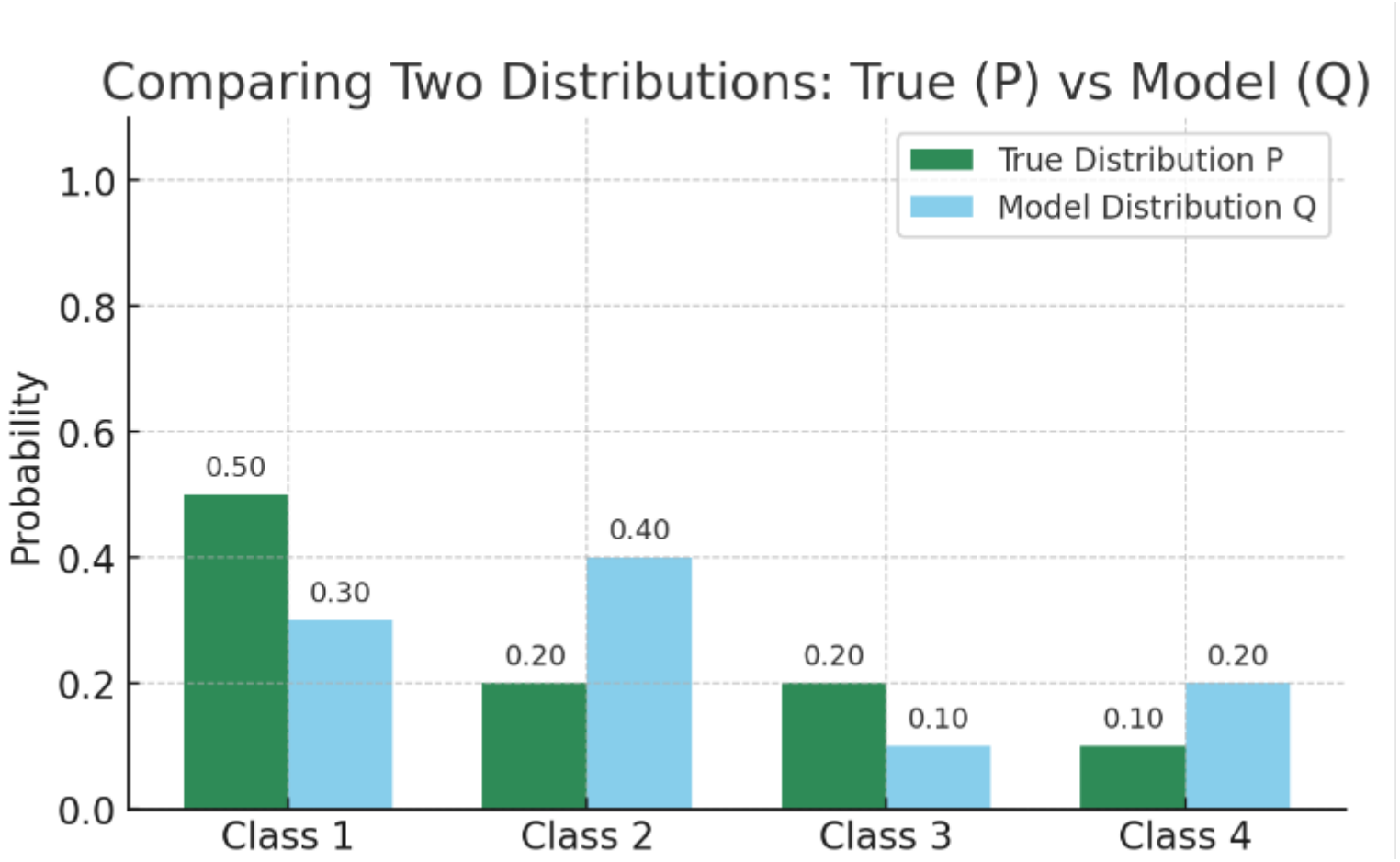
👉 The **problem** is:

How do we measure how close or far Q is from P?

"Comparing two distributions means quantifying how inefficient, uncertain, or different our model's belief Q is when the world runs according to P."

"How should we measure the difference between P and Q?"

Given two distributions P (truth) and Q (model), find a principled measure of their difference that captures uncertainty and mismatch in a meaningful way.



Teacher–Student Analogy

Who won the **2024 Nobel Prize in Physics** for foundational contributions to machine learning?

Options:

- **A)** Geoffrey Hinton ✅ (correct)
- **B)** Yann LeCun
- **C)** Yoshua Bengio
- **D)** Demis Hassabis

🔹 Teacher's Truth Distribution (P)

Since only one option is correct (A), the teacher encodes:

$$P = [1, 0, 0, 0]$$

Different categories of student outputs with just their probability distributions

1. **Confident Correct**

$$Q = [0.90,\ 0.05,\ 0.03,\ 0.02]$$

2. **Correct but Not Confident**

$$Q = [0.40,\ 0.30,\ 0.20,\ 0.10]$$

3. **Confident Wrong**

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

Completely Confused (Uniform)

$$Q = [0.25,\ 0.25,\ 0.25,\ 0.25]$$

Setup:

- Question: "Who won the 2024 Nobel Prize in Physics for ML?"
- Correct answer = **A) Geoffrey Hinton** ✅.
- Students give probability distributions Q.
- Teacher's job = decide *how strongly to penalize each student*.

1. Confident Correct

$$Q = [0.90,\ 0.05,\ 0.03,\ 0.02]$$

- Student puts very high confidence on the correct answer.
- **Teacher's response:** *Give only a very small penalty* → "You're doing well, just refine a bit."

2. Correct but Not Confident

$$Q = [0.40,\ 0.30,\ 0.20,\ 0.10]$$

- Student picks the correct answer but spreads probability across others.
- **Teacher's response:** *Give a medium penalty* → "You got it, but you need more confidence in the truth."

3. Confident Wrong

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

- Student is very sure about the wrong option.
- **Teacher's response:** *Give a very large penalty* → "You're very wrong and too confident — need strong correction."

Desired teaching strategy:

- Confident correct → small penalty.
- Unsure correct → medium penalty.
- Confident wrong → large penalty.

- **Reward confident correct.**
- **Give some penalty for lack of confidence.**
- **Punish confident wrong answers severely.**

Agenda

- Complete module 2
- Discussion on Mid-term topic

Recap

- Prediction of the model is a class probability distribution (Q)
- Initially predicted distribution (Q) is very poor. Hence learning algorithm should penalize the model appropriately, so that the model parameters can be updated significantly using Gradient Descent.
- To penalize, we need a measure of deviation (D) between True distribution (P) and the predicted distribution (Q) → MSE (not a good option)
- D(P,Q) → computed based on Uncertainty measurement in Information theory (Entropy)
- **ML terminology → D(P,Q) → Loss function**

$$P = [1, 0, 0, 0]$$

$$Q = [0.4, 0.3, 0.2, 0.1]$$

Loss function checks:

"How well does Q align with P?"

Teacher–Student Analogy

Who won the **2024 Nobel Prize in Physics** for foundational contributions to machine learning?

Options:

- **A)** Geoffrey Hinton ✅ (correct)
- **B)** Yann LeCun
- **C)** Yoshua Bengio
- **D)** Demis Hassabis

🔹 Teacher's Truth Distribution (P)

Since only one option is correct (A), the teacher encodes:

$$P = [1, 0, 0, 0]$$

Different categories of student outputs with just their probability distributions

1. **Confident Correct**

$$Q = [0.90,\ 0.05,\ 0.03,\ 0.02]$$

2. **Correct but Not Confident**

$$Q = [0.40,\ 0.30,\ 0.20,\ 0.10]$$

3. **Confident Wrong**

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

Completely Confused (Uniform)

$$Q = [0.25,\ 0.25,\ 0.25,\ 0.25]$$

Desired teaching strategy:

- Confident correct → small penalty.
- Unsure correct → medium penalty.
- Confident wrong → large penalty.

- **Reward confident correct.**
- **Give some penalty for lack of confidence.**
- **Punish confident wrong answers severely.**

***We need a mathematical way* to formalize this penalty.**

Defining Student's Knowledge (Information Theory View)

1. Teacher–Student Setup

- Teacher asks a **4-option MCQ**.
- Student answers with a probability distribution:

$$Q = [q_1, q_2, q_3, q_4], \quad \sum_i q_i = 1$$

This distribution reflects **what the student knows** (their confidence levels).

2. Linking Knowledge ↔ Uncertainty

- A student who knows the answer well will put **high probability on one option** → less uncertainty.
- A confused student will spread probabilities across options → more uncertainty.

👉 So the **amount of knowledge** a student has can be seen as the **inverse of their uncertainty**.

In information theory, this “uncertainty of a distribution” is measured by **entropy**:

$$H(Q) = -\sum_{i=1}^{4} q_i \log q_i$$

- This is the **expected uncertainty** of the student’s prediction over all possible options.
- It quantifies how much “information” is still missing for the student to solve the problem.

Should We Define the Penalty This Way?

Penalty = Student's Uncertainty – Teacher's Uncertainty

1. Student's Uncertainty = Entropy of Student's Distribution

$$H(Q) = -\sum_i q_i \log q_i$$

This measures **how uncertain the student feels** about their own answer.

2. Teacher's Uncertainty = Entropy of True Distribution

$$H(P) = -\sum_i p_i \log p_i$$

This measures **how uncertain the teacher is about the truth**.

Is it a correct penalty for improving the student (model)?

Penalty = Student's Uncertainty – Teacher's Uncertainty

$$= H(Q) - H(P)$$

Is it a correct penalty for improving the student (model)?

👉 It ignores the **alignment** between student's belief Q and truth P.

Penalty = Student's Uncertainty – Teacher's Uncertainty

$$= H(Q) - H(P)$$

What $H(Q)$ Measures

$$H(Q) = -\sum_i q_i \log q_i$$

This is the **uncertainty of the student's belief**.

It only depends on how **spread out** the probabilities are in Q.

It does **not** look at which class is actually correct.

It only measures *internal confusion*, not *alignment with teacher's truth*.

- If Q is **uniform** (e.g., $Q = [0.25, 0.25, 0.25, 0.25]$), then $H(Q)$ is maximal → student is "totally confused."
- If Q is **peaked** (e.g., $Q = [0.9, 0.05, 0.03, 0.02]$), then $H(Q)$ is low → student is confident.

So entropy $H(Q)$ measures **how spread out the student's belief is**.

2. Problem: Same Entropy, Different Correctness

Two students can have the same entropy but very different relation to the truth.

- **Case A (Correct Leaning):**

$$Q = [0.6,\ 0.4,\ 0.0,\ 0.0], \quad P = [1, 0, 0, 0]$$

  Student leans toward the correct answer (60%).

- **Case B (Wrong Leaning):**

$$Q = [0.4,\ 0.6,\ 0.0,\ 0.0], \quad P = [1, 0, 0, 0]$$

  Student leans toward the wrong answer (60%).

👉 Both distributions have the same entropy $H(Q)$ (they're equally "uncertain").

But from the teacher's perspective, **Case A is much better than Case B**.

- **Entropy $H(Q)$:** only tells us *how confused the student is.*
- **Cross-Entropy $H(P, Q)$:** checks if the **student's belief lines up with teacher's truth**.

$$H(P, Q) = -\sum_{i} P(i) \log Q(i)$$

👉 This focuses **only on the probability the student assigns to the correct option.**

"$H(Q)$ measures only the student's inner confusion. It ignores whether their belief aligns with the truth P. That's why we need cross-entropy or KL divergence, which compare the student's belief Q directly with the teacher's truth P."

KL Divergence:

$$D_{KL}(P \parallel Q) = H(P, Q) - H(P)$$

"Subtracting uncertainties $H(Q) - H(P)$ only tells us how confused the student is compared to the teacher, not whether their answers align with the truth. That's why we use **KL divergence**, which directly measures the mismatch between the student's knowledge distribution and the teacher's truth distribution."

Kullback–Leibler divergence

In mathematical statistics, the **Kullback–Leibler** (**KL**) **divergence** (also called **relative entropy** and **I-divergence**[1]), denoted $D_{\mathrm{KL}}(P \parallel Q)$, is a type of statistical distance: a measure of how one reference probability distribution P is different from a second probability distribution Q.[2][3] Mathematically, it is defined as

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x)\ \log\left(\frac{P(x)}{Q(x)}\right).$$

- ML task: *Object classification with 4 classes (say Cat 🐱, Dog 🐶, Car 🚗, Plane ✈️).*
- Each data point: (x, y).
- Label y represented as one-hot probability distribution:

$$P = [1, 0, 0, 0] \quad \text{if Cat is true.}$$

- Model predicts probabilities:

$$Q = [q_1, q_2, q_3, q_4], \quad \sum q_i = 1$$

KL Divergence in ML

In machine learning classification:

- P = true distribution of labels (ground truth, teacher).
  - Usually one-hot (e.g., Cat = [1,0,0,0])
- Q = model's predicted distribution (student's belief).

The **KL divergence** between P and Q is:

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} = H(P,Q) - H(P)$$

- In classification with one-hot labels ($H(P) = 0$):

$$D_{KL}(P \parallel Q) = H(P,Q)$$

👉 So in supervised ML, **KL divergence = Cross-Entropy**.

Cross-Entropy Loss Formula

For one training example:

$$L = H(P, Q) = -\sum_{i=1}^{C} P(i) \log Q(i)$$

- P = true distribution (one-hot in supervised learning)
- Q = model's predicted distribution
- C = number of classes

Suppose we are classifying **images into 4 classes**:

- Cat 🐱 (true)
- Dog 🐶 (similar to Cat)
- Car 🚗 (totally different)
- Airplane ✈️ (totally different)

So the **true distribution** is:

$$P = [1, 0, 0, 0] \quad (\text{Cat is correct})$$

- True label = Class 1 (e.g., **Cat**)
- So:

$$P = [1, 0, 0, 0]$$

- Model prediction:

$$Q = [0.7, 0.1, 0.1, 0.1]$$

Apply Formula

$$L = -(1 \cdot \log 0.7 + 0 \cdot \log 0.1 + 0 \cdot \log 0.1 + 0 \cdot \log 0.1)$$

Natural logarithm (base e) is used by default.

$$\boxed{L \approx -(-0.357) = 0.357}$$

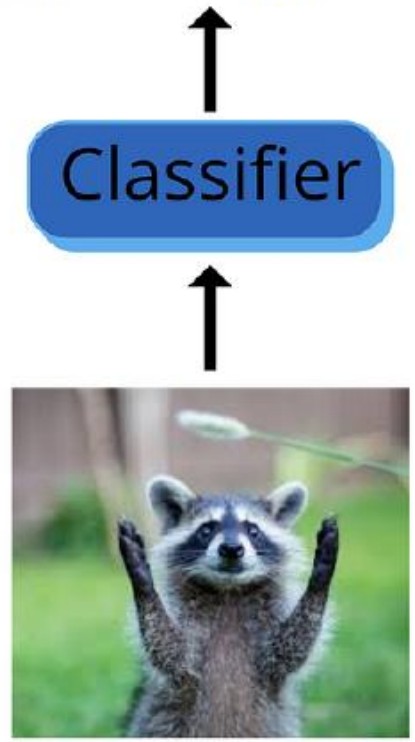
- If the model had predicted $Q = [0.9, 0.05, 0.03, 0.02]$, loss = $-\log 0.9 = 0.105$ → very small.
- If it predicted $Q = [0.25, 0.25, 0.25, 0.25]$, loss = $-\log 0.25 = 1.386$ → larger.
- If it predicted $Q = [0.01, 0.9, 0.05, 0.04]$, loss = $-\log 0.01 = 4.605$ → huge penalty.

"In supervised ML with one-hot labels, CE loss reduces to $-\log$ of the probability the model assigns to the true class."

	CAT	DOG	FOX	RED PANDA	RACCOON	COW	BEAR
True Distribution:	0%	0%	0%	0%	100%	0%	0%
Predicted Distribution:	5%	2%	45%	10%	30%	1%	2%

Cross-Entropy is -1*log(0.3) = — log(0.3) = 1.203

	CAT	DOG	FOX	RED PANDA	RACCOON	COW	BEAR
True Distribution:	0%	0%	0%	0%	100%	0%	0%
Predicted Distribution:	5%	2%	45%	10%	30%	1%	2%



Characteristics of Cross-Entropy (CE) Loss

1. Non-Negative

$$CE(P, Q) = -\sum_i P(i) \log Q(i) \quad \geq 0$$

- Loss is always ≥ 0.
- Minimum is 0 when the model predicts the truth with probability 1.

2. Penalizes Confident Wrong Predictions Heavily

- If model says true class probability $Q(y)$ is very small (e.g., 0.01),

$$L = -\log 0.01 = 4.605$$

"Confidently wrong is worse than being unsure."

- **Confident & Correct** → CE very small.
- **Correct but not confident** → CE moderate.
- **Completely confused (uniform)** → CE larger.
- **Confident & Wrong** → CE huge.

Insensitive to Inter-Class Similarity

- CE only looks at the probability of the true class:

$$L = -\log Q(y_{\text{true}})$$

- Example:
  - Predicting **Dog** instead of **Cat** (similar animals) is penalized the same as predicting **Airplane** instead of **Cat**.
- 👉 CE does not account for relationships between classes.

Example: CE Ignores Class Relationships

Suppose we are classifying **images into 4 classes**:

- Cat 🐱 (true)
- Dog 🐶 (similar to Cat)
- Car 🚗 (totally different)
- Airplane ✈️ (totally different)

So the **true distribution** is:

$$P = [1, 0, 0, 0] \quad (\text{Cat is correct})$$

$$P = [1, 0, 0, 0] \quad \text{(Cat is correct)}$$

Case 1: Predicts Dog with High Confidence

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

- CE $= -\log 0.05 = 2.996.$

Case 2: Predicts Airplane with High Confidence

$$Q = [0.05,\ 0.02,\ 0.03,\ 0.90]$$

- CE $= -\log 0.05 = 2.996.$

$$P = [1, 0, 0, 0] \quad \text{(Cat is correct)}$$

Case 1: Predicts Dog with High Confidence

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

- CE = $-\log 0.05 = 2.996$.

Case 2: Predicts Airplane with High Confidence

$$Q = [0.05,\ 0.02,\ 0.03,\ 0.90]$$

- CE = $-\log 0.05 = 2.996$.

Observation

- In **both cases**, CE loss is **exactly the same** (2.996).
- But intuitively, misclassifying **Cat → Dog** feels "less wrong" than **Cat → Airplane**.

“Cross-Entropy loss only cares about the probability on the true class. It doesn’t recognize that some mistakes (Cat → Dog) are less severe than others (Cat → Airplane).”

Case 1: Predicts Dog with High Confidence

$$Q = [0.05,\ 0.90,\ 0.03,\ 0.02]$$

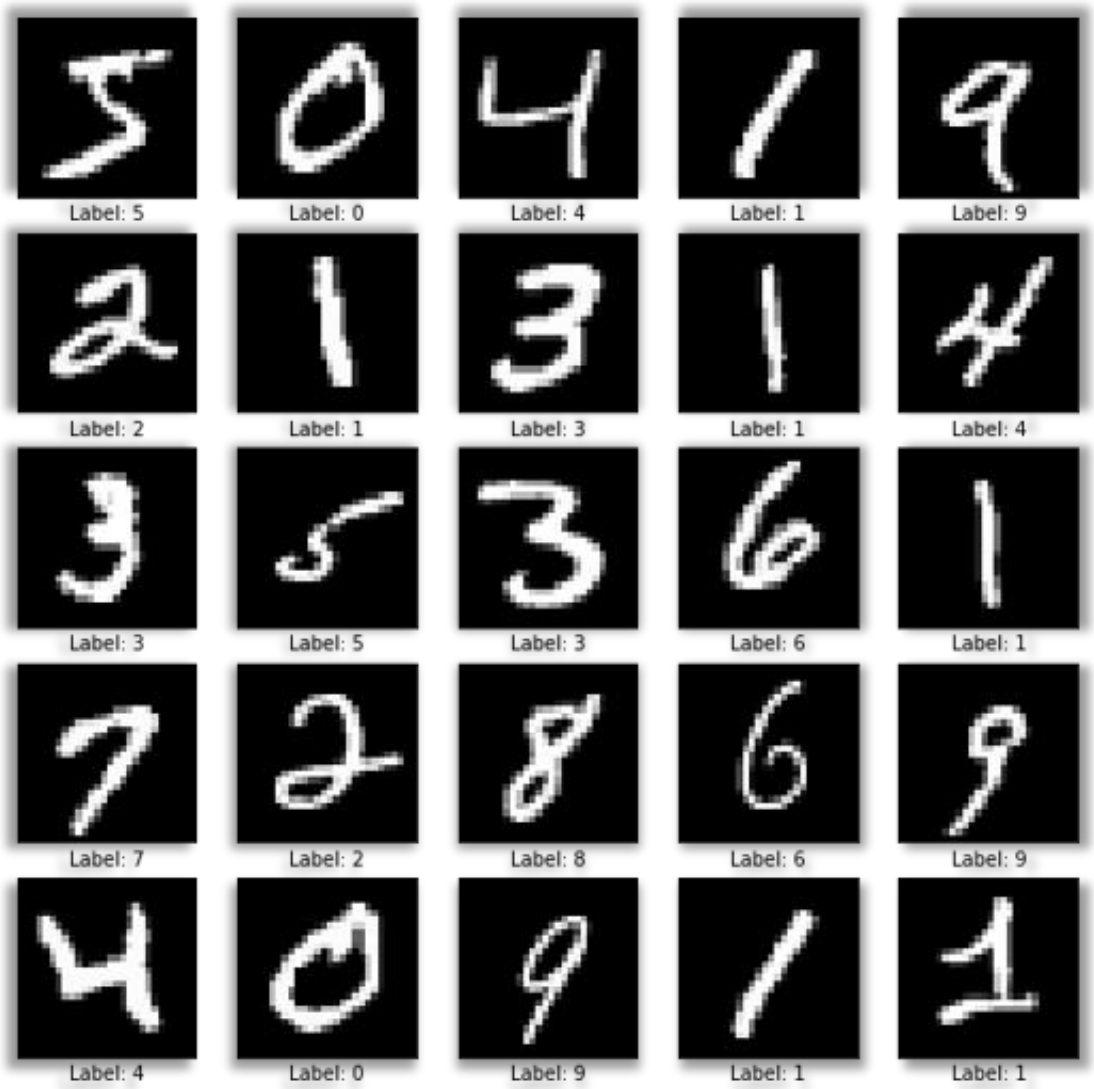
- $\text{CE} = -\log 0.05 = 2.996.$

Case 2: Predicts Airplane with High Confidence

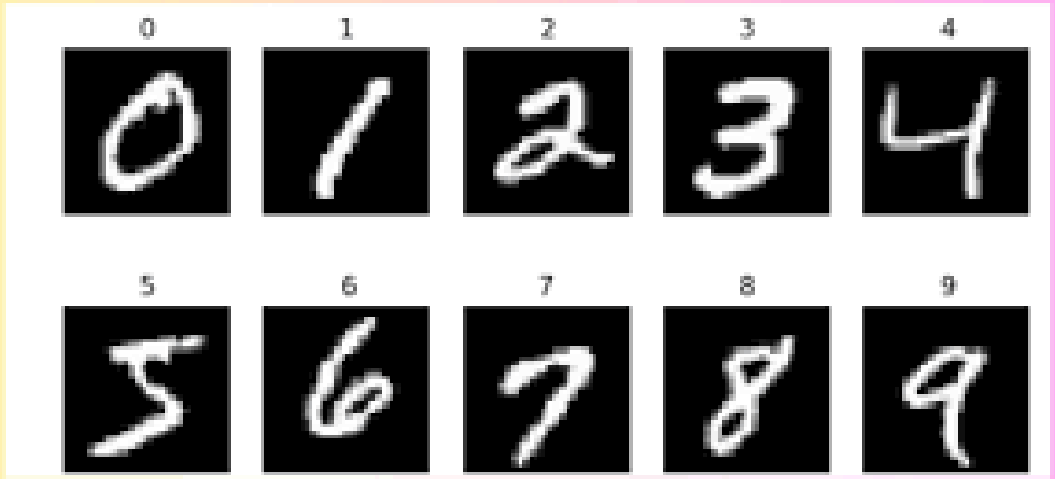
$$Q = [0.05,\ 0.02,\ 0.03,\ 0.90]$$

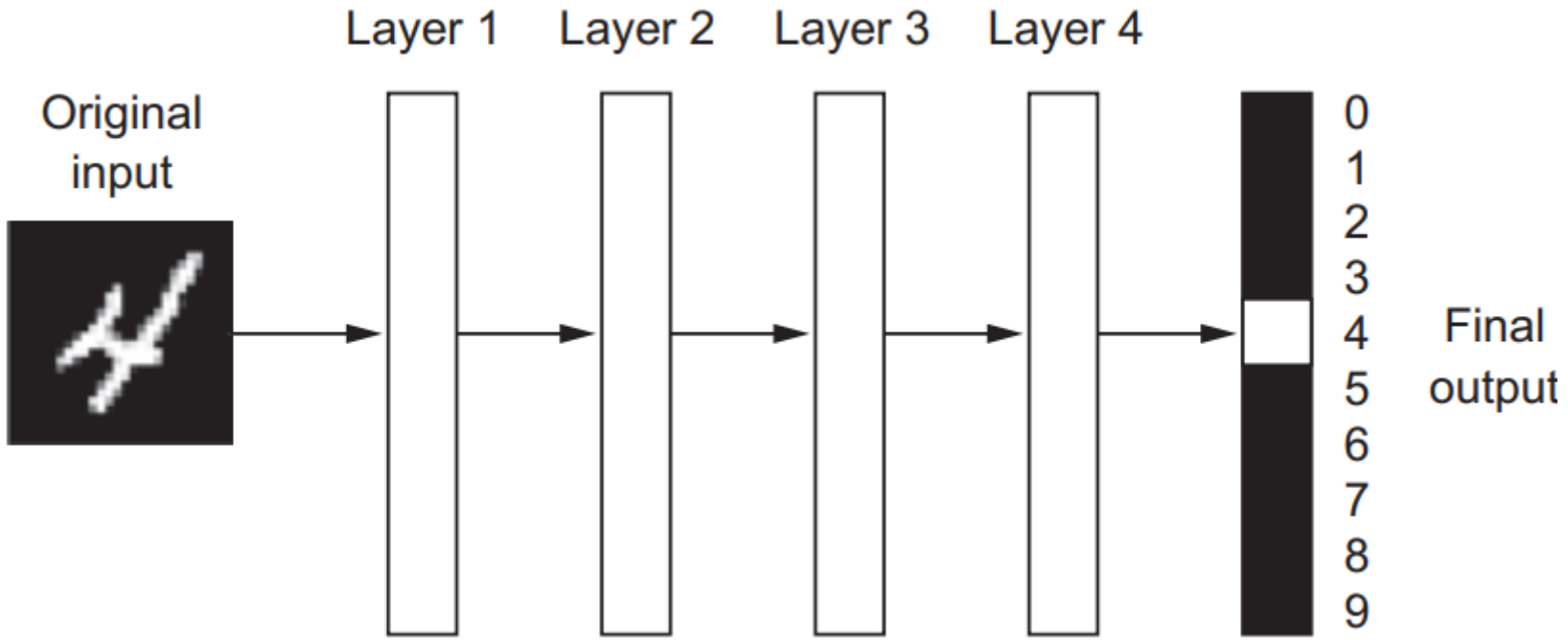
- $\text{CE} = -\log 0.05 = 2.996.$

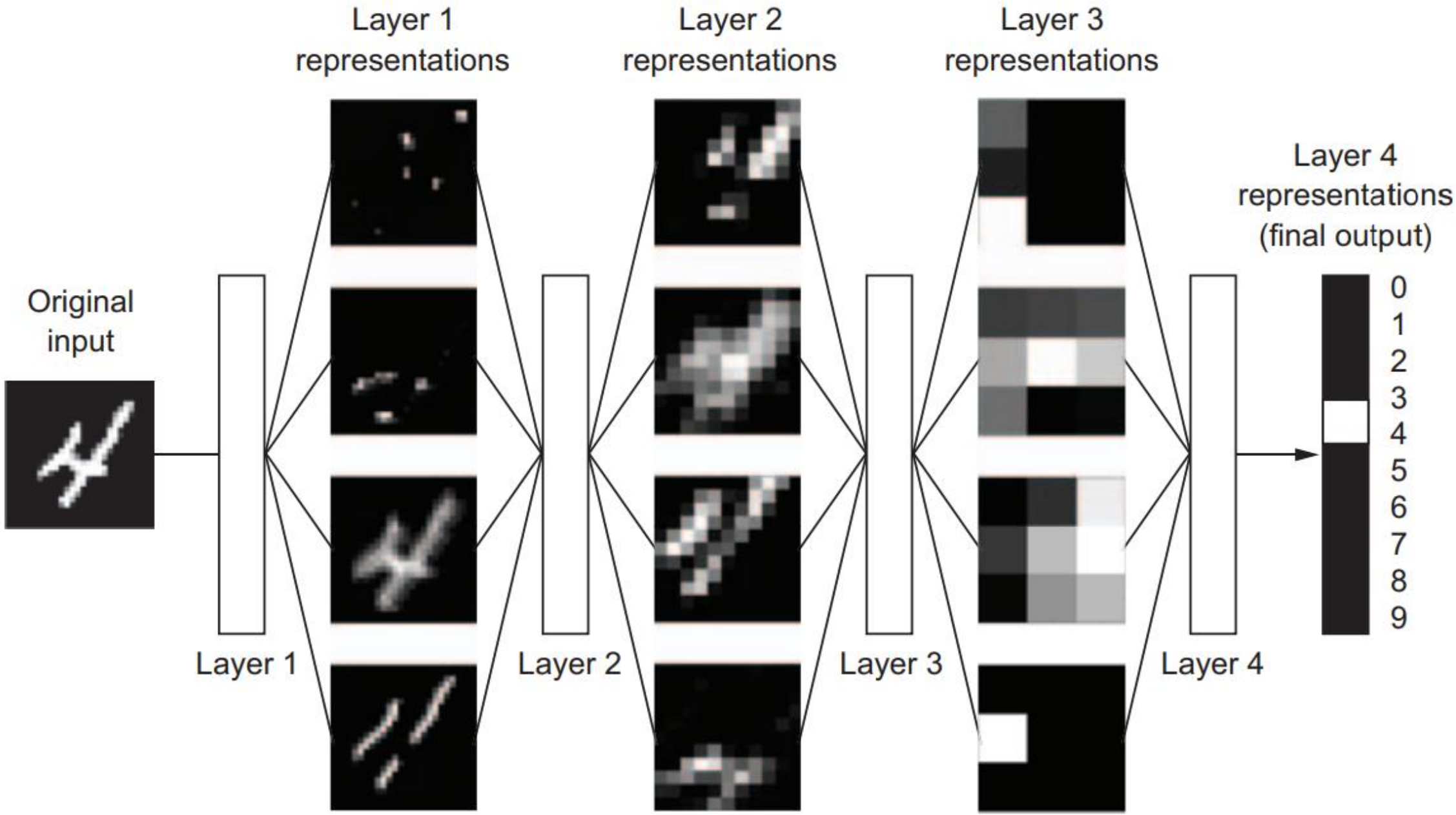
Deep learning: Handwritten Digit Image classifier



MNIST Handwritten Digit Recognition using ANN






Layer 1 representations
Layer 2 representations
Layer 3 representations
Layer 4 representations (final output)
Original input
Layer 1
Layer 2
Layer 3
Layer 4
0
1
2
3
4
5
6
7
8
9

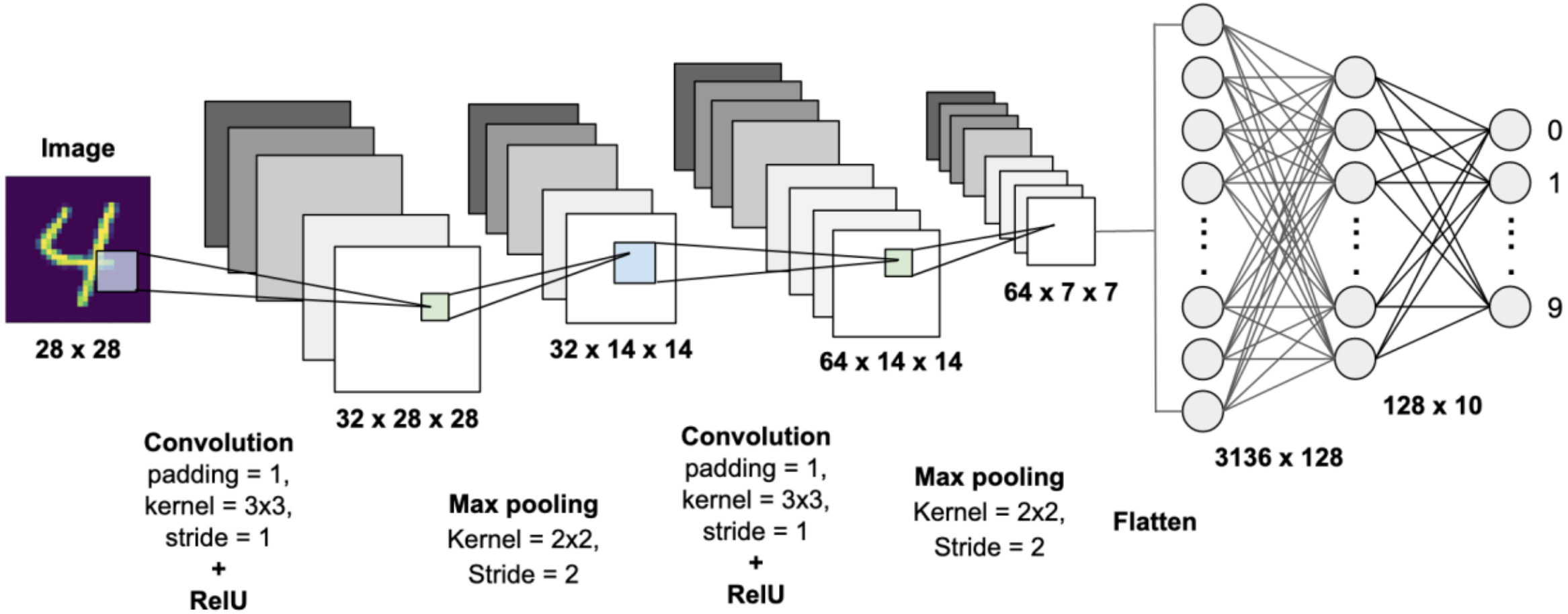
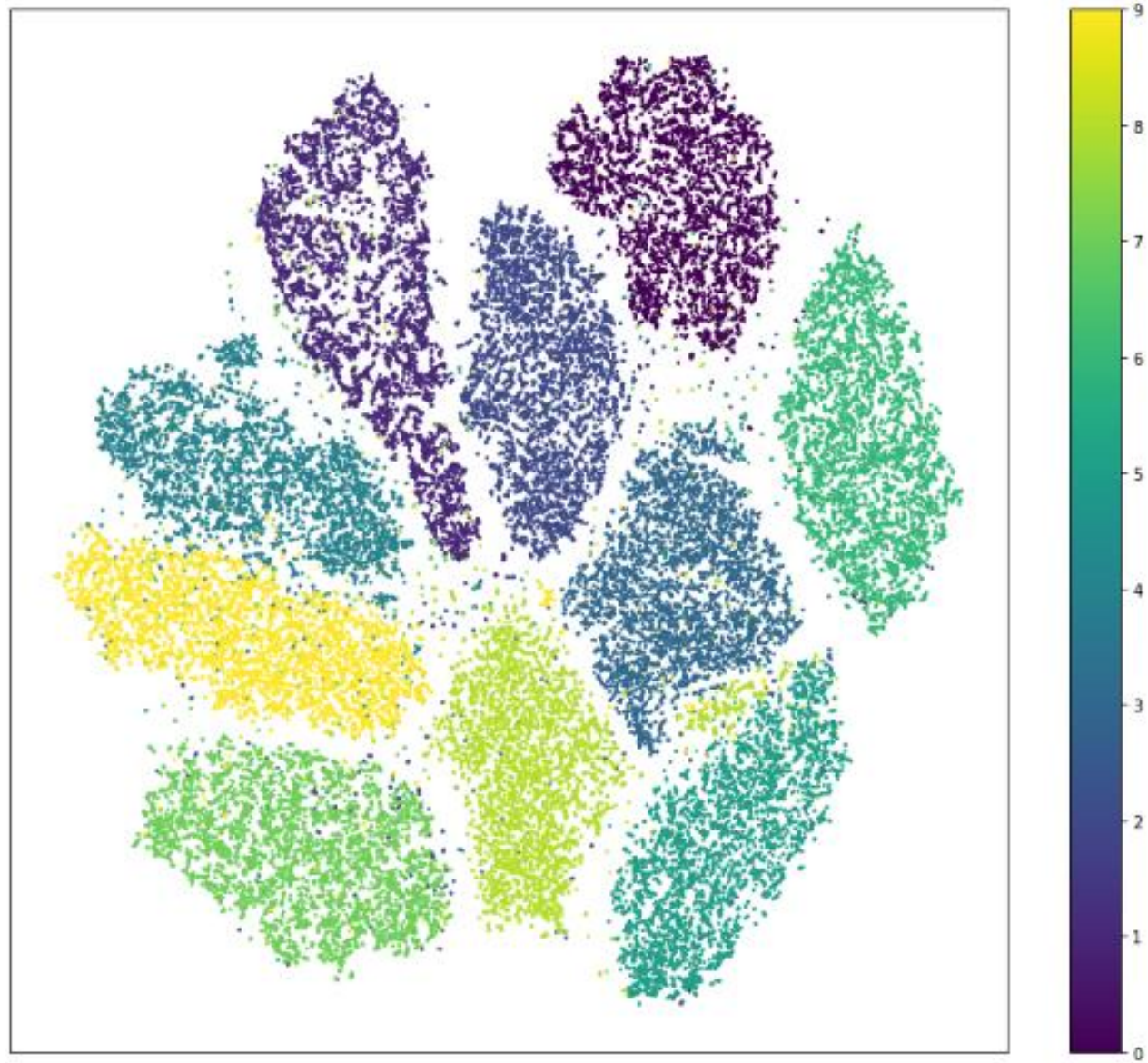
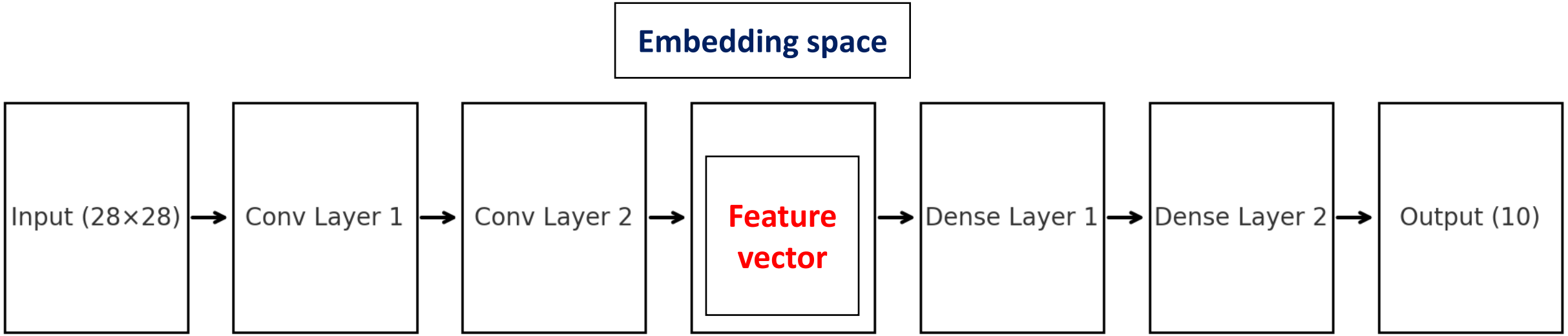


Image
28 x 28
32 x 28 x 28
32 x 14 x 14
64 x 14 x 14
64 x 7 x 7
3136 x 128
128 x 10
0
1
9
Convolution
padding = 1,
kernel = 3x3,
stride = 1
+
RelU
Max pooling
Kernel = 2x2,
Stride = 2
Convolution
padding = 1,
kernel = 3x3,
stride = 1
+
RelU
Max pooling
Kernel = 2x2,
Stride = 2
Flatten


9
8
7
6
5
4
3
2
1
0

Digit Classification with a Convolutional Neural Network



digit classification CNN (2 conv + 2 dense + output):

$$X \longrightarrow z_1\,(\text{MAC}) \longrightarrow a_1\,(\text{Activation}) \longrightarrow z_2\,(\text{MAC}) \longrightarrow a_2\,(\text{Activation}) \longrightarrow \cdots \longrightarrow z_L \longrightarrow a_L$$

Deep Tensor Transformation Network

1. **Input tensor:**
   $X \in \mathbb{R}^{28\times 28}$
2. **Conv Layer 1:**
   $z_1 = W_1 * X + b_1$
   $a_1 = f(z_1)$ (e.g., ReLU)
3. **Conv Layer 2 (Feature Maps):**
   $z_2 = W_2 * a_1 + b_2$
   $a_2 = f(z_2)$
4. **Embedding Vector** (flattened feature maps):
   $z_3 = W_3 \cdot \mathrm{vec}(a_2) + b_3$
   $a_3 = f(z_3)$
5. **Dense Layer 1:**
   $z_4 = W_4 \cdot a_3 + b_4$
   $a_4 = f(z_4)$
6. **Dense Layer 2 (Output logits):**
   $z_5 = W_5 \cdot a_4 + b_5$

Z5 → simply the raw output from the final linear layer of a neural network

It is called the logit vector.

z_5 is the logit vector of size 10

These are just raw scores from weighted sums:

$$z = W \cdot h + b$$

They can be positive, negative, large, small — **not normalized**.

- *In deep learning, the term logits layer is popularly used for the last neuron layer of neural network for classification task which produces raw prediction values as real numbers ranging from [-infinity, +infinity].*

- *Logits are the raw scores output by the last layer of a neural network. Before activation takes place.*

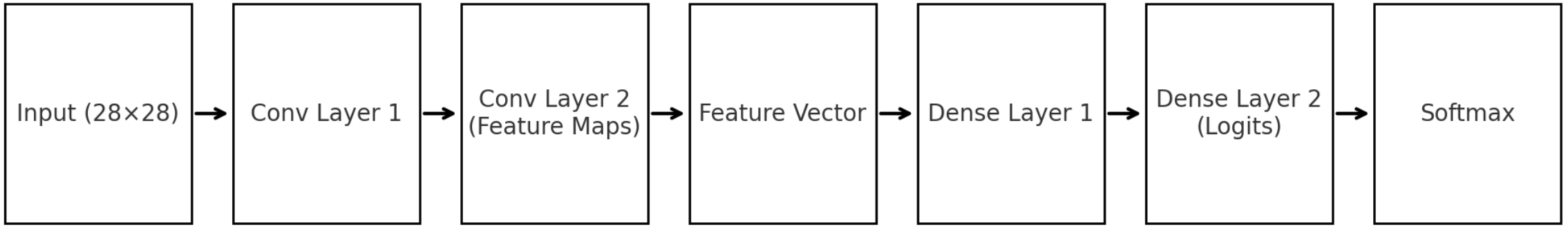
Why is it called logit?

- The term **logit** comes from statistics (logistic regression).
- For a binary classification, the **logit function** is:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

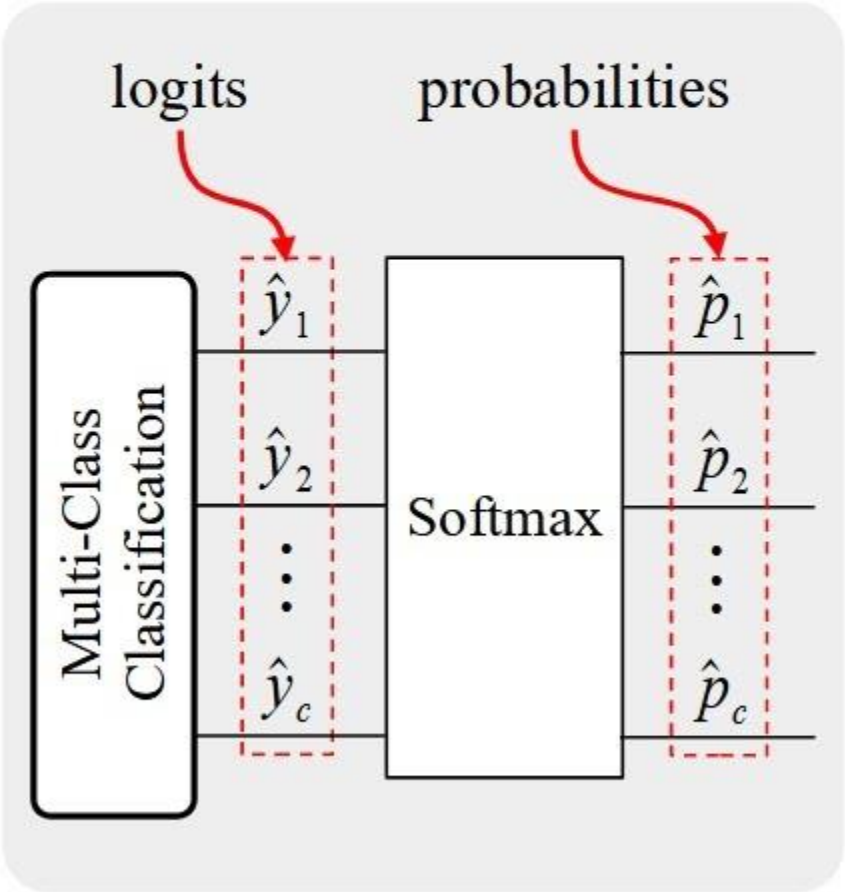
  It maps probabilities $[0, 1]$ back to the real line $(-\infty, +\infty)$.
- In neural nets, the last **linear layer output before applying sigmoid/softmax** is also called a *logit*, because it plays the same role: the unbounded "score" that will be transformed into a probability.
- So in your CNN, z_5 are logits: the raw pre-softmax scores for each class.

Digit Classification CNN (Superficial Block Diagram)



Softmax converts arbitrary outputs (logits) into a valid probability distribution, making classification interpretable and trainable with cross-entropy.

LOGIT AND PROBABILITY



We want to turn raw **logits** $z = [z_1, z_2, \ldots, z_K]$ into a **probability distribution** $p = [p_1, p_2, \ldots, p_K]$ such that:

1. $p_i \geq 0$ (non-negative)
2. $\sum_{i=1}^{K} p_i = 1$ (normalization)
3. Relative magnitude of logits should determine how probability mass is distributed.
    - Larger $z_i \rightarrow$ larger p_i.

Principle of Maximum Entropy

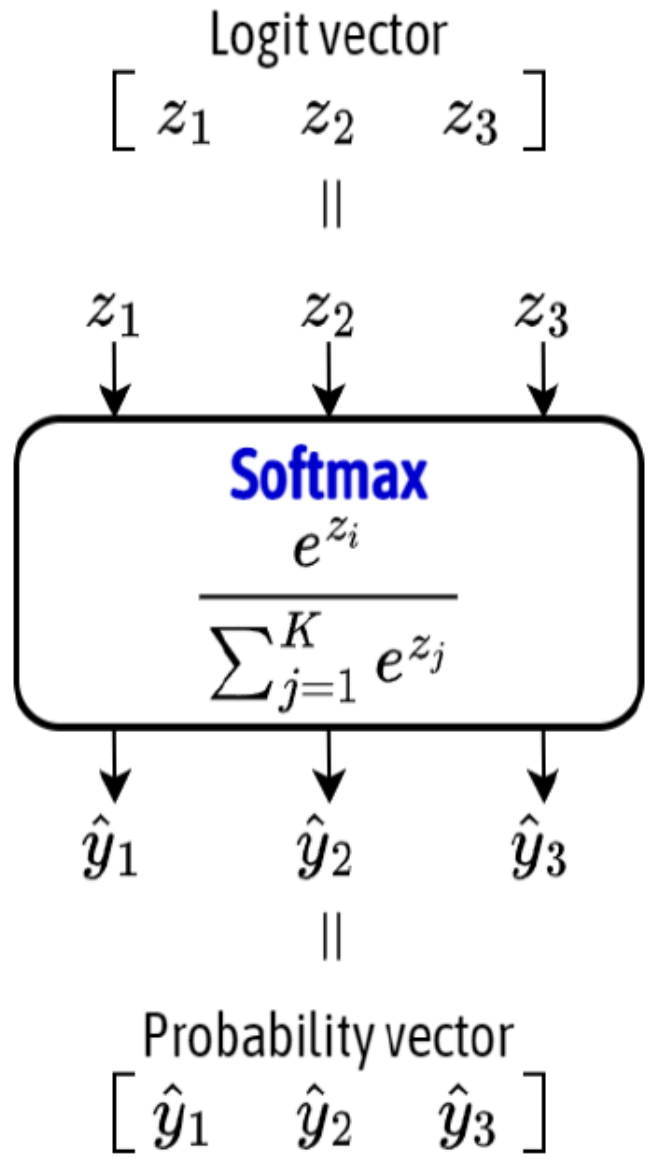
From information theory:

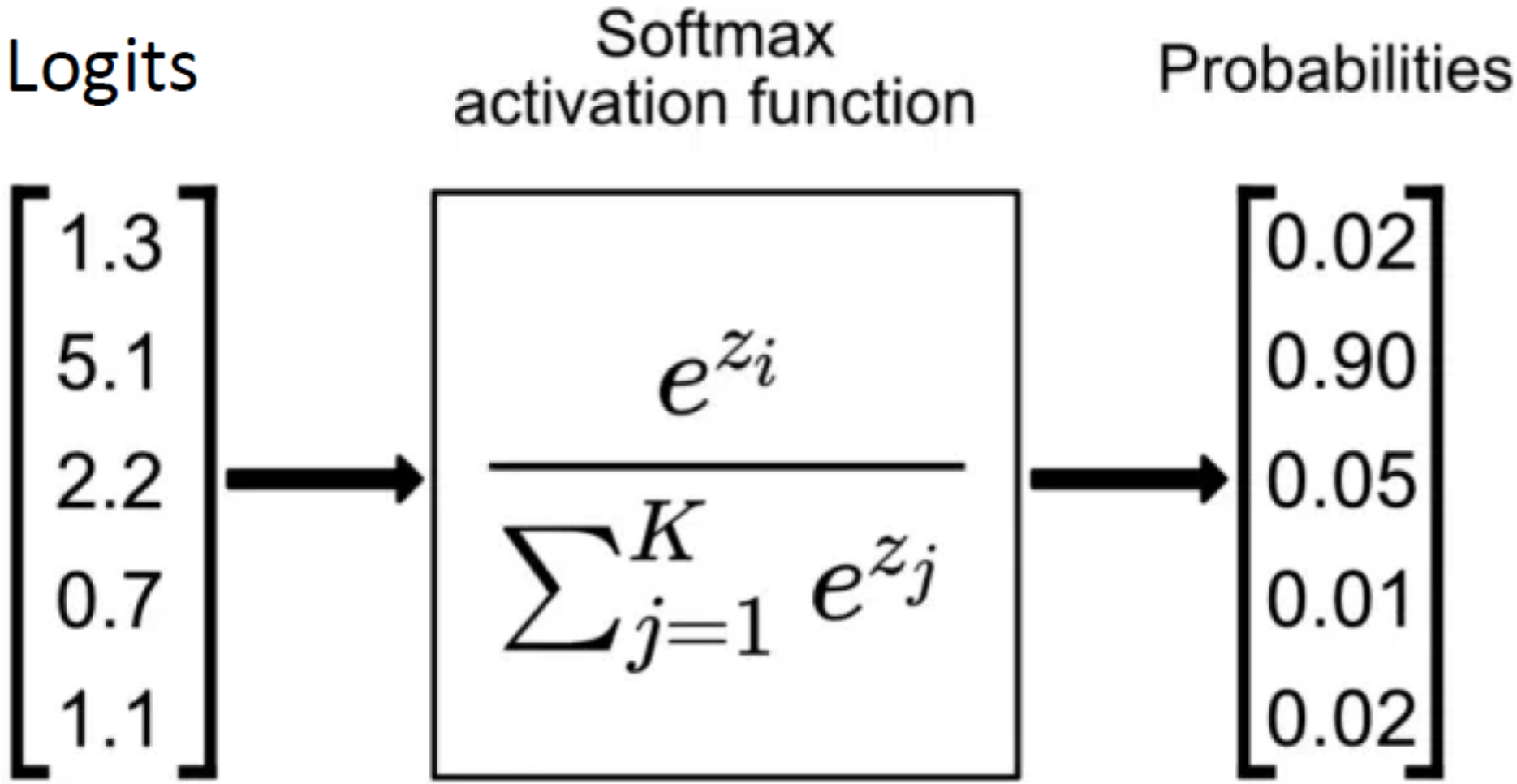
If all we know are the **scores (energies)** z_i, the most natural distribution is the **maximum entropy distribution** under normalization.

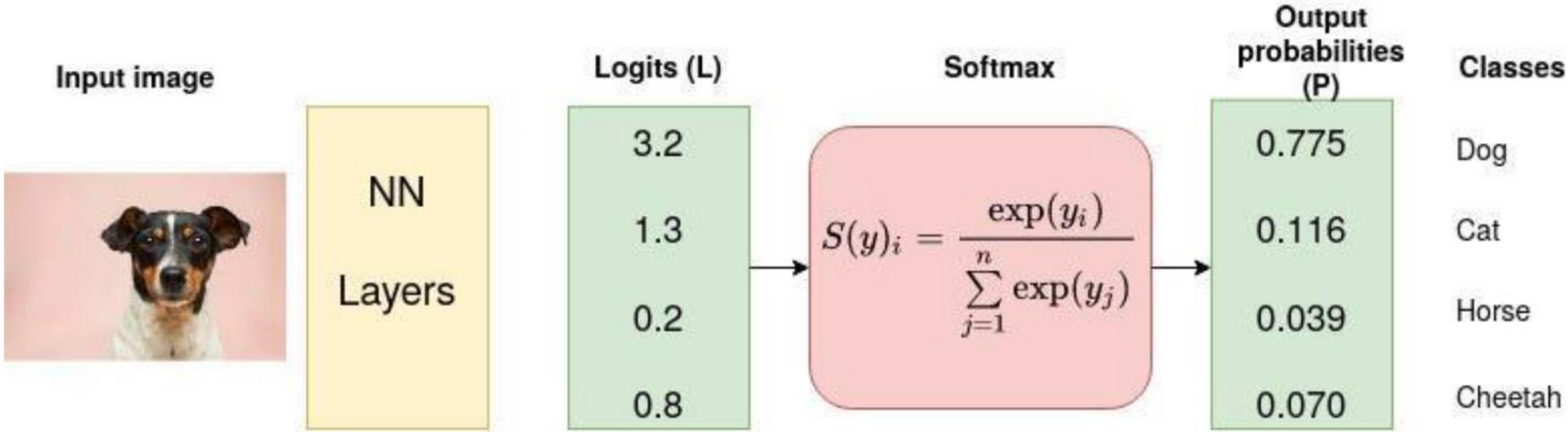
This leads to a **Gibbs (Boltzmann) distribution** in statistical mechanics:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

So **softmax is exactly the Gibbs distribution** with logits as "energies."


Logit vector
$[\ z_1 \quad z_2 \quad z_3\ ]$
z_1 z_2 z_3
Softmax
$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
$\hat{y}_1$ $\hat{y}_2$ $\hat{y}_3$
Probability vector
$[\ \hat{y}_1 \quad \hat{y}_2 \quad \hat{y}_3\ ]$


Logits
Softmax
activation function
Probabilities
1.3
5.1
2.2
0.7
1.1
$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
0.02
0.90
0.05
0.01
0.02



Softmax *is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.*

It is an activation function that scales logits into probabilities. The output of a Softmax is a vector (say v) with probabilities of each possible outcome. The probabilities in vector v sums to one for all possible outcomes or classes.

$$\exp(3.2) = 24.5325$$
$$\exp(1.3) = 3.6693$$
$$\exp(0.2) = 1.2214$$
$$\exp(0.8) = 2.2255$$

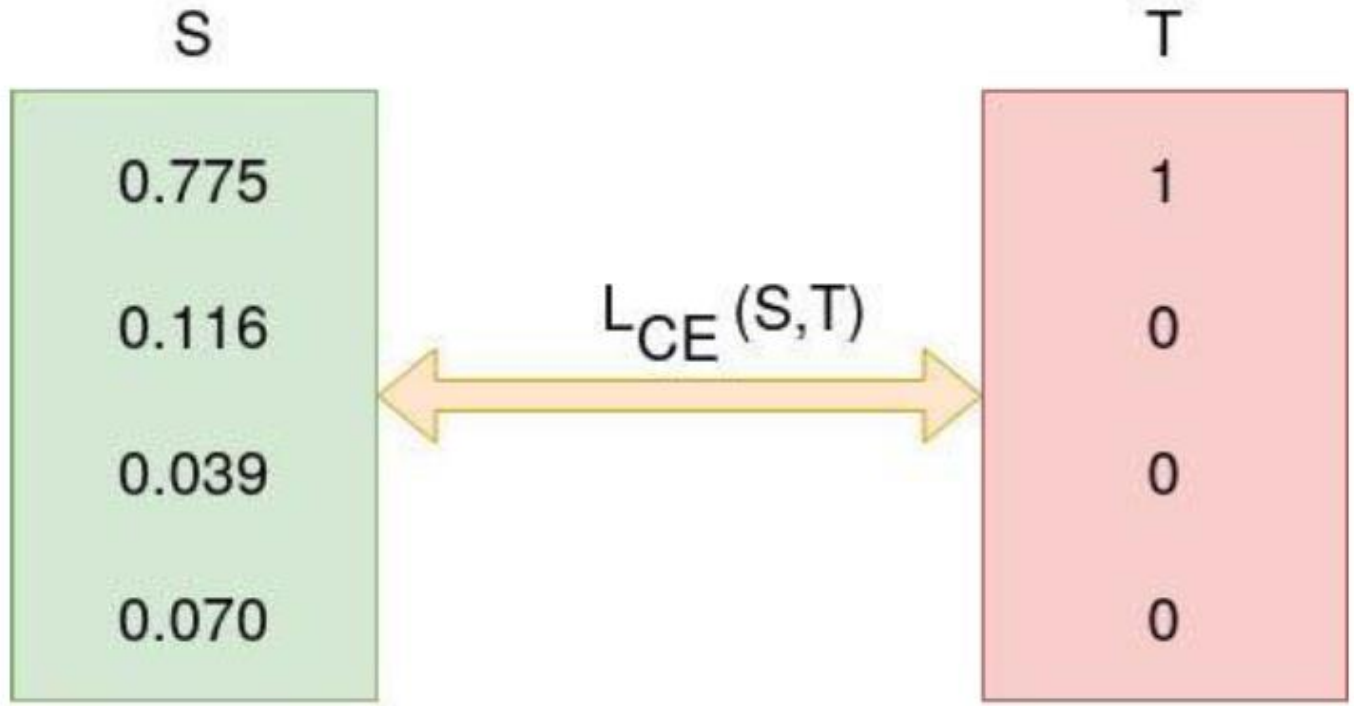
and therefore,

$$\begin{aligned} S(3.2) &= \frac{\exp(3.2)}{\exp(3.2) + \exp(1.3) + \exp(0.2) + \exp(0.8)} \\ &= \frac{24.5325}{24.5325 + 3.6693 + 1.2214 + 2.2255} \\ &= 0.775 \end{aligned}$$

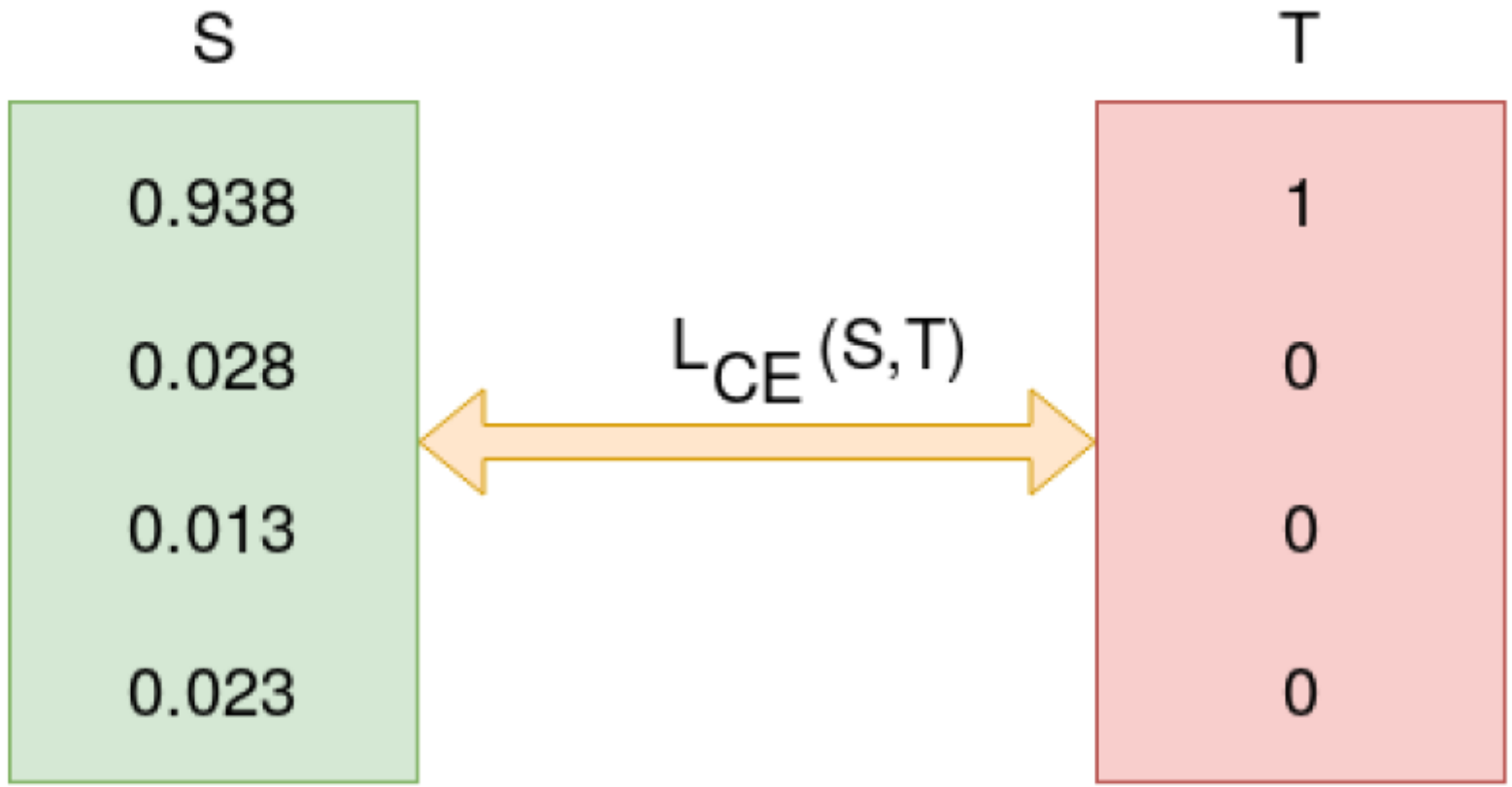
Cross-Entropy Loss Function

$$L_{\mathrm{CE}} = -\sum_{i=1}^{n} t_i \log(p_i), \quad \text{for n classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the i^{th} class.



$$
\begin{aligned}
L_{CE} &= -\sum_{i=1} T_i \log(S_i) \\
&= -\left[1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)\right] \\
&= -\log_2(0.775) \\
&= 0.3677
\end{aligned}
$$



$$L_{CE} = -1 \log_2(0.936) + 0 + 0 + 0$$
$$= 0.095$$

Inference

At test time, we don't need the full probability distribution for evaluation.

We just want the **most likely class**.

1. Softmax Output

In a classification model with C classes, the final layer usually outputs logits $z = [z_1, z_2, \ldots, z_C]$.

The **softmax** converts logits into probabilities:

$$Q(i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

So after softmax:

$$Q = [q_1, q_2, \ldots, q_C], \quad \sum q_i = 1$$

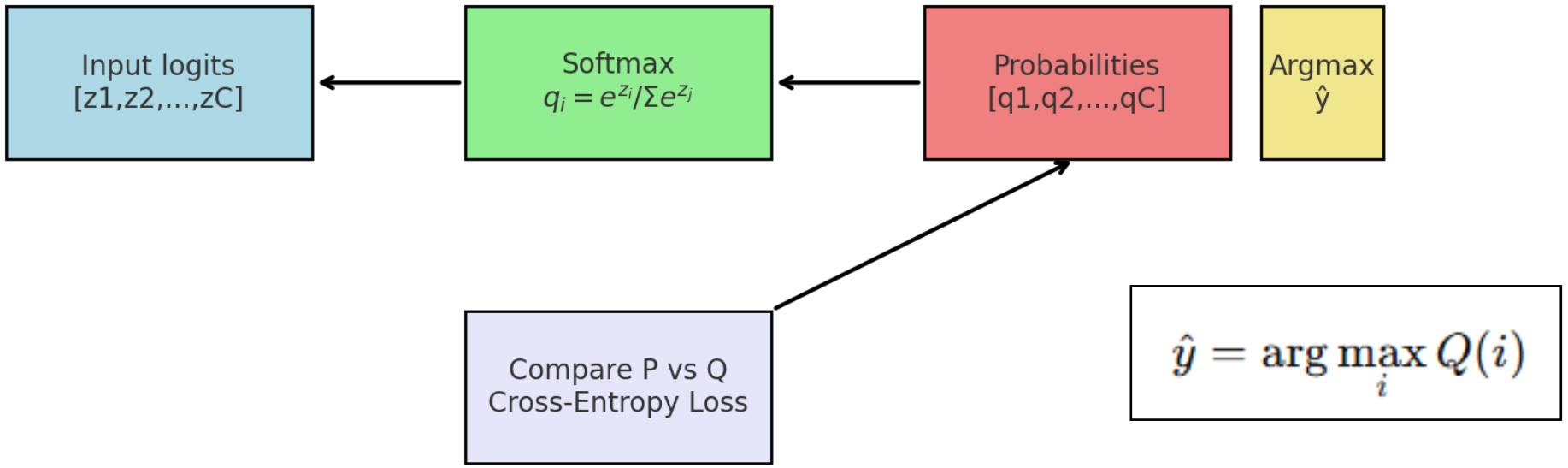
Inference Stage (Making a Prediction)

$$\hat{y} = \arg\max_i Q(i)$$

This picks the index of the class with the **highest probability**.

Argmax & Softmax

- **Logits → Softmax → Probabilities → Argmax (Inference)**
- During **training**, probabilities are compared with the truth P using **Cross-Entropy Loss.**



"Softmax turns logits into probabilities, and argmax picks the most probable class as the model's final answer during inference."

Binary classification

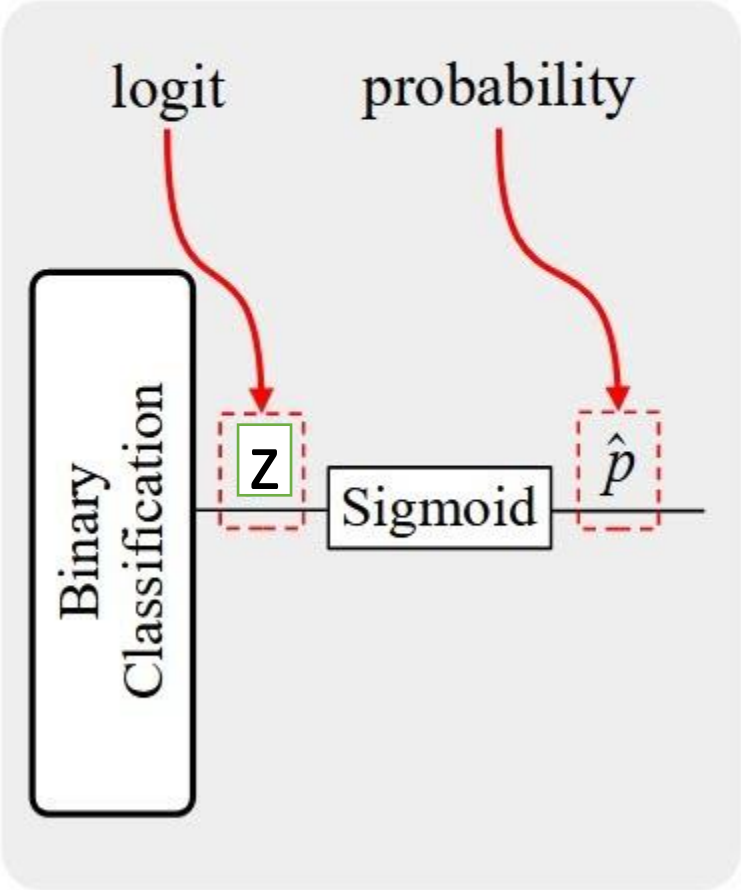
In **binary classification**, each sample belongs to either class **0** or **1**.

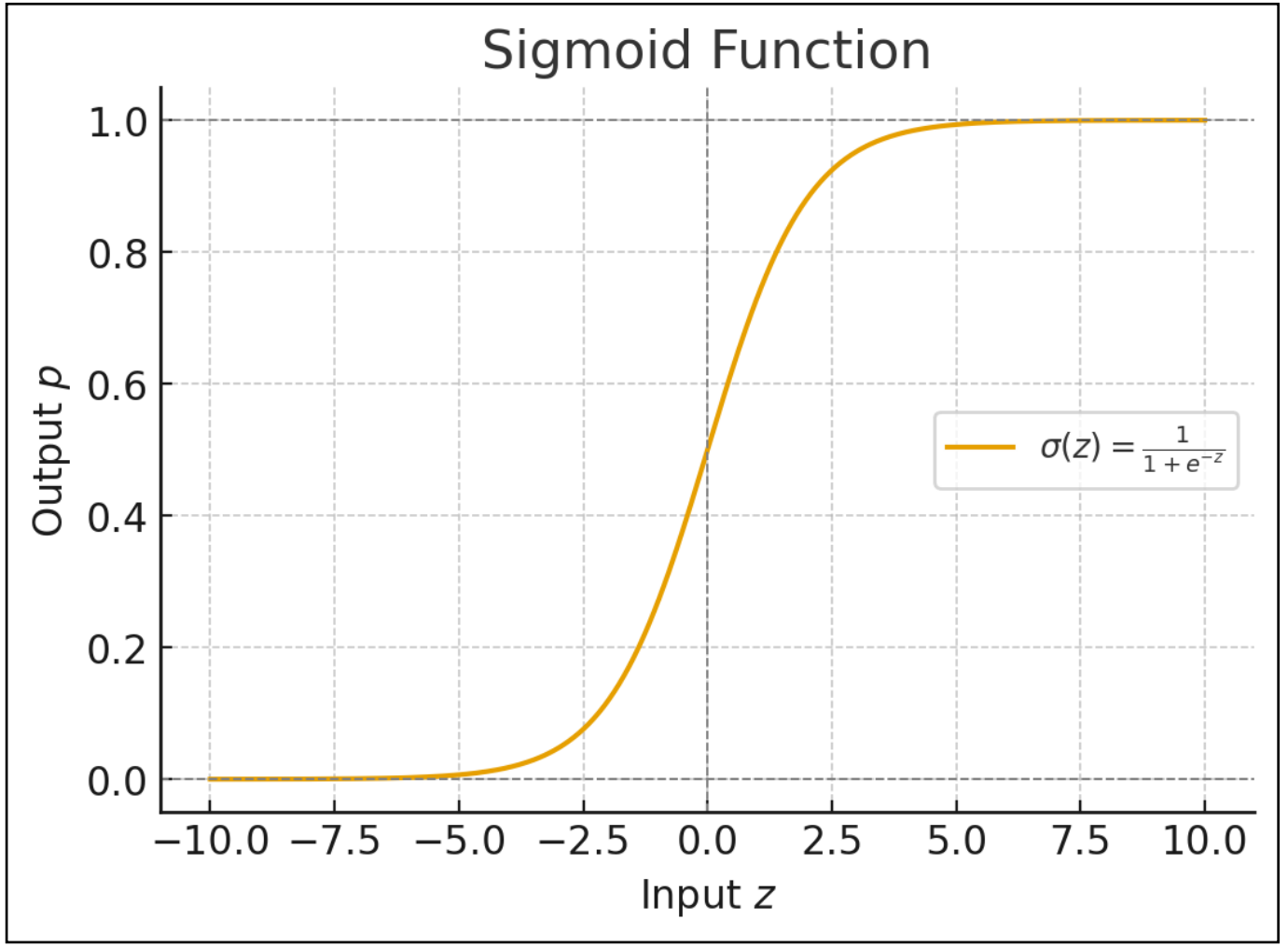
- Model outputs a probability $\hat{y} \in (0, 1)$ (often via **sigmoid** of a logit z).
- True label: $y \in \{0, 1\}$.

Model usually outputs a **logit** z. Then:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

LOGIT AND PROBABILITY





Binary Cross-Entropy (BCE) Loss.

The **binary cross-entropy loss** for one sample is:

$$\mathcal{L}(y, \hat{y}) = -\Big[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})\Big]$$

Meaning:

- If $y = 1$, loss = $-\log(\hat{y})$ → penalizes if model assigns low probability to class 1.
- If $y = 0$, loss = $-\log(1 - \hat{y})$ → penalizes if model assigns high probability to class 1.

Suppose:

- True label: $y = 1$
- Prediction: $\hat{y} = 0.9$

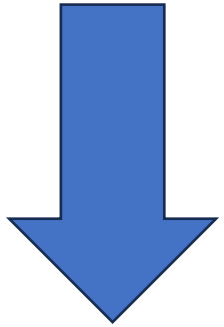
Loss = $-\log(0.9) \approx 0.105$ → small, good prediction.

If $\hat{y} = 0.1$:

Loss = $-\log(0.1) \approx 2.302$ → large, penalizes heavily.

KL Divergence Definition → BCE derivation possible?

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$



$$D_{\mathrm{KL}}(P \parallel Q) = -\left[y \log(\hat{y}) + (1-y) \log(1-\hat{y})\right]$$

Remarks (concluding module 2…)

- Bias-variance is one of the most important aspects of ML engineering
- Optimization is the heart of the learning algorithm
- Loss function sets the objective of learning and guides the optimizer towards better generalization
- Machine learning : Learning parameters using **Dataset**

  Algorithm: **Optimization** (objective function → **loss function**) on the Dataset

  Learning evaluation : Observe **Bias-Variance during the optimization**

Mid-term evaluation topics

Appendix

👉 Information is inversely related to probability:

Which one could be a headline for the newspaper?

"It snowed in Leh Ladakh this winter"

"It snowed in Durgapur today" → extr

Saying *"It snowed in Leh Ladakh this winter"* → very common → **almost no information**

Saying *"It snowed in Durgapur today"* → extremely rare → **huge information value.**

common events = little information, rare events = lots of information.

Probabilities

- **In Leh Ladakh (winter):**

  Snow is common → probability ≈ 0.8.

- **In Durgapur (West Bengal):**

  Snow is extremely rare → probability ≈ 0.0001.

- X → Rainfall in Ladakh
- Y → Winning chance of India in Eden Garden Cricket

$$\boxed{I(x,y) = I(x) + I(y)}$$

The **log function** is the only function (up to a constant scaling) that satisfies the additivity requirement.

- If $p(x, y) = p(x) \cdot p(y)$, then:

$$I(x, y) = f(p(x) \cdot p(y))$$

For additivity, we need:

$$f(p(x) \cdot p(y)) = f(p(x)) + f(p(y))$$

- The only solution for f is:

We use $-\log p(x)$ because it's the only function that naturally makes information positive, decreases with higher probability, and adds up for independent events.

1. Information of a single event

For an event x with probability $p(x)$:

$$I(x) = -\log p(x)$$

Compute Information

- **Leh Ladakh:**

$$I = -\log(0.8) \approx 0.22$$

→ Very little information. Everyone already expects snow there.

- **Durgapur:**

$$I = -\log(0.0001) \approx 9.21$$

→ Massive information! If it snows in Durgapur, it's headline news.

In information theory, this “uncertainty of a distribution” is measured by **entropy**:

$$H(Q) = -\sum_{i=1}^{4} q_i \log q_i$$

- This is the **expected uncertainty** of the student’s prediction over all possible options.
- It quantifies how much “information” is still missing for the student to solve the problem.